数学

Morning Flower and Evening Oath

目录

第一章 高精度计算 **1**

1.1 高精度整数：有符号加减乘与小整数除法 1

第二章 快速幂 **7**

2.1 无符号 64 位模乘与模幂 7

第三章 数论 **8**

3.1 模算术简介 8

3.1.1 动态模整数与合数模逆元 8

3.1.2 静态模整数 12

3.1.3 模运算的适用条件 15

3.1.3.1 除法与解同余式不是一回事 15

3.1.3.2 知道整数可整除时，先升模数再整除 15

3.1.3.3 组合数的整数性不等于分母可逆 15

3.1.3.4 大指数要同时保留余数和阈值信息 15

3.2 素数 16

3.2.1 确定性 64 位 Miller–Rabin 判素 16

3.2.2 64 位判素与质因数分解 17

3.3 欧拉函数 18

3.3.1 单个数的欧拉函数 18

3.3.2 单个数的欧拉函数 18

3.4 筛法 19

3.4.1 区间筛素数 19

3.4.2 线性筛、欧拉函数与 Möbius 函数 20

3.4.3 线性筛：三张表的约定 20

3.4.4 区间筛：只保存目标区间 21

3.5 分解质因数 21

3.5.1 Pollard–Rho 因数分解 21

3.6 裴蜀定理 & 一次不定方程 23

3.6.1 扩展欧几里得与 Bézout 系数 23

3.6.2 一次不定方程：特解与正整数解范围 24

3.6.2.1 有符号矩形范围：把两维约束化成一个参数区间 24

3.7 费马小定理 & 欧拉定理 25

3.7.1 十进制大指数：扩展欧拉降幂 25

3.7.2 Carmichael：最小公共指数 26

3.8 模逆元 26

3.8.1 任意正模数的单个逆元 26

3.8.2 任意数组的批量逆元 27

3.8.3 连续整数的线性逆元表 29

3.8.4 任意数组的批量逆元 29

3.8.5 连续整数的线性逆元表 29

3.9 线性同余方程 30

3.9.1 线性同余方程 30

3.9.2 线性同余：最小解与最小周期 . . . 30
3.10 中国剩余定理 . . . 31
3.10.1 广义 CRT：逐项合并同余 . . . 31
3.10.2 Garner：大周期 CRT 与混合进制 . . . 33
3.10.3 扩展 CRT：逐式合并 . . . 35
3.10.4 Garner：总周期可以超过整数范围 . . . 35
3.11 升幂引理 . . . 36
3.11.1 素因子估值与升幂：先约去公共因子 . . . 36
3.11.1.1 基本运算与公共因子 . . . 36
3.11.1.2 奇素数：差式与和式的条件不同 . . . 36
3.11.1.3 素数 2：先分指数奇偶 . . . 36
3.11.1.4 把大整数整除转成指数条件 . . . 36
3.12 阶乘取模 . . . 37
3.12.1 Legendre 赋值与 Kummer 进位判据 . . . 37
3.12.1.1 组合数：逐位相加时进了几次位 . . . 37
3.12.1.2 素数幂为零与 Lucas 的边界 . . . 37
3.12.2 阶乘表：组合数与排列数 . . . 37
3.13 卢卡斯定理 . . . 38
3.13.1 Lucas：小素数模的大下标组合数 . . . 38
3.13.2 合数模数组合数（扩展 Lucas） . . . 39
3.13.3 Lucas：按素数进制逐位计算 . . . 42
3.13.4 合数模数组合数与插值 . . . 42
3.14 二次剩余 . . . 43
3.14.1 素数模平方根（Cipolla） . . . 43
3.14.2 素数模平方根 . . . 45
3.15 阶 & 原根 . . . 45
3.15.1 乘法阶与原根 . . . 45
3.15.2 乘法阶与原根 . . . 48
3.15.3 自动准备模数分解与原根 . . . 48
3.16 离散对数 . . . 49
3.16.1 扩展 BSGS . . . 49
3.17 高次剩余 & 单位根 . . . 51
3.17.1 素数模高次剩余：全部根的紧凑表示 . . . 51
3.17.2 素数模高次剩余与单位根 . . . 51
3.17.3 素数幂模高次剩余与根的提升 . . . 52
3.17.4 一般合数模高次剩余：CRT 组合 . . . 52
3.18 数论分块 . . . 52
3.18.1 整除分块：前缀和来源决定算法 . . . 52
3.18.1.1 按数据与资源选前缀和 . . . 52
3.19 狄利克雷卷积 . . . 53
3.19.1 幂的约数和：不除以 $p-1$ . . . 53
3.20 莫比乌斯反演 . . . 53
3.20.1 整除与反演：先辨认求和方向 . . . 53
3.20.1.1 约数和与倍数和不能混用 . . . 53
3.20.1.2 GCD 权值先反演，再交换求和 . . . 53
3.20.2 GCD 计数：缩放、边界与对称性 . . . 53
3.20.2.1 恰好等于 k：先缩放整除对象，再做矩形容斥 . . . 53

3.20.2.2 去掉顺序之前，先检查交换是否仍在定义域内 . . . 53
3.20.3 矩形 GCD 计数：缩放后容斥 . . . 54
3.21 狄利克雷双曲线法 & 杜教筛 . . . 54
3.21.1 杜教筛 . . . 54
3.21.2 杜教筛：预筛与缓存都不可省略 . . . 55
3.22 Min_25 筛 . . . 56
3.22.1 Min_25 积性函数求和 . . . 56
3.23 类欧几里德算法 . . . 60
3.23.1 有符号整除和：类欧几里德 . . . 60
3.23.2 整除和、带权和与平方和 . . . 62
3.23.3 类欧几里德：有符号整除求和 . . . 64
3.23.4 类欧几里德的三个求和量 . . . 65

第四章 多项式与生成函数 66
4.1 快速傅里叶变换 . . . 66
4.1.1 复数 FFT（可复用单位根） . . . 66
4.1.2 拆系数 FFT 任意模卷积 . . . 68
4.1.3 CDQ 分治卷积递推 . . . 71
4.1.4 浮点 FFT：舍入前先约束最坏误差 . . . 72
4.2 快速数论变换 . . . 73
4.2.1 可指定素数与原根的 NTT 卷积 . . . 73
4.2.2 精确有符号整数卷积 . . . 76
4.2.3 参数化 NTT 与模数适配 . . . 77
4.3 快速沃尔什变换 . . . 78
4.3.1 按位 OR、AND、XOR 卷积 . . . 78
4.3.2 不相交子集卷积 . . . 80
4.3.3 集合变换与不相交子集卷积 . . . 81
4.3.4 集合变换：逆变换对模数的要求 . . . 82
4.4 Chirp Z 变换 . . . 82
4.4.1 Chirp Z：等比点求值 . . . 82
4.5 多项式牛顿迭代 . . . 84
4.5.1 形式幂级数求逆 . . . 84
4.5.2 多项式带余除法 . . . 86
4.5.3 多项式带余除法：先反转再求逆 . . . 87
4.6 多项式多点求值 | 快速插值 . . . 88
4.6.1 任意点多项式多点求值 . . . 88
4.6.2 任意点多项式快速插值 . . . 91
4.7 多项式初等函数 . . . 93
4.7.1 形式幂级数开根（含前导零与无解） . . . 93
4.7.2 形式幂级数微积分、对数与指数 . . . 96
4.7.3 形式幂级数快速幂（十进制大指数） . . . 98
4.7.4 级数开根：前导零和自由尾项 . . . 100
4.7.5 ln 与 exp：只处理所需前缀 . . . 100
4.7.6 幂函数的大指数：保留三份指数信息 . . . 100
4.8 常系数齐次线性递推 . . . 101
4.8.1 特征多项式求线性递推第 k 项 . . . 101
4.8.2 快速递推与有理生成函数单项 . . . 102
4.8.3 线性递推：系数方向与外推条件 . . . 104

4.8.4 Bostan–Mori 与快速递推单项 . . . 105
4.9 多项式平移 | 连续点值平移 . . . 106
4.9.1 多项式系数平移 . . . 106
4.9.2 Taylor 平移与等比点求值 . . . 107
4.10 符号化方法 . . . 108
4.10.1 生成函数建模到模板：三个可核验例子 . . . 108
4.10.1.1 例一：不同盒子、相同球、每盒有上下界（OGF 乘积） . . . 108
4.10.1.2 例二：步长 1、2 的有序序列（OGF 求逆） . . . 108
4.10.1.3 例三：不同元素分成单点与无序对（EGF 指数） . . . 108
4.11 Lagrange 反演 . . . 109
4.11.1 隐式生成函数与 Lagrange 反演 . . . 109
4.11.1.1 适用条件与取系数公式 . . . 109
4.11.1.2 模型一按内部结点计数的满有序多叉树 . . . 109
4.11.1.3 模型二带标号有根树的 EGF . . . 109
4.11.1.4 落到现有接口及模数边界 . . . 109
4.12 普通生成函数 . . . 110
4.12.1 生成函数与递推 . . . 110

第五章 组合数学 111
5.1 排列组合 . . . 111
5.1.1 阶乘组合数 . . . 111
5.1.2 计数建模：先确认哪些对象被区分 . . . 112
5.1.2.1 OGF、EGF 的选择来自乘法，而非题目用词 . . . 112
5.1.3 球盒问题：八种情形 . . . 112
5.1.4 组合数选型：先看模数，再看下标与预算 . . . 113
5.2 容斥原理 . . . 114
5.2.1 容斥与二项式反演：恰好与至少 . . . 114
5.2.1.1 从交集和恢复恰好数量 . . . 114
5.2.1.2 至少数量与变换方向 . . . 114
5.2.1.3 例：有标号球放入有标号盒 . . . 114
5.2.1.4 直接使用现有组合数与模整数 . . . 114
5.2.2 组合计数 . . . 115
5.3 错位排列 . . . 115
5.3.1 错位排列 . . . 115
5.4 卡特兰数 . . . 115
5.4.1 Catalan 与投票问题：把前缀限制变成反射双射 . . . 115
5.4.1.1 弱前缀条件：减去首次走到负一的坏路径 . . . 115
5.4.1.2 严格领先：先取走必需的第一步 . . . 115
5.4.1.3 初始有余额：反射起点，而非只改终点 . . . 116
5.4.1.4 取模时优先使用差式，除法条件不能省 . . . 116
5.5 斯特林数与斯特林反演 . . . 117
5.5.1 定义与性质 . . . 117
5.5.1.1 行列快速求解：推导与选择 . . . 117
5.5.2 无符号第一类斯特林数：同一行 . . . 118
5.5.3 无符号第一类斯特林数：同一列 . . . 120
5.5.4 第二类斯特林数：同一行前缀 . . . 121
5.5.5 第二类斯特林数：同一列 . . . 123
5.5.6 斯特林反演与应用 . . . 125

5.6 贝尔数 . . . 126
5.6.1 贝尔数 . . . 126
5.7 伯努利数 . . . 126
5.7.1 伯努利数批量计算 . . . 126
5.7.2 伯努利数与自然数幂和 . . . 126
5.8 Eulerian Number . . . 127
5.8.1 欧拉数（Eulerian number） . . . 127
5.9 分拆数 . . . 128
5.9.1 整数分拆与重数上限 . . . 128
5.9.2 至多 m 个部分的分拆表 . . . 129
5.9.3 至多 m 个部分的分拆表（生成函数） . . . 130
5.9.4 整数分拆：重数限制的差一问题 . . . 131
5.9.4.1 三个容易混淆的限制 . . . 131
5.9.4.2 重数限制与禁止倍数部分 . . . 131
5.10 范德蒙德卷积 . . . 131
5.10.1 范德蒙德卷积及常用变形 . . . 131
5.11 Pólya 计数 . . . 132
5.11.1 Burnside 平均：任意模数下的精确除法 . . . 132
5.11.2 置换的循环长度分解 . . . 133
5.11.3 Pólya 环染色：旋转与翻转 . . . 134
5.11.4 群作用计数：先确定什么算同一个方案 . . . 135
5.11.5 项链与手链：从置换循环推公式 . . . 135
5.11.6 指定颜色库存：给每个循环分配整份颜色 . . . 135
5.11.7 群阶不可逆：先提升模数，再作整数除法 . . . 136

第六章 线性代数 137
6.1 矩阵 . . . 137
6.1.1 模矩阵乘法与幂 . . . 137
6.1.2 素数模矩阵求逆 . . . 139
6.1.3 Max-plus 矩阵幂 . . . 141
6.1.4 线性代数与生成树 . . . 141
6.2 行列式 . . . 142
6.2.1 任意正模数行列式：欧几里得消元 . . . 142
6.2.2 任意正模数行列式 . . . 143
6.3 线性基 . . . 144
6.3.1 64 位异或线性基 . . . 144
6.3.2 带位置限制的异或线性基 . . . 146
6.3.3 异或线性空间求交 . . . 149
6.3.4 Zassenhaus：同时求和空间与交空间 . . . 151
6.4 特征多项式 . . . 153
6.4.1 有限状态计数到线性递推：先证明阶数，再用 BM . . . 153
6.4.1.1 先把状态、方向和输出写清楚 . . . 153
6.4.1.2 Cayley–Hamilton 给出阶数至多 d . . . 153
6.4.1.3 为什么前 2d 项足以恢复这个标量序列 . . . 153
6.4.1.4 模型：不含连续子串 101 的二进制串 . . . 153
6.4.1.5 按目标选算法，并把采样成本算进去 . . . 153

第七章 线性规划 **155**
7.1 单纯形法 . . . 155
7.1.1 线性规划：两阶段单纯形 . . . 155

第八章 概率论 **161**
8.1 随机变量的数字特征 . . . 161
8.1.1 期望选型：计数拆指标，等待拆尾和 . . . 161
8.1.2 条件期望：先定义状态，再解自环 . . . 161
8.1.3 模概率：可逆的不只是输入分母 . . . 161

第九章 博弈论 **162**
9.1 公平组合游戏 . . . 162
9.1.1 SG 的前提、递推与异或策略 . . . 162
9.1.2 减法游戏小例与三类失效边界 . . . 162
9.1.3 巴什博弈 . . . 162
9.1.4 Nim 游戏 . . . 162
9.1.5 威佐夫博弈 . . . 162
9.1.6 斐波那契博弈 . . . 163
9.1.7 阶梯博弈 . . . 163
9.1.8 反 Nim（取走最后一颗者负） . . . 163
9.1.9 Nim-k（每次至多动 k 堆） . . . 163
9.1.10 树上删边（Green Hackenbush） . . . 163
9.1.11 二分图博弈（不重复走点） . . . 164
9.1.12 对称策略：环形取石子 . . . 164
9.1.13 策略窃取：Chomp . . . 164

第十章 数值算法 **165**
10.1 插值 . . . 165
10.1.1 拉格朗日插值与连续点优化 . . . 165
10.1.2 横坐标连续的 O(n) 拉格朗日插值 . . . 168
10.2 数值积分 . . . 169
10.2.1 有工作上限的自适应 Simpson 积分 . . . 169
10.2.2 数值积分：局部估计、误差预算与采样盲区 . . . 172
10.2.3 数值算法与几何 . . . 172
10.3 高斯消元 . . . 172
10.3.1 压位模 2 高斯消元与全部解 . . . 172
10.3.2 素数模高斯消元与全部解 . . . 174
10.3.3 实数高斯消元与数值解空间 . . . 177
10.3.4 模线性方程组：工厂工期 . . . 179
10.3.5 模 2 压位消元与完整解空间 . . . 180
10.3.6 模 2 消元后的最少开关用法 . . . 180

第十一章 Berlekamp–Massey 算法 **181**
11.1 Berlekamp–Massey：最短线性递推 . . . 181

第一部分 跨类依赖 **183**

附录 A 树状数组 **184**
A.1 树状数组第 k 小与组合依赖 . . . 184

附录 B 最短路问题 **186**
B.1 最短路 186
B.1.1 Floyd 全源最短路与原边方案 186

附录 C 图的匹配 **189**
C.1 二分图最大匹配 189
C.1.1 Hopcroft–Karp 最大匹配与最小点覆盖 189
C.2 一般图最大匹配 192
C.2.1 一般图最大基数匹配：带花树 192

附录 D 矩阵树定理 **196**
D.1 任意正模数带权矩阵树 196

第一章　高精度计算

1.1　高精度整数：有符号加减乘与小整数除法

功能：有符号任意精度整数：可由 `long long` 或十进制字符串构造并转回字符串，支持比较、取负、加、减、乘，除以一个 `int` 得商和余数，以及非负整数次幂。

核心思路：绝对值按 10^9 进制、低位在前存于 `d`，符号单独存于 `sign`；零为空数组且符号为正，运算后用 `trim` 去掉高位零保持规范。同号加法逐节进位；异号加法化为绝对值大者减小者，结果取绝对值较大者的符号；减法即加相反数。乘法是逐节竖式，用 `long long` 携带进位。除以小整数从高节往低节做长除法，商向零截断，余数随被除数的符号。幂用二进制快速幂。

思路中的情况	代码中的处理	行
由字符串构造	从末尾起每 9 位数字一组转成一节	30
转为字符串	最高节原样输出，其余节补足 9 位前导零	51
异号相加	若本数绝对值较小则交换两数，再逐节借位相减	81
同号相加	逐节相加并进位	100
乘法	第 i 节乘以 b 的每一节，累加到第 $i+j$ 节并进位	126
除以小整数	高节到低节长除法，`rem` 为当前余数	145

接口契约：BigInteger(x) 接收 long long，BigInteger(s) 接收可带正负号及前导零的非空十进制串；str() 返回规范化十进制，cmp(b) 返回 $-1/0/1$。支持一元负号、加减乘，divmod(int b) 返回商和余数，b≠0；商向零截断，余数与被除数同号或为 0。pow(unsigned e) 计算非负整数幂，约定 0 的 0 次方为 1。

d 是低位在前的 10^9 进制绝对值，sign 为 ±1；零为空 d 且 sign=1，勿直接修改。n,m 为分节数：加减比较 O(n+m)，乘法 O(nm+n+m)，小整数除法 O(n)，结果空间 O(n+m)；幂执行 O(log(e+1)) 次乘法，位数增长另计。字符串长度、所有中间分节数及两操作数长度之和须小于 INT_MAX 并留进位空间。不含大整数除大整数；不用浮点或 Boost。

```
struct BigInteger
{
    static constexpr int base = 1000000000;
    vector<int> d;
    int sign = 1;
    void trim()
    {
        while (!d.empty() && d.back() == 0)
            d.pop_back();
        if (d.empty())
            sign = 1;
    }
    BigInteger(long long x = 0)
    {
        sign = x < 0 ? -1 : 1;
        auto v = x < 0 ? 0ULL - (unsigned long long)x : (unsigned long long)x;
        while (v)
        {
            d.push_back(v % base);
            v /= base;
        }
        trim();
    }
```

十进制构造、输出与比较（接上页结构体）：

```
    explicit BigInteger(const string &s)
    {
        assert(!s.empty());
        int first = s[0] == '-' || s[0] == '+';
        sign = s[0] == '-' ? -1 : 1;
        assert(first < (int)s.size());
        for (int r = int(s.size()); r > first; r -= 9)
        {
            int v = 0;
            for (int i = max(first, r - 9); i < r; i++)
            {
                assert('0' <= s[i] && s[i] <= '9');
                v = v * 10 + s[i] - '0';
            }
            d.push_back(v);
        }
        trim();
    }
    string str() const
    {
        if (d.empty())
            return "0";
        string s = sign < 0 ? "-" : "";
        s += to_string(d.back());
        for (int i = int(d.size()) - 2; i >= 0; i--)
        {
            string t = to_string(d[i]);
            s += string(9 - t.size(), '0') + t;
        }
        return s;
    }
    int abs_cmp(const BigInteger &b) const
    {
        if (d.size() != b.d.size())
            return d.size() < b.d.size() ? -1 : 1;
        for (int i = int(d.size()) - 1; i >= 0; i--)
            if (d[i] != b.d[i])
                return d[i] < b.d[i] ? -1 : 1;
        return 0;
    }
    int cmp(const BigInteger &b) const
    {
        if (sign != b.sign)
            return sign < b.sign ? -1 : 1;
        return sign * abs_cmp(b);
    }
```

取负与加减法（接上页结构体）：

```
    BigInteger operator-() const
    {
        BigInteger c = *this;
        if (!c.d.empty())
            c.sign = -c.sign;
        return c;
    }
    BigInteger operator+(const BigInteger &b) const
    {
        if (sign != b.sign)
        {
            if (abs_cmp(b) < 0)
                return b + *this;
            BigInteger c = *this;
            int borrow = 0;
            for (int i = 0; i < (int)d.size(); i++)
            {
                int v = d[i] - borrow;
                if (i < (int)b.d.size())
                    v -= b.d[i];
                borrow = v < 0;
                c.d[i] = v + borrow * base;
            }
            c.trim();
            return c;
        }
        BigInteger c;
        c.sign = sign;
        int carry = 0;
        int n = max(d.size(), b.d.size());
        for (int i = 0; i < n || carry; i++)
        {
            int v = carry;
            if (i < (int)d.size())
                v += d[i];
            if (i < (int)b.d.size())
                v += b.d[i];
            c.d.push_back(v % base);
            carry = v / base;
        }
        return c;
    }
    BigInteger operator-(const BigInteger &b) const
    {
        return *this + (-b);
    }
```

乘法、小整数除法与整数幂（接上页结构体）：

```
    BigInteger operator*(const BigInteger &b) const
    {
        BigInteger c;
        c.sign = sign * b.sign;
        c.d.resize(d.size() + b.d.size());
        for (int i = 0; i < (int)d.size(); i++)
        {
            long long carry = 0;
            for (int j = 0; j < (int)b.d.size(); j++)
            {
                long long v = 1LL * d[i] * b.d[j] + c.d[i + j] + carry;
                c.d[i + j] = v % base;
                carry = v / base;
            }
            c.d[i + b.d.size()] = carry;
        }
        c.trim();
        return c;
    }
    pair<BigInteger, int> divmod(int b) const
    {
        assert(b != 0);
        long long v = b < 0 ? -1LL * b : b;
        long long rem = 0;
        BigInteger q;
        q.sign = sign * (b < 0 ? -1 : 1);
        q.d.resize(d.size());
        for (int i = int(d.size()) - 1; i >= 0; i--)
        {
            rem = rem * base + d[i];
            q.d[i] = rem / v;
            rem %= v;
        }
        q.trim();
        return {q, int(sign * rem)};
    }
    BigInteger pow(unsigned int e) const
    {
        BigInteger a = *this;
        BigInteger r = 1;
        while (e)
        {
            if (e & 1)
                r = r * a;
            e >>= 1;
            if (e)
                a = a * a;
        }
        return r;
    }
};
```

模板题使用：P1480 高精度除法 I

题意：0≤a≤10^5000，1≤b≤10^9；输出整数商。divmod 同时返回余数，这里只打印 first。完整协议本地通过，在线状态见验证记录。

所需模板：BigInteger（第 1 页，1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P1480

```
int main()
{
    string s;
    int b;
    cin >> s >> b;
    BigInteger a(s);
    cout << a.divmod(b).first.str() << '\n';
}
```

第二章　快速幂

2.1　无符号 64 位模乘与模幂

功能： 64 位无符号整数的模乘 `mul` 与快速幂 `power`，供判素、分解等数论模板使用。

核心思路： `mul` 把乘数提升到 `__uint128_t` 再取模，乘积不会溢出且结果精确；`power` 是普通二进制快速幂，从低位到高位扫描指数，底数逐次平方。

接口契约： Mod64::mul(a,b,m) 与 power(a,e,m)，a、b、e 支持完整 unsigned long long，m 为非零无符号 64 位模数。乘积用 uint128 后取模，不通过浮点近似；m=1 返回 0，e=0 返回 1 mod m。mul 为 O(1)，power 为 O(log(e+1)) 次模乘、O(1) 空间。它也是 64 位判素的底层组件。

```
struct Mod64
{
    using ull = unsigned long long;
    using u128 = __uint128_t;

    static ull mul(ull a, ull b, ull m) { return u128(a) * b % m; }

    static ull power(ull a, ull b, ull m)
    {
        assert(m);
        ull r = 1 % m;
        a %= m;
        for (; b; b >>= 1, a = mul(a, a, m))
            if (b & 1) r = mul(r, a, m);
        return r;
    }
};
```

第三章　数论

3.1　模算术简介

3.1.1　动态模整数与合数模逆元

功能：运行时设定模数的取模整数，模数不要求是素数。支持加、减、乘、快速幂；与模数互素时可求逆、做除法，不可逆时 try_inv 返回空。不同 tag 各自持有独立的模数。

核心思路：成员 v 始终是 $[0,p)$ 内的代表，加减只做一次条件校正。乘法用 Barrett 约减代替 64 位除法：set_mod 预存 im$\approx\lceil 2^{64}/p\rceil$，对乘积 z 用 $\lfloor z\cdot \mathrm{im}/2^{64}\rfloor$ 估计商 q，q 至多偏大 1，因此 $z-qp$ 为负时补一个 p。求逆对 (v,p) 做迭代扩展欧几里得并维护 v 的系数，最后余数为 1 才可逆，系数即逆元。

接口契约：mint<tag> 按 tag 独立保存运行时模数，默认 998244353。先 set_mod(p)，1≤p≤INT_MAX，再构造该模数下的所有对象；更换同 tag 的模数后，旧对象、容器元素和缓存全部失效，必须重建，不能跨模混算。构造支持 long long 全范围，v 为规范余数；加减乘及复合赋值、相等比较为 O(1)，加减只作一次区间校正，乘法使用 Barrett32，不依赖素数。pow(k) 的 k 为 uint64_t 非负指数，0 次幂为 1 mod p。try_inv() 以扩展欧几里得求逆，gcd(v,p)≠1 返回 nullopt；inv() 与除法要求可逆，否则违反断言前提。p=1 时唯一余数为 0，乘法单位元也为 0，try_inv 返回 0。幂和求逆为 O(log p) 或 O(log k)，除法包含求逆。可与 Fenwick 区间和组合，不能用于模数和的 kth。mod、reducer 只读，必须通过 set_mod 同步切换；Barrett32 本身支持完整 uint32，但此包装仍受 int 上界保护；不支持超过 INT_MAX 的模数，也不提供线程间安全切换模数。

```
template <int tag = 0> struct mint
{
    static inline int mod = 998244353;
    static inline unsigned long long im = -1ULL / 998244353 + 1; // Barrett reciprocal
    int v;

    static void set_mod(int p)
    {
        assert(p >= 1);
        mod = p;
        im = -1ULL / p + 1;
    }

    mint(long long x = 0) : v(x % mod)
    {
        if (v < 0)
            v += mod;
    }

    mint operator+(mint b) const
    {
        mint r;
        long long sum = (long long)v + b.v;
        r.v = sum >= mod ? sum - mod : sum;
        return r;
    }
```

模整数（接上页同一结构体）：

```
    mint operator-(mint b) const
    {
        mint r;
        r.v = v - b.v;
        if (r.v < 0)
            r.v += mod;
        return r;
    }

    mint operator*(mint b) const
    {
        mint r;
        unsigned long long z = 1ULL * v * b.v, y = (unsigned long long)((__uint128_t(z) * im) >> 64) * mod;
        r.v = int(z - y + (z < y ? mod : 0));
        return r;
    }

    mint &operator+=(mint b)
    {
        return *this = *this + b;
    }

    mint &operator-=(mint b)
    {
        return *this = *this - b;
    }

    mint &operator*=(mint b)
    {
        return *this = *this * b;
    }

    mint pow(uint64_t k) const
    {
        mint a = *this, r = 1;
        while (k)
        {
            if (k & 1)
                r *= a;
            a *= a;
            k >>= 1;
        }
        return r;
    }
```

模整数（接上页同一结构体）：

```
    optional<mint> try_inv() const
    {
        long long a = v, b = mod, x = 1, y = 0;
        while (b)
        {
            long long q = a / b;
            tie(a, b) = pair{b, a - q * b};
            tie(x, y) = pair{y, x - q * y};
        }
        if (a != 1)
            return nullopt;
        return mint(x);
    }

    mint inv() const
    {
        auto x = try_inv();
        assert(x.has_value());
        return *x;
    }

    mint operator/(mint b) const
    {
        return *this * b.inv();
    }

    mint &operator/=(mint b)
    {
        return *this = *this / b;
    }

    bool operator==(const mint &) const = default;
};
```

模板题使用：Luogu P5431 【模板】模意义下的乘法逆元 2

题意：输入 1≤n≤5000000、2≤k<p≤10^9、1≤a_i<p 且 p 为素数，求 ∑k^i/a_i 模 p。先设置 mint 的模数 p，再构造数组；求前缀积后只对总积求一次逆（try_inv），倒序乘出各项逆元。权重从 1 开始，每项先乘 k 再累加。read 与 main 一起抄。更换同 tag 模数后旧对象失效，计算期间不能切换。核心支持合数单位，本题只覆盖素数；动态模数、构造、乘法及累加、v 输出在此直接使用，合数求逆接口另由核心测试核验。

所需模板：mint（第 8 页，3.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5431

```
int read()
{
    int c = getchar_unlocked();
    while (c < '0' || c > '9')
    {
        if (c == EOF) return 0;
        c = getchar_unlocked();
    }
    int value = 0;
    while ('0' <= c && c <= '9')
    {
        value = value * 10 + c - '0';
        c = getchar_unlocked();
    }
    return value;
}

int main()
{
    int n = read(), p = read(), k = read();
    mint<>::set_mod(p);
    vector<mint<>> a(n);
    for (auto &x : a) x = read();
    // Batch inverse: one inversion of the prefix product.
    vector<mint<>> prefix(n + 1, 1);
    for (int i = 0; i < n; i++) prefix[i + 1] = prefix[i] * a[i];
    auto total = prefix[n].try_inv();
    if (!total) return 1;
    mint<> suffix = *total, answer = 0;
    vector<mint<>> inverse(n);
    for (int i = n - 1; i >= 0; i--)
    {
        inverse[i] = suffix * prefix[i];
        suffix *= a[i];
    }
    mint<> weight = 1;
    for (auto x : inverse)
    {
        weight *= mint<>(k);
        answer += weight * x;
    }
    printf("%d\n", answer.v);
    return 0;
}
```

3.1.2 静态模整数

功能：编译期固定模数的取模整数（模数可为合数），支持加、减、乘、快速幂；与模数互素时可求逆、做除法，不可逆时 `try_inv` 返回空。

核心思路：成员 `v` 始终保存 $[0, \mathrm{mod})$ 内的规范代表，构造时把负余数调正。加减各做一次条件校正，乘法用 `long long` 乘积取模，快速幂按二进制拆分指数。求逆不依赖费马小定理：对 (v, mod) 做迭代扩展欧几里得并维护 v 的系数 `x`，结束时 gcd 为 1 则该系数就是逆元。

接口契约：固定 int 模数 1≤mod≤INT_MAX，可为合数。构造接受 signed64 并正规化，v 保持 0..mod-1；不要手动破坏正规化。加减乘 O(1)，pow 非负 signed64 指数、时间 O(log(e+1))。try_inv 用欧几里得求逆：gcd(v,mod)=1 时返回 optional 值，否则 nullopt；mod=1 返回代表 0。inv、/、/= 要求分母可逆；断言开启时诊断违反前提，关闭断言不承诺拒绝非法除法。逆元 O(log(mod+1))、额外空间 O(1)，只用 long long，不新增辅助模板依赖。+=、-=、*=、/= 返回自身引用，支持自赋值与链式运算，可作为 Fenwick 区间和值类型。类型的扩展不放宽 Binomial、NTT、高斯或 FPS 自身的素数等前提。

```
template <int mod> struct ModInt
{
    static_assert(mod >= 1);
    int v;

    ModInt(long long x = 0)
    {
        v = x % mod;
        if (v < 0)
            v += mod;
    }

    ModInt operator+(ModInt b) const
    {
        long long x = (long long)v + b.v;
        if (x >= mod)
            x -= mod;
        ModInt r;
        r.v = x;
        return r;
    }

    ModInt operator-(ModInt b) const
    {
        int x = v - b.v;
        if (x < 0)
            x += mod;
        ModInt r;
        r.v = x;
        return r;
    }

    ModInt operator*(ModInt b) const
    {
        ModInt r;
        r.v = 1LL * v * b.v % mod;
        return r;
    }
```

模整数（接上页同一结构体）：

```
    ModInt pow(long long e) const
    {
        assert(e >= 0);
        ModInt a = *this, r = 1;
        while (e)
        {
            if (e & 1)
                r = r * a;
            a = a * a;
            e >>= 1;
        }
        return r;
    }

    optional<ModInt> try_inv() const
    {
        long long a = v, b = mod, x = 1, y = 0;
        while (b)
        {
            long long q = a / b;
            a -= q * b;
            swap(a, b);
            x -= q * y;
            swap(x, y);
        }
        if (a != 1)
            return nullopt;
        return ModInt(x);
    }

    ModInt inv() const
    {
        auto r = try_inv();
        assert(r);
        return *r;
    } // requires an invertible residue
```

模整数（接上页同一结构体）：

```
    ModInt operator/(ModInt b) const
    {
        return *this * b.inv();
    }

    ModInt &operator+=(ModInt b)
    {
        return *this = *this + b;
    }

    ModInt &operator-=(ModInt b)
    {
        return *this = *this - b;
    }

    ModInt &operator*=(ModInt b)
    {
        return *this = *this * b;
    }

    ModInt &operator/=(ModInt b)
    {
        return *this = *this / b;
    }
};
```

3.1.3 模运算的适用条件

模逆元存在当且仅当 $\gcd(a,m)=1$；费马逆元 a^{p-2} 仅适用于素数模数且 $a\not\equiv 0$。$x\equiv a\pmod m$、$x\equiv b\pmod n$ 可合并当且仅当 $\gcd(m,n)\mid(b-a)$，新模数 $\operatorname{lcm}(m,n)$。Lucas 仅限素数模；素数模阶乘表上限须小于模数。

3.1.3.1 除法与解同余式不是一回事

$g=\gcd(a,m)$：$ax+b\equiv 0\pmod m$ 在 $g\nmid b$ 时无解，否则

$$\frac{a}{g}x+\frac{b}{g}\equiv 0\pmod{m/g}$$

有唯一解 x_0，原模下全部解为 $x_0+t(m/g)$，$0\le t<g$。约去 g 时模数也要除以 g。`linear_congruence(a,b,m)` 解 $ax+b\equiv 0$，返回（最小非负解，最小正周期），无解 $(-1,-1)$；$ax\equiv c$ 须先把 c 规范到 $[0,m)$ 再取负传入。见第 30 页。`mod_inverse` 不可逆返回 -1，`ModInt::try_inv` 返回 `nullopt`；`inv` 与除法以可逆为前提。见第 26 页、第 12 页。$m=1$ 时代表与逆元均为 0、周期 1；域上算法（高斯、NTT、FPS、组合数）不推广到模 1。

3.1.3.2 知道整数可整除时，先升模数再整除

已知 $d\mid X$，求 $(X/d)\bmod m$（$\gcd(d,m)$ 任意）：取规范剩余 $r=X\bmod(md)\in[0,md)$，则

$$\frac{X}{d}\equiv\frac{r}{d}\pmod m.$$

须能计算 md 及其下的乘模（必要时多精度）；新模数可能是合数，不能套素数模模板；生成 X 的过程含除法时需重验。应用见第 5.11.7 节（第 136 页）。

3.1.3.3 组合数的整数性不等于分母可逆

`Binomial` 要求素数 p 且 $N<p$；合数模下 $N<m$ 不够。大下标素数模用 Lucas；合数模用素数幂分解、单位部分与 CRT（exLucas）。见第 111 页。插值节点差、级数积分分母、模概率分母都须各自检查可逆。

3.1.3.4 大指数要同时保留余数和阈值信息

$a\ge 0$，$b\ge 0$，$m\ge 1$，$\phi=\varphi(m)$：

$$a^b\equiv a^e\pmod m,\qquad e=\begin{cases}b, & b<\phi,\\ b\bmod\phi+\phi, & b\ge\phi.\end{cases}$$

零次幂为 1，$m=1$ 返回 0；$\gcd(a,m)=1$ 时可直接对 ϕ 取模。十进制大指数逐位维护 $b\bmod\phi$ 与 $\min(b,\phi)$。`euler_power(a,b,m,phi)` 要求传入真实 $\varphi(m)$，基数与模数为 unsigned64（负数先规范化），降幂指数可达 65 位，内部用 128 位。见第 25 页。

3.2 素数

3.2.1 确定性 64 位 Miller–Rabin 判素

问题描述：判断一个 64 位无符号整数 n 是否为素数，结果是确定的而不是概率性的。

核心思路：先用 $2 \sim 37$ 的小素数试除，整除时只有 n 恰为该素数才是素数。否则写 $n-1 = d \cdot 2^s$（d 为奇数），对 7 个固定底数做 Miller–Rabin：$x = a^d$ 为 1 或 $n-1$ 则该底数通过；否则再平方至多 $s-1$ 次，途中出现 $n-1$ 才通过，否则 n 是合数。这组底数在 64 位范围内没有反例；底数是 n 的倍数时跳过。

代码依赖：Mod64（第 7 页，2.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：Prime64::prime(n) 接收完整 unsigned long long，n<2 返回 false。先试除小素数，再使用 2、325、9375、28178、450775、9780504、1795265022 七个底数，结果在 64 位输入范围内确定。继承 Mod64，模乘用 uint128；O(log n) 次模乘，O(1) 空间，不包含因数分解。

```
struct Prime64 : Mod64
{
    static bool prime(ull n)
    {
        if (n < 2) return false;
        for (ull p : {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37})
            if (n % p == 0) return n == p;
        ull d = n - 1;
        int s = 0;
        while (!(d & 1))
        {
            d >>= 1;
            ++s;
        }
        for (ull a :
             {2ULL, 325ULL, 9375ULL, 28178ULL, 450775ULL, 9780504ULL, 1795265022ULL})
        {
            if (a % n == 0) continue;
            ull x = power(a, d, n);
            if (x == 1 || x == n - 1) continue;
            bool ok = false;
            for (int j = 1; j < s; j++)
            {
                x = mul(x, x, n);
                if (x == n - 1)
                {
                    ok = true;
                    break;
                }
            }
            if (!ok) return false;
        }
        return true;
    }
};
```

模板题使用：Library Checker primality_test

题意：逐个判断给定正整数是否为素数，输出 Yes 或 No。此题输入不超过 10^{18}；模板支持的完整范围另见接口约定。

所需模板：Mod64（第 7 页，2.1 节）、Prime64（第 16 页，3.2.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/primality_test

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int q;
    cin >> q;
    while (q--)
    {
        unsigned long long n;
        cin >> n;
        cout << (Prime64::prime(n) ? "Yes" : "No") << '\n';
    }
}
```

3.2.2 64 位判素与质因数分解

Miller–Rabin 用七个固定底数，覆盖无符号 64 位整数：

$$2,\ 325,\ 9375,\ 28178,\ 450775,\ 9780504,\ 1795265022.$$

$n-1=d2^s$，检查 a^d 及平方链；$n<2$ 非素数；底数模 n 为 0 时跳过。模乘用无符号 128 位。

Pollard–Rho 迭代 $f(x)=x^2+c \pmod n$，取 $\gcd(|x-y|,n)$；只接受 $1<d<n$，否则换参数重试，无确定最坏上界。`factor` 返回排序且保留重复的质因子，`factor(1)` 为空，取首尾前先判空。

3.3 欧拉函数

3.3.1 单个数的欧拉函数

问题描述：求单个正整数 n 的欧拉函数 $\varphi(n)$。提供两个入口：试除法，以及借助 `PollardRho` 分解后计算（适合含大质因子的 n）。

核心思路：公式 $\varphi(n)=n\prod_{p\mid n}(1-1/p)$。试除版从 $p=2$ 枚举到 $\sqrt{n}$，遇到质因子就把答案改为 answer/p·($p-1$) 并把 n 中的 p 除尽，最后剩下的 $n>1$ 是唯一的大质因子。分解版对升序质因子列表去重后做同样的乘法。先除后乘，中间值不超过原数。

代码依赖：PollardRho（第 21 页，3.5.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：euler_phi(n) 使用试除，O($\sqrt{n}$) 时间、O(1) 额外空间；euler_phi(n,solver) 复用 PollardRho，时间由随机分解主导，分解结果与递归占 O(log n) 空间。1≤n≤ULLONG_MAX，phi(1)=1。大素数或平衡大半素数应选分解器入口，不要用试除穷扫平方根；批量求 1..N 用 LinearSieve。先除质因子再乘 p-1，结果及中间乘积均不超过原 n。只抄试除函数时不需要 Prime64/PollardRho。

```
unsigned long long euler_phi(unsigned long long n)
{
    assert(n >= 1);
    unsigned long long answer = n;
    for (unsigned long long p = 2; p <= n / p; p++)
        if (n % p == 0)
        {
            answer = answer / p * (p - 1);
            while (n % p == 0) n /= p;
        }
    if (n > 1) answer = answer / n * (n - 1);
    return answer;
}

unsigned long long euler_phi(unsigned long long n, PollardRho &solver)
{
    assert(n >= 1);
    auto factors = solver.factor(n);
    unsigned long long answer = n;
    unsigned long long previous = 0;
    for (auto p : factors)
        if (p != previous)
        {
            answer = answer / p * (p - 1);
            previous = p;
        }
    return answer;
}
```

联合使用示例见第 26 页（Luogu P5091），其中直接调用本模板。

3.3.2 单个数的欧拉函数

实现见第 18 页。对 $n\ge 1$，

$$\varphi(n)=n\prod_{p\mid n}\left(1-\frac{1}{p}\right),$$

按 ans ← (ans/p)($p-1$) 先除后乘，不会溢出。试除入口用 $p\le n/p$，$O(\sqrt{n})$，不适合 64 位大素数；传入 `PollardRho` 对象时先分解再去重更新。$\varphi(1)=1$，0 不在定义域内。

3.4 筛法

3.4.1 区间筛素数

问题描述：给定区间 $[l, r]$，列出其中全部素数；r 可以很大，只要区间长度可承受。

核心思路：区间内的合数必有不超过 $\sqrt{r}$ 的质因子。先用埃氏筛（`composite`）求出 $[2, \lfloor\sqrt{r}\rfloor]$ 的素数，同时对每个素数 p，从 $\max(p^2, \lceil l/p \rceil p)$ 开始，在区间数组 `removed` 上按步长 p 标记倍数（下标偏移 $j - l$）。最后从 $\max(l, 2)$ 起收集未被标记的数。

接口契约：segmented_primes(l,r) 返回闭区间内递增的全部素数，0≤l≤r≤INT_MAX；区间长度须能分配内存。独立函数，每次重建底筛；不依赖预存素数表。令 W=r-l+1，时间 O((√r+W)log log(r+3))，空间 O(√r+W)。倍数起点、递增变量均用 long long，避免 INT_MAX 端点溢出；0 和 1 不进入结果。

```
vector<int> segmented_primes(int l, int r)
{
    assert(0 <= l && l <= r);
    int limit = 0;
    while (1LL * (limit + 1) * (limit + 1) <= r) limit++;
    vector<char> composite(limit + 1);
    vector<char> removed(1LL * r - l + 1);
    for (int p = 2; p <= limit; p++)
        if (!composite[p])
        {
            for (long long j = 1LL * p * p; j <= limit; j += p) composite[j] = true;
            long long start = max(1LL * p * p, (l + 1LL * p - 1) / p * p);
            for (long long j = start; j <= r; j += p) removed[j - l] = true;
        }
    vector<int> primes;
    for (long long x = max(l, 2); x <= r; x++)
        if (!removed[x - l]) primes.push_back(x);
    return primes;
}
```

3.4.2 线性筛、欧拉函数与 Möbius 函数

功能：一次预处理 $0..n$，得到递增素数表 `prime`、最小质因子 `lp`、欧拉函数 `phi` 和 Möbius 函数 `mu`；反复除以 `lp` 可快速分解表内的数。

核心思路：欧拉筛：每个合数 $j = i \cdot p$ 只被它的最小质因子 p 筛一次。外层枚举 i，内层从小到大枚举已有素数 p 并令 `lp[j]`$= p$；一旦 $p \mid i$，更大的素数就不再是 $i \cdot p$ 的最小质因子，立即 `break`。积性函数同步转移：$p \mid i$ 时 $\varphi(ip) = \varphi(i)\,p$、$\mu(ip) = 0$；否则 $\varphi(ip) = \varphi(i)(p-1)$、$\mu(ip) = -\mu(i)$。

接口契约：构造参数 0≤n<INT_MAX，且数组须能分配内存。prime 为递增质数表；lp/phi/mu 为 0..n 数组。x≥2 时 lp[x] 是最小质因子，lp[x]==x 等价于 x 为素数；反复除以 lp[x] 可分解质因数。lp[0]=lp[1]=0，不能直接用 lp[x]==x 判断 0。n≥1 时 phi[1]=mu[1]=1；第 0 项均为占位 0。O(n) 时间和空间，分解单个表内正整数耗时 O(log x)。n=0 不可访问下标 1。

```
struct LinearSieve
{
    vector<int> prime, lp, phi, mu;

    LinearSieve(int n) : lp(n + 1), phi(n + 1), mu(n + 1)
    {
        if (n) phi[1] = mu[1] = 1;
        for (int i = 2; i <= n; i++)
        {
            if (!lp[i])
            {
                lp[i] = i;
                prime.push_back(i);
                phi[i] = i - 1;
                mu[i] = -1;
            }
            for (int p : prime)
            {
                if (p > n / i) break;
                int j = i * p;
                lp[j] = p;
                if (i % p == 0)
                {
                    phi[j] = phi[i] * p;
                    mu[j] = 0;
                    break;
                }
                phi[j] = phi[i] * (p - 1);
                mu[j] = -mu[i];
            }
        }
    }
};
```

3.4.3 线性筛：三张表的约定

`LinearSieve` 构造最小质因子、φ、μ，每个合数只由 lp 生成一次，$O(n)$。若 $p \mid i$，$\varphi(ip) = p\varphi(i)$、$\mu(ip) = 0$；否则为 $(p-1)\varphi(i)$ 与 $-\mu(i)$。判素用 $x \geq 2$ 且 $\mathrm{lp}(x) = x$；反复除 $\mathrm{lp}(x)$ 分解，0 不可分解。要求 $0 \leq n <$ `INT_MAX`，乘积前检查 $p \leq n/i$。相关恒等式：$\sum_{d|x} \varphi(d) = x$，$\sum_{d|x} \mu(d) = [x = 1]$，$\varphi(n) = \sum_{d|n} \mu(d)(n/d)$。

3.4.4 区间筛：只保存目标区间

实现见第 19 页。设 $0 \le L \le R \le$ INT_MAX，筛出不超过 $\lfloor\sqrt{R}\rfloor$ 的素数，对每个 p 从

$$\max\bigl(p^2, \lceil L/p\rceil p\bigr)$$

开始标记区间倍数，输出跳过 0 和 1。令 $W = R - L + 1$，空间 $O(\sqrt{R} + W)$，时间 $O((\sqrt{R} + W)\log\log(R + 3))$；每次调用重建底筛。$p^2$、取整加法与遍历变量用 long long，否则 $R =$ INT_MAX 时溢出。

3.5 分解质因数

3.5.1 Pollard–Rho 因数分解

问题描述：把 64 位正整数 n 分解质因数，返回升序且含重数的质因子列表。

核心思路：split 递归：n 为素数（Prime64）就记为一个因子，否则用 rho 找一个非平凡因子 d，分别递归 d 与 n/d，最后排序。rho 对偶数直接返回 2；否则随机取 c 和起点，迭代 $f(v) = v^2 + c \bmod n$，用 Floyd 判圈（x 走一步、y 走两步），每步计算 $\gcd(|x - y|, n)$。得到 $1 < d < n$ 即返回；得到 n 或轮数用尽，就换一组随机参数重试。

代码依赖：Mod64（第 7 页，2.1 节）、Prime64（第 16 页，3.2.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：factor(n) 返回升序质因子（含重数），1≤n≤ULLONG_MAX。n=1 返回空数组；n≥2 时 size()==1 等价于 n 为素数，否则 front()/back() 分别取最小/最大质因子。依赖 Mod64 与 Prime64。公开使用 factor；rho/split 为内部辅助，rho 只接受大于 1 的合数，不能直接传素数。Las Vegas 随机运行时间，固定默认种子方便复现，可由构造参数更换；无确定性最坏时间保证。每次 rho 尝试最多 200000 轮，失败后换随机参数重试，不将失败当作素数。递归仅沿真因子拆分，深度 O(log n)。

```
struct PollardRho
{
    using ull = unsigned long long;
    using u128 = __uint128_t;
    mt19937_64 rng;

    PollardRho(ull seed = 712367821) : rng(seed) {}

    ull rho(ull n)
    {
        if (n % 2 == 0) return 2;
        for (;;)
        {
            ull c = rng() % (n - 1) + 1, x = rng() % n, y = x, d = 1;
            auto f = [&](ull v)
            {
                return (Mod64::mul(v, v, n) + u128(c)) % n;
            };
            // Retry bounded attempts; Las Vegas: only exact divisors returned.
            for (int i = 0; i < 200000 && d == 1; i++)
            {
                x = f(x);
                y = f(f(y));
                d = gcd(x > y ? x - y : y - x, n);
            }
            if (1 < d && d < n) return d;
        }
    }
```

```
29
30      void split(ull n, vector<ull> &a)
31      {
32          if (n == 1) return;
33          if (Prime64::prime(n))
34          {
35              a.push_back(n);
36              return;
37          }
38          ull d = rho(n);
39          split(d, a);
40          split(n / d, a);
41      }
42
43      vector<ull> factor(ull n)
44      {
45          assert(n >= 1);
46          vector<ull> a;
47          split(n, a);
48          sort(a.begin(), a.end());
49          return a;
50      }
51  };
```

模板题使用：Luogu P4718

题意：逐个判断 n 是否为素数，是则输出 Prime，否则输出最大质因子。题目保证 n≥2，factor 返回升序含重数的质因子，因此可取 back；若用于 n=1，须先处理空结果。

所需模板：Mod64（第 7 页，2.1 节）、Prime64（第 16 页，3.2.1 节）、PollardRho（第 21 页，3.5.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4718

```
1   int main()
2   {
3       int t;
4       scanf("%d", &t);
5       PollardRho solver;
6       while (t--)
7       {
8           unsigned long long n;
9           scanf("%llu", &n);
10          auto factors = solver.factor(n);
11          if (factors.size() == 1)
12              puts("Prime");
13          else
14              printf("%llu\n", factors.back());
15      }
16      return 0;
17  }
```

3.6 裴蜀定理 & 一次不定方程

3.6.1 扩展欧几里得与 Bézout 系数

问题描述：给定非负整数 a, b，求 $g = \gcd(a, b)$ 以及一组整数 x, y 使 $ax + by = g$（Bézout 系数），据此可解一般的一次不定方程。

核心思路：按欧几里得算法递归：$b = 0$ 时取 $(x, y) = (1, 0)$。否则对 $(b, a \bmod b)$ 递归，并把 `y`、`x` 交换位置传入，得到 $bx' + (a \bmod b)y' = g$，此时 `x` 中已是 y'、`y` 中是 x'。由 $a \bmod b = a - \lfloor a/b \rfloor b$ 得 $x = y'$、$y = x' - \lfloor a/b \rfloor y'$，只需再执行 `y -= a / b * x`。

接口契约：extended_gcd(a,b,x,y) 约定 0≤a,b≤2^63，参数和输出系数为 int128，返回非负 gcd 并满足 ax+by=gcd(a,b)。a=b=0 返回 0、x=1、y=0；递归实现，中间乘加为 int128；O(log(min(a,b)+1)) 时间与递归空间。解 ax+by=c：令 g=gcd(a,b)，c 不是 g 的倍数时无解；否则 x0=x · c/g、y0=y · c/g，通解 x=x0+k · b/g、y=y0-k · a/g；最小非负 x 为 x0 对 b/g 取非负余数。乘 c/g 时注意 int128 范围。

```
__int128_t extended_gcd(__int128_t a, __int128_t b, __int128_t &x, __int128_t &y)
{
    using i128 = __int128_t;
    if (!b)
    {
        x = 1;
        y = 0;
        return a;
    }
    i128 d = extended_gcd(b, a % b, y, x);
    y -= a / b * x;
    return d;
}
```

模板题使用：AOJ NTL_1_E Extended Euclid Algorithm

题意：给出正整数 1≤a,b≤10^9，求 ax+by=gcd(a,b)，先最小化 |x|+|y|，仅在并列最小时取 x≤y。直接调用 extended_gcd；该正整数范围内递归所得解已满足最小性，a=b 时返回 (0,1)。系数用 ___int128_t 接收，输出转 long long 安全；不应对所有答案强行要求 x≤y。时间、递归空间均 O(log(min(a,b)+1))。

所需模板：extended_gcd（第 23 页，3.6.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.u-aizu.ac.jp/onlinejudge/description.jsp?id=NTL_1_E

```
int main()
{
    long long a, b;
    scanf("%lld%lld", &a, &b);
    __int128_t x, y;
    extended_gcd(a, b, x, y);
    printf("%lld %lld\n", (long long)x, (long long)y);
    return 0;
}
```

3.6.2 一次不定方程：特解与正整数解范围

底层接口见第 3.6.1 节（第 23 页）。a, b 不全为零时令 $g = \gcd(|a|, |b|) > 0$；当且仅当 $g \mid c$ 有整数解。由 $|a|u + |b|v = g$ 乘 c/g 并恢复符号得特解 $ax_0 + by_0 = c$，全部解为

$$x = x_0 + \frac{b}{g}t, \qquad y = y_0 - \frac{a}{g}t, \qquad t \in \mathbb{Z}.$$

$a = b = 0$ 时 $c = 0$ 全体整数对为解，否则无解。`exgcd` 约定非负系数、上限 2^{63}，返回非负 g；取负、缩放后的特解都用 128 位保存。

正系数 a, b, c 求正整数解：$d_x = b/g, d_y = a/g$，把 x_0 调到 $x_{\min} \in [1, d_x]$，$y_{\max} = (c - ax_{\min})/b$。若 $y_{\max} > 0$，

$$N = \left\lfloor \frac{y_{\max} - 1}{d_y} \right\rfloor + 1, \quad x_{\max} = x_{\min} + (N-1)d_x, \quad y_{\min} = y_{\max} - (N-1)d_y.$$

否则无正解，x, y 的最小正值分别按周期归一化（通常不是同一组解）；正值归一化用 `(v % d + d - 1) % d + 1`。

3.6.2.1 有符号矩形范围：把两维约束化成一个参数区间

求 $L_x \le x \le R_x$、$L_y \le y \le R_y$ 内的解数（a, b, c 可负可零）。空区间答案为 0；$a = b = 0$ 时 $c \neq 0$ 无解，$c = 0$ 答案 $(R_x - L_x + 1)(R_y - L_y + 1)$。否则取特解与带符号步长 $d_x = b/g$、$d_y = -a/g$，对 $z = z_0 + dt$：

$d > 0$	$\lceil (L - z_0)/d \rceil \le t \le \lfloor (R - z_0)/d \rfloor$
$d < 0,\ k = -d > 0$	$\lceil (z_0 - R)/k \rceil \le t \le \lfloor (z_0 - L)/k \rfloor$
$d = 0$	$L \le z_0 \le R$ 时不限制t，否则无解

两区间取交 $[T_l, T_r]$，解数 $\max(0, T_r - T_l + 1)$。floor/ceil 为数学取整：正分母 D、C++ 商余 $q = n/D, r = n\%D$ 时

$$\lfloor n/D \rfloor = q - [r < 0], \qquad \lceil n/D \rceil = q + [r > 0].$$

差值、区间长度可能超出 64 位（全零系数覆盖全部 64 位坐标时答案为 2^{128}）；不能证明范围时用任意精度整数。求特解后只需常数次运算。

3.7 费马小定理 & 欧拉定理

3.7.1 十进制大指数：扩展欧拉降幂

问题描述：给定底数 a、模数 m 及 $\varphi(m)$，以及以十进制字符串给出的超大指数 b，求 $a^b \bmod m$；a 与 m 不必互素。

核心思路：扩展欧拉定理：$b < \varphi(m)$ 时指数保持 b；$b \geq \varphi(m)$ 时 $a^b \equiv a^{(b \bmod \varphi(m))+\varphi(m)} \pmod m$。逐位扫描字符串，同时维护 `rem`$= b \bmod \varphi$ 和饱和值 `cap`$= \min(b, \varphi)$；扫描结束时 `cap` 等于 φ 就说明 $b \geq \varphi$，指数加上 φ。降幂后的指数小于 2φ，再用 `Mod64::mul` 做快速幂。

代码依赖：Mod64（第 7 页，2.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：euler_power(a,b,m,phi) 求 a^b mod m，a、m 为 unsigned long long，m>0；b 为非空十进制数字串，可为 0、可含前导零，约定 0^0=1。调用者必须传入正确的 phi=φ(m)，包括 $\varphi(1)=1$；函数只检查数值范围，不验证 phi 正确性。指数小于 phi 时保持原值，否则使用 b mod phi + phi，非互质底数也适用。相加和十进制更新都用 int128，支持完整 unsigned64 模数。O(|b|+log m) 时间、O(1) 额外空间，不含求 phi 的成本；依赖 Mod64::mul。

```
unsigned long long euler_power(unsigned long long a,
                               string_view b,
                               unsigned long long m,
                               unsigned long long phi)
{
    using U = unsigned long long;
    using I = __uint128_t;
    assert(m && phi && phi <= m && !b.empty());
    U rem = 0, cap = 0;
    for (char c : b)
    {
        assert('0' <= c && c <= '9');
        int d = c - '0';
        rem = (I(rem) * 10 + d) % phi;
        cap = min(I(phi), I(cap) * 10 + d);
    }
    I e = I(rem) + (cap == phi ? I(phi) : 0);
    U ans = 1 % m;
    a %= m;
    for (; e; e >>= 1, a = Mod64::mul(a, a, m))
        if (e & 1) ans = Mod64::mul(ans, a, m);
    return ans;
}
```

模板题使用：Luogu P5091

题意：输入顺序为底数 a、模数 m、指数 b；b 最多 20000001 位，以字符串读入。先用试除法计算 $\varphi(m)$，再做扩展欧拉降幂；题目 m≤10^8，不能把输入顺序写成 a,b,m。

所需模板：Mod64（第 7 页，2.1 节）、Prime64（第 16 页，3.2.1 节）、PollardRho（第 21 页，3.5.1 节）、euler_phi（第 18 页，3.3.1 节）、euler_power（第 25 页，3.7.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5091

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    unsigned long long a, m;
    string b;
    cin >> a >> m >> b;
    auto phi = euler_phi(m);
    cout << euler_power(a, b, m, phi) << '\n';
}
```

3.7.2 Carmichael：最小公共指数

$\lambda(n)$ 为所有可逆元阶的最小公倍数，$\lambda(1)=1$，$\lambda(n) \mid \varphi(n)$。

$$\lambda(n)=\operatorname{lcm}_i \lambda(p_i^{e_i}), \qquad \lambda(p^e)=\begin{cases} 2^{e-2}, & p=2,\ e\ge 3, \\ p^{e-1}(p-1), & \text{其他情形.} \end{cases}$$

仅可逆底数可把指数对 $\lambda(n)$ 取模。合数 n 是 Carmichael 数当且仅当 $\lambda(n) \mid (n-1)$，等价于无平方因子且每个素因子 p 满足 $(p-1) \mid (n-1)$（须排除素数）；Fermat 检验不能代替素性判定。

3.8 模逆元

3.8.1 任意正模数的单个逆元

问题描述：给定整数 a 和正整数 m（可为合数），求 a 模 m 的乘法逆元；$\gcd(a,m)\neq 1$ 时逆元不存在。

核心思路：逆元 x 满足 $ax+my=1$。先把 a 规范到 $[0,m)$，调用 extended_gcd 求 Bézout 系数；gcd 不为 1 则不可逆，否则把系数 x 调整到 $[0,m)$ 返回。不使用费马小定理，因此不要求 m 为素数。

代码依赖：extended_gcd（第 23 页，3.6.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：mod_inverse(a,m) 支持完整 long long a 和正 long long m，将 a 规范化后用 extended_gcd 求逆。不可逆返回-1，可逆返回 0..m-1 内的代表；模数 1 返回 0。允许合数模数，不使用费马小定理。O(log m) 时间与递归空间。

```
long long mod_inverse(long long a, long long m)
{
    using i128 = __int128_t;
    assert(m > 0);
    a %= m;
    if (a < 0) a += m;
    i128 x, y;
    if (extended_gcd(a, m, x, y) != 1) return -1;
    return (x % m + m) % m;
}
```

3.8.2 任意数组的批量逆元

问题描述：给定 n 个整数和模数（可为合数），一次求出它们各自的模逆元；只要有一个不可逆就整体判定失败。

核心思路：前缀积技巧：`prefix[i]` 为前 i 个数之积。只对总积 `prefix[n]` 调用一次 `mod_inverse`（总积不可逆当且仅当某一项不可逆）。再从后往前扫：当前 `inverse` 是前 $i+1$ 项积的逆，乘 `prefix[i]` 得到 a_i^{-1}，乘 a_i 得到前 i 项积的逆。结果原地写回数组。

代码依赖：mod_inverse（第 26 页，3.8.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：batch_inverse(a,mod) 返回与 a 同序的逆元向量。要求 mod 为正 long long，每个输入可为完整 long long；只要某一项与 mod 不互素，整个结果返回 nullopt，不输出部分逆元。空输入返回有效的空向量；模数 1 约定返回全零，与单个逆元接口一致。模数可以是合数。函数按值接收数组并归一化；不再需要原数组时传 move(a) 可避免输入副本。依赖 mod_inverse 和 extended_gcd。仅求一次逆元，时间 O(n+log mod)，空间 O(n)；128 位中间乘法防止溢出。

```
optional<vector<long long>> batch_inverse(vector<long long> a, long long mod)
{
    assert(mod > 0);
    int n = a.size();
    vector<long long> prefix(n + 1, 1 % mod);
    for (int i = 0; i < n; i++)
    {
        a[i] %= mod;
        if (a[i] < 0) a[i] += mod;
        prefix[i + 1] = __int128_t(prefix[i]) * a[i] % mod;
    }
    long long inverse = mod_inverse(prefix[n], mod);
    if (inverse == -1) return nullopt;
    for (int i = n - 1; i >= 0; i--)
    {
        long long value = a[i];
        a[i] = __int128_t(inverse) * prefix[i] % mod;
        inverse = __int128_t(inverse) * value % mod;
    }
    return a;
}
```

模板题使用：Luogu P5431 【模板】模意义下的乘法逆元 2

题意：给定 1≤n≤5000000、2≤k<p≤10^9、1≤a_i<p，p 为素数，输出 ∑(i=1..n)k^i/a_i 模 p。先批量求全部逆元再累乘权重，注意首项权重是 k。完整抄写 read 与 main；read 仅适用于本题的非负十进制输入。move(a) 避免输入副本，此后不再读取 a。本题所有元素可逆；一般模数或零元素须检查 nullopt。乘积在 long long 范围内，批量核心内部用 int128 保护一般模数。

所需模板：extended_gcd（第 23 页，3.6.1 节）、mod_inverse（第 26 页，3.8.1 节）、batch_inverse（第 27 页，3.8.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5431

```
int read()
{
    int c = getchar_unlocked();
    while (c < '0' || c > '9')
    {
        if (c == EOF) return 0;
        c = getchar_unlocked();
    }
    int value = 0;
    while ('0' <= c && c <= '9')
    {
        value = value * 10 + c - '0';
        c = getchar_unlocked();
    }
    return value;
}

int main()
{
    int n = read(), p = read(), k = read();
    vector<long long> a(n);
    for (auto &x : a) x = read();
    auto inverse = batch_inverse(move(a), p);
    if (!inverse) return 1;
    long long answer = 0, weight = 1;
    for (auto x : *inverse)
    {
        weight = weight * k % p;
        answer = (answer + weight * x) % p;
    }
    printf("%lld\n", answer);
    return 0;
}
```

3.8.3 连续整数的线性逆元表

问题描述：给定素数 p 和 $n < p$，线性时间求出 $1, 2, \ldots, n$ 模 p 的逆元表。

核心思路：设 $p = qi + r$，$q = \lfloor p/i \rfloor$，$r = p \bmod i < i$。由 $qi + r \equiv 0 \pmod p$，两边乘 $i^{-1}r^{-1}$ 得 $i^{-1} \equiv -q \cdot r^{-1} \pmod p$。因为 $r < i$，r^{-1} 已经算出，按 i 递增一次递推即可；代码写成 $p - (q \cdot r^{-1} \bmod p)$，使结果落在 $[1, p)$。**接口契约：**inverse_table(n,p) 返回长度 n+1 的 int 向量，位置 i 保存 i 模 p 的逆元，1≤i≤n。位置 0 固定为 0，仅为占位，不表示零有逆元；n=0 返回单个占位项。要求 p 为素数、2≤p≤INT_MAX 且 0≤n<p，不检测素性。时间和空间均 O(n)，中间乘法用 long long；独立可抄，不依赖其他模板。不能将同一递推直接用于合数模数，即使当前 i 与模数互素；一般模数使用扩展欧几里得或任意数组批量逆元。

```
vector<int> inverse_table(int n, int p)
{
    assert(p >= 2 && 0 <= n && n < p);
    vector<int> inverse(n + 1);
    if (n) inverse[1] = 1;
    for (int i = 2; i <= n; i++) inverse[i] = p - 1LL * (p / i) * inverse[p % i] % p;
    return inverse;
}
```

模板题使用：Luogu P3811 【模板】模意义下的乘法逆元

题意：输入 1≤n≤3000000、n<p<20000528，p 保证为素数，输出 1..n 各自的逆元，每项一行。inverse_table 返回 n+1 项，0 是占位且不输出。该递推要求素数，不能照搬到合数；一般数组使用 batch_inverse。

所需模板：inverse_table（第 29 页，3.8.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3811

```
int main()
{
    int n, p;
    scanf("%d%d", &n, &p);
    auto inverse = inverse_table(n, p);
    for (int i = 1; i <= n; i++) printf("%d\n", inverse[i]);
    return 0;
}
```

3.8.4 任意数组的批量逆元

设 $s_0 = 1$，$s_i = \prod_{j<i} a_j \pmod m$，只对 s_n 求一次逆元，再从右向左：若 $t = s_{i+1}^{-1}$，则

$$a_i^{-1} = ts_i \pmod m, \qquad s_i^{-1} = ta_i \pmod m.$$

先保存原 a_i 再覆盖。总乘积与 m 互素当且仅当每项互素，模数无需为素数。实现见第 27 页：无解返回空的可选值，空输入返回空向量，模数 1 返回全零。

3.8.5 连续整数的线性逆元表

素数 p、$1 < i < p$ 时

$$i^{-1} \equiv -\lfloor p/i \rfloor \, (p \bmod i)^{-1} \pmod p,$$

从 $1^{-1} = 1$ 递推，$O(N)$。实现见第 29 页，第 0 项占位，允许 $N = 0$；要求 $N < p$。合数模数不能照搬（$p \bmod i$ 可能不可逆），单个逆元用扩展欧几里得（$\gcd(a, m) = 1$ 才有解）或素数下的 a^{p-2}。

3.9 线性同余方程

3.9.1 线性同余方程

问题描述：求解线性同余方程 $ax + b \equiv 0 \pmod m$：给出最小非负解与解的周期，或判断无解。

核心思路：方程等价于 $ax \equiv -b \pmod m$。规范化 a 后用 `extended_gcd` 求 $g = \gcd(a, m)$ 及满足 $ax_0 \equiv g \pmod m$ 的 x_0。$g \nmid b$ 时无解；否则 $x \equiv x_0 \cdot (-b/g) \pmod{m/g}$，周期为 m/g，取模后调成非负。

代码依赖：extended_gcd（第 23 页，3.6.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：linear_congruence(a,b,m) 求 a*x+b≡0 (mod m)，返回 {最小非负解, 最小正周期}，无解返回 {-1,-1}。m 为正 long long，a、b 支持完整 long long。若 g=gcd(a,m) 整除 b，周期为 m/g；包括 a=0 和 m=1，所有整数均满足时返回 {0,1}。先归一化 a，并在 int128 中对 b 取相反数和乘法。依赖 extended_gcd；递归时间、空间 O(log(m+1))。注意本接口的常数项是 +b，若题目写 a*x≡c，应传 -c；c 为 LLONG_MIN 时须避免在 64 位中取负。

```
pair<long long, long long> linear_congruence(long long a, long long b, long long m)
{
    assert(m > 0);
    using i128 = __int128_t;
    a %= m;
    if (a < 0) a += m;
    i128 x, y;
    i128 g = extended_gcd(a, m, x, y);
    if (i128(b) % g) return {-1, -1};
    i128 period = m / g;
    x = x * (-i128(b) / g) % period;
    if (x < 0) x += period;
    return {(long long)x, (long long)period};
}
```

3.9.2 线性同余：最小解与最小周期

实现见第 30 页，解 $ax + b \equiv 0 \pmod m$。a 归一化后令 $g = \gcd(a, m)$；有解当且仅当 $g \mid b$，若 $au + mv = g$，则 $x_0 = u(-b/g)$，全部解为 $x_0 + (m/g)t$，返回 $[0, m/g)$ 内最小解与最小正周期 m/g。$a = 0$ 且 $m \mid b$ 或 $m = 1$ 时返回 $(0, 1)$。b 可为 `LLONG_MIN`（内部用 `-i128(b)`）；题目为 $ax \equiv c$ 时先把 c 归一化再取负传入。exgcd 见第 24 页。追及问题 $(m - n)t + (x - y) \equiv 0 \pmod L$ 直接传速度差与起点差，不取绝对值。

3.10 中国剩余定理

3.10.1 广义 CRT：逐项合并同余

问题描述：求同时满足若干同余式 $x \equiv r_i \pmod{m_i}$ 的最小非负整数，模数不必两两互素，或判断无解。函数每次把一个新同余式合并进当前解 $x \equiv r \pmod{m}$。

核心思路：合并 $x \equiv r \pmod{m}$ 与 $x \equiv b \pmod{n}$：设 $x = r + mk$，需 $mk \equiv b - r \pmod{n}$。令 $g = \gcd(m, n)$，由 `extended_gcd` 得 $mx_0 + ny_0 = g$；$g \nmid (b - r)$ 时无解，否则 $k \equiv \frac{b-r}{g} x_0 \pmod{n/g}$，取最小非负值。新模数为 $\mathrm{lcm} = m \cdot n/g$，新余数 $r + mk$ 对它取模，原地写回 `r`、`m`。

代码依赖：extended_gcd（第 23 页，3.6.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：crt_merge(r,m,b,n) 合并 x≡r (mod m) 与 x≡b (mod n)，m,n>0，r、m 须为不同变量。从 r=0、m=1 开始逐项合并；成功后 r 为最小非负解、m 为最小正周期。余数支持完整 long long。无解返回 false；有解但新周期超过 LLONG_MAX 抛 overflow_error。两种失败都会先规范化 r，m 不变，失败后不能视为完整解。依赖 extended_gcd，中间量用 int128；O(log min(m,n)) 时间与递归空间。

```
bool crt_merge(long long &r, long long &m, long long b, long long n)
{
    using ll = long long;
    using i128 = __int128_t;
    assert(m > 0 && n > 0);
    r %= m;
    if (r < 0) r += m;
    b %= n;
    if (b < 0) b += n;
    i128 x, y;
    ll g = (ll)extended_gcd(m, n, x, y);
    i128 diff = i128(b) - r;
    if (diff % g) return false;
    i128 q = n / g, k = (diff / g * x % q + q) % q, mod = i128(m) * q;
    if (mod > LLONG_MAX) throw overflow_error("CRT modulus");
    r = (r + i128(m) * k) % mod;
    m = (ll)mod;
    return true;
}
```

模板题使用：Luogu P4777

题意：求同时满足多个同余式的最小非负整数，模数可以不互质。每行按模数、余数输入，逐式合并；合并后模数须在 long long 范围内。

所需模板：extended_gcd（第 23 页，3.6.1 节）、crt_merge（第 31 页，3.10.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4777

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int count;
    cin >> count;
    long long r = 0, modulus = 1;
    while (count--)
    {
        long long m, b;
        cin >> m >> b;
        if (!crt_merge(r, modulus, b, m))
        {
            cout << -1 << '\n';
            return 0;
        }
    }
    cout << r << '\n';
    return 0;
}
```

3.10.2 Garner：大周期 CRT 与混合进制

问题描述：给定两两互素的模数 $m_0, \ldots, m_{k-1}$ 与余数 b_i，求满足全部同余式的最小非负整数 x（可能远超 64 位）对任意目标模数的余数，或求出 x 的混合进制各位。常用于三模数 NTT 的结果合并。

核心思路：把 x 写成混合进制 $x = d_0 + d_1 m_0 + d_2 m_0 m_1 + \cdots$，$0 \le d_i < m_i$，按顺序求各位。`value[j]` 维护已确定各位之和模 m_j，`coefficient[j]` 维护 $m_0 \cdots m_{j-1} \bmod m_j$。第 i 位须满足 $\text{value}_i + \text{coef}_i \cdot d_i \equiv b_i \pmod{m_i}$，故 $d_i = (b_i - \text{value}_i) \cdot \text{coef}_i^{-1} \bmod m_i$，系数不可逆说明模数不互素；随后更新后面各 j 的两个数组。`garner` 再从高位到低位用秦九韶法在目标模数下求出 x。

代码依赖：mod_inverse（第 26 页，3.8.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：garner_digits(b,moduli) 返回按输入顺序排列的混合进制各位；garner(b,moduli,target) 返回最小非负 CRT 解对 target 的余数。两个输入向量等长，长度可为 0；模数为正且两两互素，允许模数 1，余数支持完整 long long 范围。target 为正 long long，与输入模数无需互素。输入不互素时抛 invalid_argument，即使方程有解也不适用；此时用扩展 CRT。空输入的规范解取 0。算法从不构造模数乘积，中间运算使用 int128。k 个模数耗时 $O(k^2+k \log M)$，空间 O(k)；M 为最大输入模数。依赖 mod_inverse 和 extended_gcd。

```cpp
// Pairwise coprime positive moduli. Digits use the input order.
vector<long long> garner_digits(const vector<long long> &b,
                                const vector<long long> &moduli)
{
    using ll = long long;
    using i128 = __int128_t;
    int n = b.size();
    assert(moduli.size() == b.size());
    vector<ll> value(n), coefficient(n, 1), digit(n);
    for (ll m : moduli) assert(m > 0);
    for (int i = 0; i < n; i++)
    {
        ll m = moduli[i];
        ll inverse = mod_inverse(coefficient[i], m);
        if (inverse == -1) throw invalid_argument("Garner moduli are not coprime");
        i128 delta = (i128(b[i]) - value[i]) % m;
        if (delta < 0) delta += m;
        digit[i] = delta * inverse % m;
        for (int j = i + 1; j < n; j++)
        {
            value[j] = (value[j] + i128(coefficient[j]) * digit[i]) % moduli[j];
            coefficient[j] = i128(coefficient[j]) * m % moduli[j];
        }
    }
    return digit;
}
```

规范解对目标数取模

```
// Return the least nonnegative CRT solution modulo target, not all solutions.
long long
garner(const vector<long long> &b, const vector<long long> &moduli, long long target)
{
    assert(target > 0);
    auto digit = garner_digits(b, moduli);
    long long answer = 0;
    for (int i = (int)digit.size() - 1; i >= 0; i--)
        answer = (__int128_t(answer) * moduli[i] + digit[i]) % target;
    return answer;
}
```

模板题使用：Library Checker Convolution (Mod 1,000,000,007)

题意：给出长度 n、m 的升幂系数，输出模 1000000007 的卷积，共 n+m−1 项。两数组长度均为 1 到 524288；输入系数非负且小于目标模数。分别在 167772161、469762049、1224736769 下做 NTT，再逐项调用既有 garner。真实非负系数不超过 524288 乘以 1000000006 的平方，严格小于三素数乘积。因此 CRT 最小非负代表就是原整数系数，最后对目标模数取余；仅有模数互素并不足够。输入通过 ModInt 构造器归一化。三模数公共变换容量为 2^24，本题至多需 2^20；无系数界时不能直接套用。

所需模板：extended_gcd（第 23 页，3.6.1 节）、mod_inverse（第 26 页，3.8.1 节）、garner（第 33 页，3.10.2 节）、ModInt（第 12 页，3.1.2 节）、NttConvolution（第 73 页，4.2.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/convolution_mod_1000000007

```
template <int p>
auto conv(const vector<int> &a, const vector<int> &b)
{
    using N = NttConvolution<p>;
    typename N::Poly x(a.begin(), a.end()), y(b.begin(), b.end());
    return N::multiply(move(x), move(y));
}

int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    vector<int> a(n), b(m);
    for (auto &x : a) cin >> x;
    for (auto &x : b) cin >> x;
    auto x = conv<167772161>(a, b);
    auto y = conv<469762049>(a, b);
    auto z = conv<1224736769>(a, b);
    vector<long long> r(3), p = {167772161, 469762049, 1224736769};
    for (int i = 0; i < n + m - 1; i++)
    {
        r[0] = x[i].v;
        r[1] = y[i].v;
        r[2] = z[i].v;
        cout << garner(r, p, 1000000007) << (i + 1 == n + m - 1 ? '\n' : ' ');
    }
```

```
}
```

3.10.3 扩展 CRT：逐式合并

实现见第 31 页的 `crt_merge`：从 $r = 0, m = 1$ 起依次合并 $x \equiv b \pmod{n}$，一旦无解即停止。设 $g = \gcd(m, n)$，$mu + nv = g$；若 $g \nmid (b - r)$ 无解，否则

$$q = n/g, \qquad t \equiv \frac{b-r}{g} u \pmod{q}, \qquad 0 \le t < q,$$

新状态为 $((r + mt) \bmod mq,\ mq)$。模数不必互素。要求正模数且最终周期不超过有符号 64 位；中间量用 128 位。无解返回假，周期越界抛异常。时间 $O(k \log M)$，递归栈 $O(\log M)$。

3.10.4 Garner：总周期可以超过整数范围

模数 m_i 两两互素，$P = \prod m_i$，把解写成混合进制

$$x = d_0 + d_1 m_0 + d_2 m_0 m_1 + \cdots, \qquad 0 \le d_i < m_i.$$

对每个后续模数维护已定部分余数 v_j 与位权 $c_j = \prod_{t<i} m_t \bmod m_j$，$d_i = (b_i - v_i) c_i^{-1} \bmod m_i$，然后 $v_j \leftarrow v_j + c_j d_i$，$c_j \leftarrow c_j m_i$。所得 $x \in [0, P)$ 为最小非负解；模数 1 可直接参与。实现见第 33 页。求 $x \bmod T$ 时从高位 Horner，T 无需与 m_i 互素；结果只是规范解的余数，多模 NTT 还原整数时须保证真实值在 $[0, P)$ 内。

3.11 升幂引理

3.11.1 素因子估值与升幂：先约去公共因子

p 为素数，a,b,n 为正整数，$v_p(x)$ 为非零整数 x 中 p 的重数。$v_p(0)$ **无定义**：差式先判 $a=b$。

3.11.1.1 基本运算与公共因子

$v_p(xy)=v_p(x)+v_p(y)$，$v_p(x^n)=nv_p(x)$；$v_p(x+y)\ge\min(v_p(x),v_p(y))$，估值不同才取等。记 $\alpha=v_p(a)$、$\beta=v_p(b)$：若 $\alpha\ne\beta$，$v_p(a^n\pm b^n)=n\min(\alpha,\beta)$；若 $\alpha=\beta=t$，写 $a=p^tA,b=p^tB$，则 $v_p(a^n\pm b^n)=nt+v_p(A^n\pm B^n)$，再对 A,B 用 LTE。

3.11.1.2 奇素数：差式与和式的条件不同

p 为奇素数且 $p\nmid ab$：

$$\begin{aligned} p\mid a-b,\ a\ne b \quad&\Longrightarrow\quad v_p(a^n-b^n)=v_p(a-b)+v_p(n),\\ p\mid a+b,\ n\text{ 为奇数} \quad&\Longrightarrow\quad v_p(a^n+b^n)=v_p(a+b)+v_p(n).\end{aligned}$$

差式不要求 n 奇，和式必须 n 奇；不要求 $\gcd(a,b)=1$。

3.11.1.3 素数 2：先分指数奇偶

a,b 均为奇数，差式要求 $a\ne b$：

$$v_2(a^n-b^n)=\begin{cases}v_2(a-b), & n\text{ 奇},\\ v_2(a-b)+v_2(a+b)+v_2(n)-1, & n\text{ 偶},\end{cases}\qquad v_2(a^n+b^n)=\begin{cases}v_2(a+b), & n\text{ 奇},\\ 1, & n\text{ 偶}.\end{cases}$$

若 $4\mid a-b$，差式可写 $v_2(a-b)+v_2(n)$。奇偶不同则和差均奇；都偶先约去公共 2 因子。

3.11.1.4 把大整数整除转成指数条件

$M=\prod p_i^{e_i}\mid X$（$X\ne0$）等价于逐项 $v_{p_i}(X)\ge e_i$，无需构造大幂。LTE 只处理满足条件的局部因子，不能代替乘法阶或离散对数。$a+b$、$|a-b|$、nt 须防溢出。阶乘估值用 $v_p(N!)=\sum_{j\ge1}\lfloor N/p^j\rfloor$（ExLucas，第 39 页），不套 LTE；素数幂开根还须检查单位部分有根，估值条件不充分。

3.12 阶乘取模

3.12.1 Legendre 赋值与 Kummer 进位判据

p 为素数，$s_p(n)$ 为 p 进制数位和，$n \geq 0$：

$$v_p(n!) = \sum_{j \geq 1} \left\lfloor \frac{n}{p^j} \right\rfloor = \frac{n - s_p(n)}{p-1}.$$

反复 $n \leftarrow \lfloor n/p \rfloor$ 累加，$O(1+\log_p(n+1))$，不要计算 p^j（会溢出）。`ExLucas::valuation(n,p)` 见第 39 页：n 为 unsigned64，p 为正 int 素数，不验证素性。

3.12.1.1 组合数：逐位相加时进了几次位

$0 \leq k \leq n$，$h = n - k$：

$$v_p\binom{n}{k} = \frac{s_p(k) + s_p(h) - s_p(n)}{p-1}$$

等于 p 进制下 $k + h$ 的进位次数（含传播进位），即 Kummer 定理。

3.12.1.2 素数幂为零与 Lucas 的边界

$\binom{n}{k} \equiv 0 \pmod{p^a} \iff v_p\binom{n}{k} \geq a$；合数 m 须每个素数幂分量都满足，模 1 恒为 0。零进位 $\iff$ 每位 $k_i \leq n_i$，即 Lucas 非零（第 38 页）。赋值只判整除，不给出非零余数。$k > n$ 先返回 0。LTE 选型见第 3.11.1 节（第 36 页）。

3.12.2 阶乘表：组合数与排列数

素数 p，$0 \leq k \leq n < p$：

$$C(n,k) = n!\,(k!)^{-1}\,((n-k)!)^{-1}, \qquad P(n,k) = n!\,((n-k)!)^{-1} \pmod{p}.$$

扩表到 N：正向补阶乘，只对 $N!$ 求一次逆，再 $(i-1)!^{-1} = i\,(i!)^{-1}$ 反向补。查询前显式扩表，查询 $O(1)$；扩表后旧元素引用失效。见第 111 页。

3.13 卢卡斯定理

3.13.1 Lucas：小素数模的大下标组合数

问题描述：给定较小的素数 p 和很大的 n, k，求 $\binom{n}{k} \bmod p$。

核心思路：Lucas 定理：把 n, k 写成 p 进制，$\binom{n}{k} \equiv \prod_i \binom{n_i}{k_i} \pmod p$，某一位 $k_i > n_i$ 时整体为 0。构造时预处理 $0..p-1$ 的阶乘 `fac`，用费马小定理求 $(p-1)!$ 的逆，再倒推逆阶乘 `ifac`。`choose` 每轮取最低位 $a = n \bmod p$、$b = k \bmod p$，乘上 $\binom{a}{b}$，再把 n, k 除以 p。

接口契约：Lucas(p) 预处理素数模数 p，choose(n,k) 返回 C(n,k) 模 p；n、k 为完整 unsigned long long 范围，k>n 返回 0，C(0,0)=1。重新赋值 Lucas(p) 可更换模数并重新预处理；不检测素性，要求调用者保证 p 为素数且 p≥2。预处理时间 O(p+log p)，动态版空间 O(p)；单次查询 O(1+log_p(n+1))。适合小素数模数，通常 $p \le 10^6$；大素数不能因查询下标小就忽略这里整段 O(p) 的预处理。fac、ifac 下标为 0..p-1，模数和表只读；中间乘法用 long long。合数模数请使用 exLucas。

```
struct Lucas
{
    int mod;
    vector<int> fac, ifac;

    Lucas(int p)
    {
        assert(p >= 2);
        mod = p;
        fac.resize(p);
        ifac.resize(p);
        fac[0] = 1;
        for (int i = 1; i < p; i++) fac[i] = 1LL * fac[i - 1] * i % p;
        long long a = fac[p - 1], r = 1;
        for (int e = p - 2; e; e >>= 1)
        {
            if (e & 1) r = r * a % p;
            a = a * a % p;
        }
        ifac[p - 1] = r;
        for (int i = p - 1; i; i--) ifac[i - 1] = 1LL * ifac[i] * i % p;
    }
```

```
    int choose(unsigned long long n, unsigned long long k) const
    {
        if (k > n) return 0;
        long long answer = 1;
        while (n || k)
        {
            int a = n % mod, b = k % mod;
            if (b > a) return 0;
            answer = answer * fac[a] % mod;
            answer = answer * ifac[b] % mod;
            answer = answer * ifac[a - b] % mod;
            n /= mod;
            k /= mod;
        }
        return answer;
    }
};
```

模板题使用：Luogu P3807

题意：多组查询 C(n+m,n) 模 p，题目保证 p 为素数且不超过 10^5。每组构造 Lucas(p)，choose 的参数为 n+m、n；此题范围内 n+m 不会溢出。

所需模板：Lucas（第 38 页，3.13.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3807

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t;
    cin >> t;
    while (t--)
    {
        unsigned long long n, m;
        int p;
        cin >> n >> m >> p;
        Lucas solver(p);
        cout << solver.choose(n + m, n) << '\n';
    }
    return 0;
}
```

3.13.2 合数模数组合数（扩展 Lucas）

问题描述：给定任意正整数模数 M（可为合数，规模约 10^6）和很大的 n, k，求 $\binom{n}{k} \bmod M$。

核心思路：把 M 分解为互素的素数幂 $q = p^e$，分别求 $\binom{n}{k} \bmod q$ 后用 CRT 合并。模 q 时把 $n!$ 写成 $p^{v(n)}u(n)$，$u(n)$ 与 p 互素：v 用 Legendre 公式；$n!$ 中不被 p 整除的因子模 q 以 q 为周期，故 $u(n) = (\text{整段积})^{\lfloor n/q \rfloor} \cdot (\text{零头积}) \cdot u(\lfloor n/p \rfloor)$。于是 $\binom{n}{k} \equiv p^v \, u(n)\, u(k)^{-1} u(n-k)^{-1} \pmod q$，其中 $v = v(n) - v(k) - v(n-k)$，$v \geq e$ 时该分量为 0。

思路中的情况	代码中的处理	行
分解模数	试除得到每个素数幂 $q = p^e$	41
段内前缀积	`fac` 中该因子的一段：$0..q-1$ 内不被 p 整除的数之积，末尾多存一份整段积	50
CRT 系数	$c_q = (M/q) \cdot ((M/q)^{-1} \bmod q)$，模 q 为 1、模其他素数幂为 0	53
$n!$ 中 p 的指数	Legendre 公式 $\sum_{i\geq 1}\lfloor n/p^i \rfloor$	63
去掉 p 后的 $n!$	整段积的 $\lfloor n/q \rfloor$ 次幂乘零头，再对 $\lfloor n/p \rfloor$ 迭代	73
p 的指数不少于 e	该分量为 0，跳过	87
合并分量	分母单位部分用欧拉定理求逆，乘 p^v 后按 CRT 系数累加	92

接口契约： ExLucas(mod) 预处理固定模数，choose(n,k) 返回 C(n,k) 模 mod；n、k 为 unsigned long long，k>n 返回 0，mod=1 返回 0。适合 mod≤10^6 的合数或素数模数，不能用于 10^9 级素数的整段预处理。分解 mod 为互素素数幂 q=p^e，预处理每段内不含 p 因子的前缀积；unit(n,f) 去除 n! 中全部 p 因子，valuation 统计指数。分子分母指数之差不少于 e 时该项为零，否则只对互素单位部分求逆，最后 CRT 合并。预处理 O(√mod+∑q)，空间 O(∑(q+1))；每次查询 O(∑(log_p(n+1) · log(n+1)+log q))，求和遍历不同素因子。所有乘法用 long long；同一对象可重复查询，factors、fac 与辅助函数视为内部数据。

```
struct ExLucas
{
    using ull = unsigned long long;
    using ll = long long;

    struct Factor
    {
        int p, q, exponent, start, crt;
    };

    int mod;
    vector<int> fac;
    vector<Factor> factors;

    static ll power(ll a, ull b, int m)
    {
        ll result = 1;
        while (b)
        {
            if (b & 1) result = result * a % m;
            a = a * a % m;
            b >>= 1;
        }
        return result;
    }

    ExLucas(int m) : mod(m)
    {
        assert(m >= 1);
        int x = m;
        for (int p = 2; p <= x / p; p++)
        {
            if (x % p) continue;
            int q = 1, e = 0;
            do
            {
                x /= p;
                q *= p;
```

```
            e++;
        } while (x % p == 0);
        add_factor(p, q, e);
    }
    if (x > 1) add_factor(x, x, 1);
}

void add_factor(int p, int q, int e)
{
    int start = fac.size();
    fac.push_back(1);
    for (int i = 1; i < q; i++) fac.push_back(ll(fac.back()) * (i % p ? i : 1) % q);
    fac.push_back(fac.back());
    int phi = q - q / p;
    int crt = ll(mod / q) * power(mod / q % q, phi - 1, q) % mod;
    factors.push_back({p, q, e, start, crt});
}

static ull valuation(ull n, int p)
{
    ull result = 0;
    while (n)
    {
        n /= p;
        result += n;
    }
    return result;
}

ll unit(ull n, const Factor &f) const
{
    ll result = 1;
    while (n)
    {
        result = result * power(fac[f.start + f.q], n / f.q, f.q) % f.q;
        result = result * fac[f.start + n % f.q] % f.q;
        n /= f.p;
    }
    return result;
}
```

```
    int choose(ull n, ull k) const
    {
        if (k > n || mod == 1) return 0;
        ll answer = 0;
        for (const auto &f : factors)
        {
            ull e = valuation(n, f.p) - valuation(k, f.p) - valuation(n - k, f.p);
            if (e >= ull(f.exponent)) continue;
            ll denominator = unit(k, f) * unit(n - k, f) % f.q;
            ll residue =
                unit(n, f) * power(denominator, f.q - f.q / f.p - 1, f.q) % f.q;
            residue = residue * power(f.p, e, f.q) % f.q;
            answer = (answer + residue * f.crt) % mod;
        }
        return answer;
    }
};
```

模板题使用：Luogu P4720

题意：求 C(n,m) 模 p，p 可以为合数且不超过 10^6。构造 ExLucas(p) 后 choose(n,m)；预处理与模数规模有关，不能直接用于 10^9 级模数。

所需模板：ExLucas（第 39 页，3.13.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4720

```
int main()
{
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    unsigned long long n, m;
    int p;
    std::cin >> n >> m >> p;
    ExLucas comb(p);
    std::cout << comb.choose(n, m) << '\n';
}
```

3.13.3 Lucas：按素数进制逐位计算

p 为素数，n_i, k_i 为 p 进制位：$\binom{n}{k} \equiv \prod_i \binom{n_i}{k_i} \pmod p$，某位 $k_i > n_i$ 则为零。只预处理 $0!, \dots, (p-1)!$ 及逆元；每位常数次乘法。实现见第 38 页，参数为 n, k；自行求和时检查 64 位溢出。

3.13.4 合数模数组合数与插值

$q = p^e$，$v_p(n!) = \sum_{j \ge 1} \lfloor n/p^j \rfloor$，$U_p(n) = n!/p^{v_p(n!)}$，$t = v_p(n!) - v_p(k!) - v_p((n-k)!)$。$t \ge e$ 时为零，否则

$$\binom{n}{k} \equiv U_p(n)\, U_p(k)^{-1}\, U_p(n-k)^{-1}\, p^t \pmod q,$$

各素数幂结果 CRT 合并，见第 39 页。

素数域中 n 个两两不同的 x_i，$w_i = y_i / \prod_{j \ne i}(x_i - x_j)$：

$$f(k) = \sum_{i=0}^{n-1} w_i \prod_{j<i}(k - x_j) \prod_{j>i}(k - x_j),$$

查询点可等于样本点。$x_i = i$ 时分母为 $i!\,(n-1-i)!\,(-1)^{n-1-i}$，要求 $n \le p$。见第 165 页。$S_d(k) = \sum_{i \le k} i^d$ 在 $0 \le d \le p-2$ 时取 $0, \ldots, d+1$ 共 $d+2$ 个样本。

3.14 二次剩余

3.14.1 素数模平方根（Cipolla）

问题描述：给定素数 p 与整数 n，求方程 $x^2 \equiv n \pmod p$ 的全部解。

核心思路：$n \equiv 0$ 或 $p = 2$ 时唯一解就是 n。否则用 Euler 判别法，$n^{(p-1)/2} \neq 1$ 时无解。$p \equiv 3 \pmod 4$ 时 $n^{(p+1)/4}$ 直接是一个根。其余情况用 Cipolla：随机找 a 使 $w = a^2 - n$ 为非二次剩余，在扩域 $\mathbb{F}_p[\sqrt{w}]$ 中计算 $(a + \sqrt{w})^{(p+1)/2}$，元素用（实部，虚部）表示，乘法中 $(\sqrt{w})^2 = w$，结果的实部就是一个根。另一个根为 $p - x$，排序后一并返回。

思路中的情况	代码中的处理	行
$n \equiv 0$ 或 $p = 2$	只有一个根 n	6
Euler 判别	n 不是二次剩余则返回空	18
$p \equiv 3 \pmod 4$	根为 $n^{(p+1)/4}$	21
找非剩余 w	随机 a 直到 $w^{(p-1)/2} = p - 1$	30
扩域乘法	$(x_1 + y_1\sqrt{w})(x_2 + y_2\sqrt{w})$ 的实部与虚部	32
两个根	x 与 $p - x$ 按大小排序返回	48

接口契约：mod_sqrt(n,p) 返回 vector<long long>，按升序列出全部不同根；无解返回空向量，零或模数 2 返回单根。要求 p 为素数，2≤p≤INT_MAX，n 支持完整 long long 范围并在入口归一化。先用 Euler 判别；p≡3 (mod 4) 时直接快速幂，其余使用 Cipolla 二次扩域。函数内保存固定种子的伪随机生成器；随机性影响寻找非剩余的尝试次数，不影响结果，通常期望 O(log p) 时间、O(1) 辅助空间。乘法用 long long，扩域实部的三因子乘积中间先取模；指数中的 p+1 先转 long long，避免 INT_MAX 溢出。此版本不处理合数模数及 64 位大素数，也不用于浮点开方。

```
vector<long long> mod_sqrt(long long n, int p)
{
    using ll = long long;
    assert(p >= 2);
    n = (n % p + p) % p;
    if (n == 0 || p == 2) return {n};
    auto power = [p](ll a, ll b)
    {
        ll result = 1;
        while (b)
        {
            if (b & 1) result = result * a % p;
            a = a * a % p;
            b >>= 1;
        }
        return result;
    };
    if (power(n, (p - 1) / 2) != 1) return {};
    ll answer;
    if (p % 4 == 3)
        answer = power(n, (ll(p) + 1) / 4);
    else
    {
        static mt19937_64 rng(712367);
        ll a, w;
```

```
        do
        {
            a = rng() % p;
            w = (a * a % p - n + p) % p;
        } while (power(w, (p - 1) / 2) != p - 1);
        using P = pair<ll, ll>;
        auto mul = [p, w](P x, P y)
        {
            ll real = (x.first * y.first + x.second * y.second % p * w) % p;
            ll imag = (x.first * y.second + x.second * y.first) % p;
            return P{real, imag};
        };
        P result{1, 0}, base{a, 1};
        ll exponent = (ll(p) + 1) / 2;
        while (exponent)
        {
            if (exponent & 1) result = mul(result, base);
            base = mul(base, base);
            exponent >>= 1;
        }
        answer = result.first;
    }
    ll other = p - answer;
    if (answer > other) swap(answer, other);
    return {answer, other};
}
```

模板题使用：Luogu P5491

题意：多组求 x*x 与 N 模 p 同余的全部解，题目保证 p 为奇素数。mod_sqrt 返回升序且去重的根，空数组表示无解并输出 Hola!。

所需模板：mod_sqrt（第 43 页，3.14.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5491

```
int main()
{
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    int tests;
    std::cin >> tests;
    while (tests--)
    {
        long long n;
        int p;
        std::cin >> n >> p;
        auto roots = mod_sqrt(n, p);
        if (roots.empty())
            std::cout << "Hola!\n";
        else
        {
            for (int i = 0; i < (int)roots.size(); i++)
                std::cout << roots[i] << (i + 1 == (int)roots.size() ? '\n' : ' ');
        }
    }
}
```

3.14.2 素数模平方根

奇素数 p、$n \not\equiv 0$：有平方根当且仅当 $n^{(p-1)/2} \equiv 1 \pmod p$。$p \equiv 3 \pmod 4$ 时 $r = n^{(p+1)/4}$，另一根 $p - r$。Cipolla：取 a 使 $w = a^2 - n$ 非平方，$\omega^2 = w$，根为 $(a+\omega)^{(p+1)/2}$。实现见第 43 页；零和模 2 单独处理；合数模不适用。

3.15 阶 & 原根

3.15.1 乘法阶与原根

问题描述：给定模数 n，求剩余类 a 的乘法阶，判断 a 是否为原根，求最小原根，或升序列出全部原根。

核心思路：构造时试除分解求 $\varphi(n)$ 及其不同素因子 `factors`，并按“n 为 $2, 4, p^k, 2p^k$（p 为奇素数）”判断原根是否存在。阶：从 $\varphi(n)$ 出发，对每个素因子 p，只要 $a^{\text{result}/p} \equiv 1$ 就把 `result` 除以 p。原根判定：a 与 n 互素，且对每个 p 都有 $a^{\varphi/p} \not\equiv 1$。最小原根从 1 起逐个试；全部原根是 g^k（$\gcd(k, \varphi) = 1$），按 k 累乘枚举后排序。

代码依赖：Mod64（第 7 页，2.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：构造 PrimitiveRoot(n)，要求 2≤n≤INT_MAX；重新赋值 PrimitiveRoot(n) 可重建。order(a) 返回 a 模 n 的乘法阶，a 不可逆时返回 0；is_root(a) 判断该剩余类是否为原根，a 可为完整 long long。minimum() 返回 1..n-1 内最小原根，无原根返回 -1；all() 返回升序的全部原根，无原根返回空向量。mod、phi、exists 只读；factors 为 phi 的不同素因子，辅助 factor 为试除分解。依赖 Mod64::power。初始化 O($\sqrt{n}$)，求阶 O($\log^2 n$)，原根判定 O((1+ω(phi))log n)。若最小原根为 G，顺序枚举最坏检查 G 个候选，不能假定 G 总是常数。枚举全部另需 O(phi log phi+c log c) 时间、O(c) 空间，c 为原根数；大模数只求最小原根或阶时，不应调用 all。

```
struct PrimitiveRoot
```

```
{
    int mod, phi;
    bool exists;
    vector<int> factors;

    static vector<int> factor(int n)
    {
        vector<int> result;
        for (int p = 2; p <= n / p; p++)
            if (n % p == 0)
            {
                result.push_back(p);
                while (n % p == 0) n /= p;
            }
        if (n > 1) result.push_back(n);
        return result;
    }
```

```
    PrimitiveRoot(int n)
    {
        assert(n >= 2);
        mod = n;
        phi = n;
        for (int p : factor(n)) phi -= phi / p;
        int odd = n;
        if (odd % 2 == 0) odd /= 2;
        exists = n == 2 || n == 4 || (odd % 2 && factor(odd).size() == 1);
        factors = factor(phi);
    }

    int order(long long a) const
    {
        a %= mod;
        if (a < 0) a += mod;
        if (gcd(a, (long long)mod) != 1) return 0;
        int result = phi;
        for (int p : factors)
            while (result % p == 0 && Mod64::power(a, result / p, mod) == 1)
                result /= p;
        return result;
    }
```

```
    bool is_root(long long a) const
    {
        if (!exists) return false;
        a %= mod;
        if (a < 0) a += mod;
        if (gcd(a, (long long)mod) != 1) return false;
        for (int p : factors)
            if (Mod64::power(a, phi / p, mod) == 1) return false;
        return true;
    }

    int minimum() const
    {
        if (!exists) return -1;
        for (int g = 1; g < mod; g++)
            if (is_root(g)) return g;
        return -1;
    }
```

```
    vector<int> all() const
    {
        int g = minimum();
        vector<int> roots;
        if (g == -1) return roots;
        long long value = 1;
        for (int k = 1; k <= phi; k++)
        {
            value = value * g % mod;
            if (gcd(k, phi) == 1) roots.push_back(value);
        }
        sort(roots.begin(), roots.end());
        return roots;
    }
};
```

模板题使用：Luogu P6091

题意：多组枚举模 n 的全部原根，先输出个数，再输出升序原根的第 d、2d、3d 等项。all 返回升序数组，下标从 d-1 开始；即使没有取到任何原根，第二行也必须输出空行。

所需模板：Mod64（第 7 页，2.1 节）、PrimitiveRoot（第 45 页，3.15.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P6091

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t;
    cin >> t;
    while (t--)
    {
        int n, d;
        cin >> n >> d;
        auto roots = PrimitiveRoot(n).all();
        cout << roots.size() << '\n';
        bool first = true;
        for (int i = d - 1; i < (int)roots.size(); i += d)
        {
            if (!first) cout << ' ';
            cout << roots[i];
            first = false;
        }
        cout << '\n';
    }
    return 0;
}
```

3.15.2 乘法阶与原根

$\gcd(a,n)=1$ 时阶 $d \mid \varphi(n)$：从 $d=\varphi(n)$ 起，对每个素因子 q，$a^{d/q}\equiv 1$ 就除去 q。不可逆 a 接口返回 0。a 为原根当且仅当 $\gcd(a,n)=1$ 且对 $\varphi(n)$ 每个素因子 q 有 $a^{\varphi(n)/q}\not\equiv 1$。$n\ge 2$ 时原根仅存在于 $2,4,p^k,2p^k$（p 奇素数）。$\operatorname{ord}(g^t)=\varphi(n)/\gcd(t,\varphi(n))$，全部原根为 $\gcd(t,\varphi(n))=1$ 的 g^t，共 $\varphi(\varphi(n))$ 个，按大小输出需排序。实现见第 45 页。$\operatorname{ord}(a^k)=d/\gcd(d,k)$；两阶互素时乘积之阶为两阶之积。

3.15.3 自动准备模数分解与原根

奇素数幂 $m=p^e$ 有原根，设 $\varphi=m/p\cdot(p-1)$，候选 g 为原根当且仅当

$$p\nmid g,\qquad g^{\varphi/q}\not\equiv 1 \pmod m \quad \text{对每个不同素因子} q\mid\varphi.$$

须检查模 p^e 的阶，不能只取模 p 的原根。准备成本含 M 与各 φ 的分解及逐个候选的模幂；同一模数多个方程应复用分解结果。

3.16 离散对数

3.16.1 扩展 BSGS

问题描述：给定 a, b, m（a 与 m 不必互质），求最小的非负整数 x 使 $a^x \equiv b \pmod m$，无解返回 -1。

核心思路：把方程看成 $k \cdot a^{x-\text{offset}} \equiv b$ 逐步化简：只要 $g = \gcd(a, m) > 1$，先检查当前指数 `offset` 是否已是解（即 $k \equiv b$），否则要求 $g \mid b$，把 b 和模数同除以 g，系数 k 乘上 a/g，偏移加一。a 与 m 互质后求 $a^y \equiv bk^{-1}$：baby step 表存 a^j（$0 \le j < s$，同值保留最小 j），giant step 从目标值出发每次乘 a^{-s}，第 i 步命中 a^j 即得 $y = is + j$。i 递增、j 取最小，首个命中就是最小解。

思路中的情况	代码中的处理	行
平凡情形	$m = 1$ 或 $b = 1$ 时 $x = 0$；底数为 1 而 $b \neq 1$ 时无解	11
$\gcd(a, m) > 1$	循环剥离公因子	14
较小指数恰为解	$k \equiv b$ 时返回当前 `offset`	16
化为互质问题	目标值乘 k^{-1}	23
baby step	记录 a^j 的最小指数	30
giant step 命中	返回 offset $+ is + j$	38

代码依赖：Mod64（第 7 页，2.1 节）、mod_inverse（第 26 页，3.8.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：依赖 number_theory.hpp 的求逆与快速幂。solve(a,b,m) 返回最小 x≥0 使 a^x≡b (mod m)，无解返回 -1；1≤m≤10^{12}，a、b 可为完整 long long 范围，入口归一化，约定 0^0=1。先剥离 gcd 并逐次检查较小指数，再在互质剩余问题上 BSGS。baby 表对同余幂保留最小指数 j，按递增大步 i 检查区间，因此首个命中即最小解。单次期望时间 O(√m+log m)，空间 O(√m)，哈希表先 reserve；基于 unordered_map 的界为期望界。每次调用独立建表；底数 1 及零指数解直接返回。

```
struct DiscreteLog
{
    using ll = long long;

    // 1<=m<=1e12; Expected O(sqrt(m)) time, O(sqrt(m)) storage. Smallest x>=0, or -1.
    static ll solve(ll a, ll b, ll m)
    {
        assert(m >= 1 && m <= 1000000000000LL);
        a = (a % m + m) % m;
        b = (b % m + m) % m;
        if (m == 1 || b == 1) return 0;
        if (a == 1) return -1;
        ll offset = 0, k = 1;
        for (ll g; (g = gcd(a, m)) > 1;)
        {
            if (b == k) return offset;
            if (b % g) return -1;
            b /= g;
            m /= g;
            k = (__int128)k * (a / g) % m;
            ++offset;
        }
        ll target = (__int128)b * mod_inverse(k, m) % m;
        ll step = sqrtl(m) + 1;
        unordered_map<ll, ll> baby;
        baby.reserve(step);
        ll cur = 1 % m;
        for (ll j = 0; j < step; j++)
```

```
        {
            if (!baby.count(cur)) baby[cur] = j;
            cur = (__int128)cur * a % m;
        }
        ll inv = mod_inverse(Mod64::power(a, step, m), m);
        cur = target;
        for (ll i = 0; i <= step; i++)
        {
            auto it = baby.find(cur);
            if (it != baby.end()) return offset + i * step + it->second;
            cur = (__int128)cur * inv % m;
        }
        return -1;
    }
};
```

模板题使用：Luogu P4195

题意：多组求 a^x 与 b 模 p 同余的最小非负整数 x，a 与 p 不必互质。输入顺序为 a、p、b，调用顺序为 solve(a,b,p)；返回 -1 时输出 No Solution，三个 0 结束输入。

所需模板：Mod64（第 7 页，2.1 节）、extended_gcd（第 23 页，3.6.1 节）、mod_inverse（第 26 页，3.8.1 节）、DiscreteLog（第 49 页，3.16.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4195

```
int main()
{
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    long long a, p, b;
    while (std::cin >> a >> p >> b)
    {
        if (a == 0 && p == 0 && b == 0) break;
        long long answer = DiscreteLog::solve(a, b, p);
        if (answer == -1)
            std::cout << "No Solution\n";
        else
            std::cout << answer << '\n';
    }
}
```

3.17 高次剩余 & 单位根

3.17.1 素数模高次剩余：全部根的紧凑表示

问题描述：给定素数 p 及其原根 g、正整数 k 与整数 a，求同余方程 $x^k \equiv a \pmod p$ 的全部解。解集以首项 `first`、公比 `ratio`、个数 `count` 的等比数列形式返回，不必展开即可得知解数。

核心思路：令 $x = g^y$，并用 `DiscreteLog` 求出 $a = g^L$ 中的 L，原方程化为指数上的线性同余 $ky \equiv L \pmod{p-1}$。`linear_congruence` 给出最小解 y_0 与周期 $T = (p-1)/\gcd(k, p-1)$，同余无解则原方程无解。于是全部根为 $g^{y_0}(g^T)^i$，$0 \le i < \gcd(k, p-1)$，即 `first` 取 g^{y_0}、`ratio` 取 g^T、`count` 取 $(p-1)/T$。$a \equiv 0$ 时唯一根为 0，单独返回。

代码依赖：DiscreteLog（第 49 页，3.16.1 节）、Mod64（第 7 页，2.1 节）、linear_congruence（第 30 页，3.9.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：solve(a,k,p,g) 求 x^k≡a (mod p)；p 必须是 2 到 1e12 的素数，g 是模 p 原根，k 为正 unsigned long long，a 支持完整 signed long long。无解返回 nullopt；否则得到 first、ratio、count，全部解为 first*ratio^i mod p，0≤i<count，无重复；first 不保证最小。a≡0 时仅有根 0，返回 {0,1,1}。枚举时每次乘 ratio，乘法先转 int128；无需展开全部根即可读取解数。给定原根后，期望 O(√p) 时间与空间，依赖 DiscreteLog、linear_congruence 及模快速幂。不验证 p 的素性或 g 的阶；合数模数不在此接口范围内。

```
struct KthResidue
{
    using ll = long long;
    ll first, ratio, count;

    // Prime p <= 1e12; g is a primitive root of p; k > 0.
    // All roots: first * ratio^i mod p, 0 <= i < count.
    static optional<KthResidue> solve(ll a, unsigned long long k, ll p, ll g)
    {
        assert(p >= 2 && p <= 1000000000000LL && g >= 1 && g < p && k > 0);
        a %= p;
        if (a < 0) a += p;
        if (a == 0) return KthResidue{0, 1, 1};
        ll logarithm = DiscreteLog::solve(g, a, p);
        if (logarithm < 0) return nullopt;
        auto [exponent, period] = linear_congruence(k % (p - 1), -logarithm, p - 1);
        if (exponent < 0) return nullopt;
        ll first = Mod64::power(g, exponent, p);
        ll ratio = Mod64::power(g, period, p);
        return KthResidue{first, ratio, (p - 1) / period};
    }
};
```

3.17.2 素数模高次剩余与单位根

实现见第 51 页；依赖离散对数、线性同余和快速幂。设 p 为素数，g 是原根，$k > 0$。$a = 0$ 时根只有零；否则 $a = g^b$、$x = g^y$，

$$ky \equiv b \pmod{p-1}.$$

令 $d = \gcd(k, p-1)$，$d \nmid b$ 时无解；否则 $y = y_0 + i(p-1)/d$，$0 \le i < d$，存为

$$\texttt{first} = g^{y_0}, \qquad \texttt{ratio} = g^{(p-1)/d}, \qquad \texttt{count} = d.$$

依次乘 ratio 枚举，不保证递增。$p \le 10^{12}$，调用者提供已验证的素数与原根；k 为 unsigned long long，a 先正规化。空 optional 表示无解，`{0,1,1}` 表示唯一零根。期望时间、空间 $O(\sqrt{p})$，枚举另需 $O(d)$；模乘须扩展到 128 位。

3.17.3 素数幂模高次剩余与根的提升

令 $M = p^e$。$a = 0$ 时根为 $p^{\lceil e/k \rceil}$ 的倍数。否则 $t = v_p(a) < e$，须 $k \mid t$，否则无解；取 $s = t/k$，$q = p^{e-t}$，$x = p^s y$，解

$$y^k \equiv a/p^t \pmod{q},$$

每个单位根 y 对应 $x = p^s y + \ell p^s q$，$0 \le \ell < p^{t-s}$。奇素数用模 p^e 的原根转线性同余；$q = 2$ 时单位根只有 1；$q \ge 4$ 为二的幂时单位写成 $(-1)^z 5^y$，z 模 2、y 模 $q/4$（$q = 4$ 只有符号分支），分别解指数同余。二的高次幂没有原根。时间由 BSGS 决定，期望 $O(\sqrt{q})$。

3.17.4 一般合数模高次剩余：CRT 组合

设 $M = \prod_t m_t$，$m_t = p_t^{e_t}$ 两两互素。分别求局部根族，任一局部无解则整体无解；整体根数为 $\prod_t c_t$。预处理

$$E_t = \frac{M}{m_t}\left(\left(\frac{M}{m_t}\right)^{-1} \bmod m_t\right) \bmod M, \qquad x = \sum_t r_t E_t \bmod M.$$

按混合进制索引可逐个取根，不需建出全部根；CRT 乘积扩展到 128 位。$M = 1$ 时唯一代表为 0。

3.18 数论分块

3.18.1 整除分块：前缀和来源决定算法

$H(t) = \sum_{i \le t} h(i)$，$H(0) = 0$，$q_A = \lfloor A/l \rfloor, q_B = \lfloor B/l \rfloor$，$l \le \min(A, B)$：

$$r = \min(\lfloor A/q_A \rfloor, \lfloor B/q_B \rfloor), \quad \text{整块贡献}(H(r) - H(l-1))q_A q_B.$$

两边都要参与取 r。段数 $O(\sqrt{A} + \sqrt{B})$（且 $\le \min(A, B)$），不是 $O(\sqrt{\min(A, B)})$。

3.18.1.1 按数据与资源选前缀和

上界可预筛，询问多： 第 20 页 `LinearSieve s(L)` 的 `s.mu`、`s.phi` 建前缀和，$O(L)$ 时间空间，查表 $O(1)$；筛上限覆盖短边。μ 前缀可负，φ 前缀勿用 `int`。

只求少量巨大前缀： 第 54 页 `DuJiao s(B)`：`s.mertens(n)`（有符号 64 位）、`s.totient_sum(n)`（128 位），$0 \le n \le 10^{11}$，$1 \le B <$ `INT_MAX`，B 是预筛上限。单次时间 $O(B + N/\sqrt{B})$，$B \approx N^{2/3}$ 得 $O(N^{2/3})$（哈希期望）。

内存：`DuJiao` 两张 64 位前缀表约 $16(B+1)$ 字节，构造时另有 `LinearSieve` 表；$N = 10^{11}$ 取 $B = N^{2/3}$ 约 345 MB，先看内存限制。不同查询增缓存；勿每次重建筛对象。正方形互质对直接 `2 * s.totient_sum(n) - 1`（$n \ge 1$）。一般矩形用 `mertens` 分块时，内层递归使总复杂度不止外层段数。

$$M(n) = 1 - \sum_{i=2}^{n} M(\lfloor n/i \rfloor), \qquad \Phi(n) = \frac{n(n+1)}{2} - \sum_{i=2}^{n} \Phi(\lfloor n/i \rfloor),$$

$M(n) = \sum_{i \le n} \mu(i)$，按等商段乘段长。先提升类型再乘；模下负和要归一化；最后除以 2 时先整数除再取模，合数模下不擅自求 2 的逆元。

3.19 狄利克雷卷积

3.19.1 幂的约数和：不除以 $p-1$

若 $a=\prod p_i^{e_i}$，则 $\sigma(a^b)=\prod_i(1+p_i+\cdots+p_i^{e_ib})$。合数模数或 $p\equiv 1$ 时分母不可逆，不能用几何级数除法公式。令 $G(a,n)=(a^n,\sum_{i=0}^{n-1}a^i)$，$G(a,0)=(1,0)$；由 $h=\lfloor n/2\rfloor$ 的 (P,S)，偶数项得 $(P^2,S(1+P))$，奇数项得 $(P^2a,S(1+P)+P^2)$。时间 $O(\log(n+1))$，任意正模数。参数是项数，约数和须传 e_ib+1；e_ib 可能超过 64 位，模乘在 128 位中做。

3.20 莫比乌斯反演

3.20.1 整除与反演：先辨认求和方向

数论函数定义在**正整数**上；下标 0 只是前缀和占位。

$$(f*g)(n)=\sum_{d\mid n}f(d)g(n/d),\qquad \varepsilon(n)=[n=1],\quad \mathbf{1}(n)=1,\quad \mathrm{id}(n)=n,\qquad \mu*\mathbf{1}=\varepsilon,\quad \varphi*\mathbf{1}=\mathrm{id}\,.$$

3.20.1.1 约数和与倍数和不能混用

约数方向。$F(n)=\sum_{d\mid n}f(d)\Rightarrow f(n)=\sum_{d\mid n}\mu(d)F(n/d)$，不要求 f 积性。
倍数方向。$E(k)$ 仅在 $1\le k\le V$ 非零，$G(d)=\sum_{j\ge 1,\,dj\le V}E(dj)$，则

$$E(k)=\sum_{t=1}^{\lfloor V/k\rfloor}\mu(t)G(kt),$$

或从大到小 $E(k)=G(k)-\sum_{j\ge 2,\,kj\le V}E(kj)$。须有限支持。互质下标对：$c_d=\#\{i:d\mid a_i\}$，答案 $\sum_{d=1}^{V}\mu(d)\binom{c_d}{2}$（重复值按下标区分）。枚举倍数求 c_d，时间 $O(V\log V)$、空间 $O(V)$；V 太大时须改为枚举实际约数。

3.20.1.2 GCD 权值先反演，再交换求和

令 $h=\mu*w$（$w=h*\mathbf{1}$），则

$$\sum_{x=1}^{A}\sum_{y=1}^{B}w(\gcd(x,y))=\sum_{d=1}^{\min(A,B)}h(d)\left\lfloor\frac{A}{d}\right\rfloor\left\lfloor\frac{B}{d}\right\rfloor.$$

互质计数 $h=\mu$，GCD 总和 $h=\varphi$；系数是卷积 $h(d)$，一般不是 $\mu(d)w(d)$。已知 h 的前缀和后按第 3.18.1 节分块。

3.20.2 GCD 计数：缩放、边界与对称性

记 $C(A,B)$ 为 $[1,A]\times[1,B]$ 中的**有序**互质数对数，任一边为 0 时为 0；不含坐标轴，不去掉 $x=y$。

3.20.2.1 恰好等于 k：先缩放整除对象，再做矩形容斥

$k\ge 1$，$a,c\ge 1$，$a\le x\le b$、$c\le y\le d$ 中 $\gcd=k$ 的对数为 $C(A,B)-C(A_0,B)-C(A,B_0)+C(A_0,B_0)$，其中

$$A=\lfloor b/k\rfloor,\quad A_0=\lfloor(a-1)/k\rfloor,\qquad B=\lfloor d/k\rfloor,\quad B_0=\lfloor(c-1)/k\rfloor.$$

先减 1 再除；$k>\min(b,d)$ 时自动为 0。多测预筛到所有查询的 $\max\min(b/k,d/k)$。

3.20.2.2 去掉顺序之前，先检查交换是否仍在定义域内

同一正整数区间 I，T 为 $I\times I$ 中 GCD 为 k 的有序对数，$\delta=[k\in I]$：$\#\{x<y\}=(T-\delta)/2$，$\#\{x\le y\}=(T+\delta)/2$。仅当定义域交换对称时可用；一般矩形不能直接除以 2。不同下标计数回到频率反演。正方形：$C(n,n)=2\Phi(n)-1$，$\Phi(n)=\sum_{i\le n}\varphi(i)$，$n\ge 1$；$n=0$ 为 0。含 0：$\gcd(0,y)=|y|$，轴上仅 $(0,k),(k,0)$ 另行判断；$(0,0)$ 按题面约定；负数处理绝对值。

3.20.3 矩形 GCD 计数：缩放后容斥

前缀矩形互质数对数

$$F(A,B)=\sum_{i=1}^{\min(A,B)}\mu(i)\left\lfloor\frac{A}{i}\right\rfloor\left\lfloor\frac{B}{i}\right\rfloor,$$

按两个商同时分块，右端点 $r=\min(A/\lfloor A/l\rfloor,B/\lfloor B/l\rfloor)$，用 μ 前缀和；只需筛到 $\min(A,B)$。$A=0$ 或 $B=0$ 时为 0。矩形 $[a,b]\times[c,d]$ 内 $\gcd(x,y)=k$ 的对数为

$$\begin{aligned}&F(\lfloor b/k\rfloor,\lfloor d/k\rfloor)-F(\lfloor(a-1)/k\rfloor,\lfloor d/k\rfloor)\\&-F(\lfloor b/k\rfloor,\lfloor(c-1)/k\rfloor)+F(\lfloor(a-1)/k\rfloor,\lfloor(c-1)/k\rfloor),\end{aligned}$$

必须先减 1 再除以 k。答案不超过面积，可存有符号 64 位；分块累加用 128 位。

3.21 狄利克雷双曲线法 & 杜教筛

3.21.1 杜教筛

问题描述：对很大的 n（约 10^{11}），求莫比乌斯函数前缀和 $M(n)=\sum_{i\le n}\mu(i)$ 与欧拉函数前缀和 $\Phi(n)=\sum_{i\le n}\varphi(i)$，支持多次查询。

核心思路：利用狄利克雷卷积 $\mu*1=\varepsilon$、$\varphi*1=\mathrm{id}$ 得 $\sum_{d=1}^{n}M(\lfloor n/d\rfloor)=1$、$\sum_{d=1}^{n}\Phi(\lfloor n/d\rfloor)=n(n+1)/2$。把 $d=1$ 项移到一边，其余 $d\ge 2$ 按 $\lfloor n/d\rfloor$ 相同的区间 $[l,r]$ 整除分块，递归求更小的值。不超过 `limit` 的值直接查线性筛预处理的前缀和 `pmu`、`pphi`，更大的值记忆化在哈希表 `mmu`、`mphi` 中。

代码依赖：LinearSieve（第 20 页，3.4.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：构造预筛上限 1≤B<INT_MAX，内存须可用；mertens(n) 返回 μ 的前缀和，totient_sum(n) 返回 φ 的前缀和（int128）。约定 0≤n≤10^{11}，n=0 返回 0。按整除分块递归并写入 mmu/mphi 缓存；重复查询复用结果，清空缓存可重算，不可修改 pmu/pphi/limit。单次最大参数 N 的常用均摊分析为 O(B+N/$\sqrt{}$B)（哈希按期望常数计算），B 约 N^(2/3) 时为 O(N^(2/3))；小 B 不构成大输入性能保证。多次不同 N 可能新增缓存，空间随实际缓存增长；递归深度 O(log N)。

```
struct DuJiao
{
    using ll = long long;
    using I = __int128_t;
    int limit;
    vector<ll> pmu, pphi;
    unordered_map<ll, ll> mmu;
    unordered_map<ll, I> mphi;

    DuJiao(int limit) : limit(limit), pmu(limit + 1), pphi(limit + 1)
    {
        assert(limit >= 1);
        LinearSieve s(limit);
        for (int i = 1; i <= limit; i++)
        {
            pmu[i] = pmu[i - 1] + s.mu[i];
            pphi[i] = pphi[i - 1] + s.phi[i];
        }
    }

    ll mertens(ll n)
    {
        if (n <= limit) return pmu[n];
        if (mmu.count(n)) return mmu[n];
        ll ans = 1;
```

```
        for (ll l = 2, r; l <= n; l = r + 1)
        {
            r = n / (n / l);
            ans -= (r - l + 1) * mertens(n / l);
        }
        return mmu[n] = ans;
    }

    I totient_sum(ll n)
    {
        if (n <= limit) return pphi[n];
        if (mphi.count(n)) return mphi[n];
        I ans = I(n) * (n + 1) / 2;
        for (ll l = 2, r; l <= n; l = r + 1)
        {
            r = n / (n / l);
            ans -= I(r - l + 1) * totient_sum(n / l);
        }
        return mphi[n] = ans;
    }
};
```

模板题使用：Luogu P4213

题意：多次查询 φ 与 μ 的前缀和，先输出 φ，再输出 μ。先读入全部 n，预筛到 min(最大 n,2000000)，同一结构复用缓存。题目 n<2^31，所以 φ 的总和可安全转为 long long；模板更大范围的 int128 返回值不能照搬此转换。

所需模板：LinearSieve（第 20 页，3.4.2 节）、DuJiao（第 54 页，3.21.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4213

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t;
    cin >> t;
    vector<long long> a(t);
    for (auto &n : a) cin >> n;
    int b = min(2000000LL, *max_element(a.begin(), a.end()));
    DuJiao d(b);
    for (long long n : a)
        cout << (long long)d.totient_sum(n) << ' ' << d.mertens(n) << '\n';
}
```

3.21.2 杜教筛：预筛与缓存都不可省略

整除反演建模（第 3.20.1 节，第 53 页）的两条卷积恒等式给出 DuJiao 的 mertens 与 totient_sum 递推：商相同的一段合并，系数为区间长度；超过预筛上限的结果必须写回缓存。最大查询 N、预筛上限 B 时单次 $O(B+N/\sqrt{B})$，取 $B\approx N^{2/3}$ 得 $O(N^{2/3})$；多个不同查询会继续增加缓存。范围 $0\le n\le 10^{11}$，欧拉函数前缀和用 128 位。取模时负值先求余，仅余数为负时加模数。

3.22 Min_25 筛

3.22.1 Min_25 积性函数求和

问题描述：给定积性函数 f：$f(p)$ 是素数 p 的至多二次多项式，$f(p^e)$ 由回调给出。求 $\sum_{i=1}^{n} f(i)$ 模素数，n 可达 10^{12}。

核心思路：所有 $\lfloor n/l \rfloor$ 只有 $O(\sqrt{n})$ 种，降序存于 `w`，经 `id` 用 `small`、`large` 两张表定位。第一阶段对每个 $x \in w$ 求素数幂和 $g(x,d)=\sum_{p\le x} p^d$ $(d=0,1,2)$：初值为 $\sum_{i=2}^{x} i^d$，依次对素数 $p \le \sqrt{n}$ 去掉最小质因子为 p 的合数。第二阶段 `dfs(x,k)` 求 $2..x$ 中最小质因子不小于第 k 个素数的数的 f 之和：素数部分由 g 的线性组合减去前 k 个素数的和，合数部分枚举最小质因子及其指数递归。

思路中的情况	代码中的处理	行
枚举所有整除商	$x=\lfloor n/l \rfloor$ 入 `w`，按 $x \le s$ 与否记入 `small[x]` 或 `large[n/x]`	21
g 的初值	把 $2..x$ 都当作素数：$\sum_{i=2}^{x} i^0, i^1, i^2$	32
判断 p 是否为素数	此时 $g(p,0)$ 已是精确的素数个数，与 $g(p-1,0)$ 相等说明 p 是合数	38
筛去最小质因子为 p 的合数	对 $x \ge p^2$：$g(x)$ $-=$ $p^d\,(g(x/p)-\sum_{q<p} q^d)$，`prefix` 是较小素数的幂和	45
`dfs` 的素数部分	$c_0g_0+c_1g_1+c_2g_2$ 减去前 k 个素数的 $f(p)$ 之和	65
`dfs` 的合数部分	枚举 p^e：加 $f(p^e)\cdot$`dfs`$(x/p^e, i+1)$ 与 $f(p^{e+1})$	70
加上 $f(1)$	`dfs` 只统计 ≥ 2 的数，最后补 1	75

代码依赖：ModInt（第 12 页，3.1.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：构造 Min25<mod>(n)，1≤n≤10^12，mod 为大于 3 的素数。sum(c, power) 求 ∑(1≤i≤n) f(i)，要求 f 为积性函数、f(1)=1，f(p)=c[0]+c[1]p+c[2]p²；power(p,e,pe) 返回 f(p^e)，其中 pe=p^e、e≥1，回调必须与一次幂系数一致。支持同一对象多次 sum；不改筛表。预处理 O(n^0.75 / log n)，回调 O(1) 时递归求和保守 O(n) 上界；O(√n) 空间、O(log n) 递归栈。w 降序存不同整除商，g[id(x)][d] 是模意义下 ∑(p≤x) p^d；id 仅允许查询这些整除商（包含 1..√n），各字段只读。依赖 ModInt；普通递归版，未承诺全部官方时限达标。

```cpp
template <int mod> struct Min25
{
    using ll = long long;
    using Z = ModInt<mod>;
    ll n;
    int s;
    vector<ll> w;
    vector<int> small, large, primes;
    vector<array<Z, 3>> g;

    Min25(ll n) : n(n)
    {
        static_assert(mod > 3);
        assert(1 <= n && n <= 1000000000000LL);
        s = sqrtl(n);
        while (1LL * (s + 1) * (s + 1) <= n) s++;
        while (1LL * s * s > n) s--;
        small.resize(s + 1);
        large.resize(s + 1);
        Z half = Z(2).inv(), sixth = Z(6).inv();
        for (ll l = 1, r; l <= n; l = r + 1)
        {
            ll x = n / l;
            r = n / x;
            int k = w.size();
            if (x <= s)
                small[x] = k;
            else
                large[n / x] = k;
            w.push_back(x);
            Z v = x;
            g.push_back(
                {v - 1, v * (v + 1) * half - 1, v * (v + 1) * (v * 2 + 1) * sixth - 1});
        }
```

```
        array<Z, 3> prefix{};
        for (int p = 2; p <= s; p++)
        {
            if (g[id(p)][0].v == g[id(p - 1)][0].v) continue;
            primes.push_back(p);
            array<Z, 3> weight{1, p, Z(p) * p};
            for (int j = 0; j < (int)w.size() && 1LL * p * p <= w[j]; j++)
            {
                int k = id(w[j] / p);
                for (int d = 0; d < 3; d++)
                    g[j][d] = g[j][d] - weight[d] * (g[k][d] - prefix[d]);
            }
            for (int d = 0; d < 3; d++) prefix[d] = prefix[d] + weight[d];
        }
    }

    int id(ll x) const { return x <= s ? small[x] : large[n / x]; }

    template <class F> Z sum(array<Z, 3> c, F power) const
    {
        vector<Z> prefix(primes.size() + 1);
        for (int i = 0; i < (int)primes.size(); i++)
        {
            Z p = primes[i];
            prefix[i + 1] = prefix[i] + c[0] + c[1] * p + c[2] * p * p;
        }
        auto dfs = [&](auto &&self, ll x, int k) -> Z
        {
            if (x < 2 || (k && x <= primes[k - 1])) return 0;
            auto a = g[id(x)];
            Z ans = c[0] * a[0] + c[1] * a[1] + c[2] * a[2] - prefix[k];
            for (int i = k; i < (int)primes.size() && primes[i] <= x / primes[i]; i++)
            {
                ll p = primes[i], pe = p;
                for (int e = 1; pe <= x / p; e++, pe *= p)
                    ans = ans + power(p, e, pe) * self(self, x / pe, i + 1) +
                          power(p, e + 1, pe * p);
            }
            return ans;
        };
        return dfs(dfs, n, 0) + 1;
    }
};
```

模板题使用：Luogu P5325

题意：求积性函数前缀和，f(p^e)=p^e(p^e-1)，模 1000000007。一次幂为 p^2-p，故系数传 {0,-1,1}；回调的 pe 已是整数 p^e，无需再快速幂。

所需模板：ModInt（第 12 页，3.1.2 节）、Min25（第 56 页，3.22.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5325

```
int main()
{
    using Z = ModInt<1000000007>;
    long long n;
    cin >> n;
    Min25<1000000007> sieve(n);
    auto power = [](long long p, int e, long long pe)
    {
        Z x = pe;
        return x * (x - 1);
    };
    cout << sieve.sum({0, -1, 1}, power).v << '\n';
}
```

3.23 类欧几里德算法

3.23.1 有符号整除和：类欧几里德

问题描述：给定 n, m, a, b（a, b 可以为负），求 $\sum_{i=0}^{n-1} \lfloor (ai+b)/m \rfloor$。

核心思路：先把 a, b 按数学向下取整规范到 $[0, m)$，分离出的商 q_a, q_b 贡献 $q_a \cdot n(n-1)/2 + q_b \cdot n$。之后循环（类欧几里得）：每轮先把 a, b 中不小于 m 的整商部分直接累加并取余；若 $y = an + b < m$，剩下各项全为 0，结束。否则把和看成直线 $(ax+b)/m$ 下方的格点数并交换坐标轴，问题化为项数 $\lfloor y/m \rfloor$、常数 $y \bmod m$、并交换 a 与 m 的同形和式，(a, m) 像欧几里得算法一样迅速缩小。

接口契约：floor_sum(n,m,a,b) 返回 i=0..n-1 的 floor((ai+b)/m) 之和，结果为 int128。要求 0≤n≤10^9、1≤m≤10^9，a、b 支持完整 long long；n=0 返回 0。先按数学向下取整规范化负系数，再迭代交换分母和余数；不能把 C++ 负数截零除法当作 floor。O(log m) 时间、O(1) 空间，不依赖完整数论模块。

```
__int128_t floor_sum(long long n, long long m, long long a, long long b)
{
    using ll = long long;
    using i128 = __int128_t;
    assert(n >= 0 && m > 0);
    i128 ans = 0;
    auto norm = [&](ll &v)
    {
        ll q = v / m;
        if (v % m < 0) --q;
        v = (ll)(i128(v) - i128(q) * m);
        return q;
    };
    ans += i128(norm(a)) * n * (n - 1) / 2;
    ans += i128(norm(b)) * n;
    while (true)
    {
        ans += i128(n) * (n - 1) / 2 * (a / m);
        a %= m;
        ans += i128(n) * (b / m);
        b %= m;
        i128 y = i128(a) * n + b;
        if (y < m) break;
        n = (ll)(y / m);
        b = (ll)(y % m);
        swap(a, m);
    }
    return ans;
}
```

模板题使用：Library Checker sum_of_floor_of_linear

题意：求 i=0..n-1 的 floor((a*i+b)/m) 之和。此题有 0≤a,b<m，因此结果可转为 long long 输出；更一般的有符号输入不能照搬该转换。

所需模板：floor_sum（第 60 页，3.23.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/sum_of_floor_of_linear

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t;
    cin >> t;
    while (t--)
    {
        long long n, m, a, b;
        cin >> n >> m >> a >> b;
        // This task has 0 <= a,b < m, so the sum is at most n*(n-1)/2.
        cout << (long long)floor_sum(n, m, a, b) << '\n';
    }
}
```

3.23.2 整除和、带权和与平方和

问题描述： 对 $i=0..n-1$ 令 $y_i=\lfloor(ai+b)/m\rfloor$，同时求 $\sum y_i$、$\sum i\,y_i$、$\sum y_i^2$（模 998244353）。

核心思路： 先把 a,b 规范到 $[0,m)$，商为 x,y，则 $y_i=xi+y+z_i$，展开后只需 z_i 的三种和 f,g,h 及 $\sum i$、$\sum i^2$（s1、s2）。求 f,g,h 时令 t 为最大值 height，交换求和次序：$z_i\ge j$ 当且仅当 $i\ge c_j$，而 $c_{j+1}=\lfloor(mj+m+a-b-1)/a\rfloor$ 仍是同形的整除式，于是以参数 $(t,a,m,m+a-b-1)$ 递归（a 与 m 互换，类欧几里得）得到 $u=\sum c$、$v=\sum j\,c$、$w=\sum c^2$，再由 $f=nt-u$、$g=t\,s_1-(w-u)/2$、$h=nt^2-2v-u$ 还原。

代码依赖：ModInt（第 12 页，3.1.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约： floor_moments(n,m,a,b) 对 i=0..n-1，令 y_i=floor((a*i+b)/m)，返回 {∑y_i, ∑i*y_i, ∑y_i²}，元素为 ModInt<998244353>，取整数用.v。0≤n≤1000000001，1≤m≤10^9；a、b 支持完整 long long。n=0 返回全零。时间与递归空间 O(log(m+1))；模数固定，499122177 与 166374059 分别是 2、6 的逆元。P5170 输入上限为闭区间，驱动传 n+1，输出索引顺序 0、2、1。

```
array<ModInt<998244353>, 3>
floor_moments(long long n, long long m, long long a, long long b)
{
    using Z = ModInt<998244353>;
    assert(n >= 0 && n <= 1000000001 && m >= 1 && m <= 1000000000);
    if (!n) return {0, 0, 0};
    auto normalize = [&](long long &v)
    {
        long long q = v / m;
        if (v % m < 0) q--;
        v = (__int128_t(v) - __int128_t(q) * m);
        return Z(q);
    };
    Z x = normalize(a);
    Z y = normalize(b);
    Z count = n;
    Z s1 = count * Z(n - 1) * Z(499122177);
    Z s2 = count * Z(n - 1) * Z(2 * n - 1) * Z(166374059);
    Z f = 0, g = 0, h = 0;
    if (a)
    {
        long long height = (a * (n - 1) + b) / m;
        auto [u, v, w] = floor_moments(height, a, m, m + a - b - 1);
        Z t = height;
        f = count * t - u;
        g = t * s1 - (w - u) * Z(499122177);
        h = count * t * t - Z(2) * v - u;
    }
    return {f + x * s1 + y * count,
            g + x * s2 + y * s1,
            h + x * x * s2 + y * y * count + Z(2) * x * y * s1 + Z(2) * x * g +
                Z(2) * y * f};
}
```

模板题使用：Luogu P5170

题意：求 i=0..n 的整除和、整除值平方和、i 乘整除值之和，模 998244353。t 至 10^5，n/a/b/c 至 10^9 且 c>0。核心参数顺序 (n+1,c,a,b)，返回顺序是和/带权和/平方和；本题输出索引 0、2、1，不能只核对第一项。

所需模板：ModInt（第 12 页，3.1.2 节）、floor_moments（第 62 页，3.23.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5170

```
int main()
{
    int t;
    scanf("%d", &t);
    while (t--)
    {
        long long n, a, b, c;
        scanf("%lld%lld%lld%lld", &n, &a, &b, &c);
        auto result = floor_moments(n + 1, c, a, b);
        printf("%d %d %d\n", result[0].v, result[2].v, result[1].v);
    }
    return 0;
}
```

3.23.3 类欧几里德：有符号整除求和

接口见第 3.23.1 节（第 60 页）。`floor_sum(n,m,a,b)` 返回

$$F(n,m,a,b)=\sum_{i=0}^{n-1}\left\lfloor\frac{ai+b}{m}\right\rfloor.$$

要求 $0\le n\le 10^9$、$1\le m\le 10^9$，a,b 为完整有符号 64 位，返回 128 位。n 是项数，下标 $0\dots N$ 时传 $N+1$。令 $a=q_am+r_a$、$b=q_bm+r_b$（余数在 $[0,m)$，负数商须向下取整），先加 $q_an(n-1)/2+q_bn$。对 $0\le a,b<m$，设 $y=an+b$；$y<m$ 时余下为 0，否则

$$F(n,m,a,b)=F(\lfloor y/m\rfloor,a,m,y \bmod m).$$

时间 $O(\log(m+1))$，额外空间 $O(1)$。

3.23.4 类欧几里德的三个求和量

实现见第 3.23.2 节（第 62 页）。令 $y_i=\lfloor(ai+b)/m\rfloor$，$0\le i<n$，求 $f=\sum y_i$、$g=\sum iy_i$、$h=\sum y_i^2$。范围 $0\le n\le 1000000001$、$1\le m\le 10^9$，a,b 为完整有符号 64 位；$n=0$ 返回 $(0,0,0)$。结果固定模 998244353，换模数须同时换 2、6 的逆元。

归一化 $a=xm+a'$、$b=ym+b'$（向下取整），规范化问题返回 (f_0,g_0,h_0)，$S_1=\sum i$、$S_2=\sum i^2$：

$$
\begin{aligned}
f&=f_0+xS_1+yn,\\
g&=g_0+xS_2+yS_1,\\
h&=h_0+x^2S_2+y^2n+2xyS_1+2xg_0+2yf_0.
\end{aligned}
$$

$a'=0$ 时剩余量为 0。$0<a'<m$ 时令 $t=\lfloor(a'(n-1)+b')/m\rfloor$，递归 $(t,a',m,m+a'-b'-1)$ 得 (u,v,w)：

$$
f_0=nt-u,\qquad g_0=tS_1-(w-u)/2,\qquad h_0=nt^2-2v-u.
$$

递归 $O(\log(m+1))$ 层。P5170 上界含 n，传 $n+1$，输出顺序为 f,h,g。

第四章　多项式与生成函数

4.1　快速傅里叶变换

4.1.1　复数 FFT（可复用单位根）

功能：对长度为 n（二次幂）的复数数组做原地离散傅里叶变换及逆变换；单位根表在构造时一次算好，可供多次变换复用，是浮点多项式乘法、拆系数任意模卷积等的底层工具。

核心思路：构造时直接按角度用 $\cos,\sin$ 算出 $\omega^i = e^{2\pi \mathrm{i}\, i/n}$ $(i < n/2)$，不靠连乘，避免误差累积。`transform` 先做位逆序重排（`j` 是 `i` 的二进制反转，增量维护），再自底向上合并：两段长度为 `half` 的子结果用单位根 `roots[j*step]` 做蝴蝶 $u \pm \omega v$。逆变换把单位根取共轭，最后整体除以 n。

接口契约：ComplexFFT(n) 预计算单位根，n 为 1..2^22 的二次幂。transform(a,inverse=false) 原地正变换，正向指数为正；inverse=true 用共轭根并除以 n。a 长度须等于 n；C=complex<long double>。根直接用角度计算，不沿整层连乘。单次变换 O(n log(n+1))，根表 O(n)，可复用。浮点环境须二进制至少 53 位、最近舍入、禁用 fast-math；整数卷积还要求单位根绝对误差 $\le 8\times 2^{-53}$，本机全根已用百位参考核验。变换本身是近似值。

```
struct ComplexFFT
{
    using Real = long double;
    using C = complex<Real>;
    static constexpr int max_size = 1 << 22;
    int n;
    vector<C> roots;

    explicit ComplexFFT(int size) : n(size)
    {
        static_assert(numeric_limits<Real>::radix == 2);
        static_assert(numeric_limits<Real>::digits >= 53);
        assert(n > 0 && n <= max_size && (n & (n - 1)) == 0);
        roots.resize(n / 2);
        Real pi = acosl(-1);
        for (int i = 0; i < n / 2; i++)
        {
            Real angle = pi * (2.0L * i / n);
            roots[i] = C(cosl(angle), sinl(angle));
        }
    }
```

正逆变换（接上页同一结构体）：

```
    void transform(vector<C> &a, bool inverse = false) const
    {
        assert(a.size() == (unsigned)n);
        for (int i = 1, j = 0; i < n; i++)
        {
            int bit = n >> 1;
            for (; j & bit; bit >>= 1) j ^= bit;
            j ^= bit;
            if (i < j) swap(a[i], a[j]);
        }
        for (int half = 1; half < n; half *= 2)
        {
            int step = n / (2 * half);
            for (int i = 0; i < n; i += 2 * half)
            {
                for (int j = 0; j < half; j++)
                {
                    C w = roots[j * step];
                    if (inverse) w = conj(w);
                    C u = a[i + j], v = a[i + j + half] * w;
                    a[i + j] = u + v;
                    a[i + j + half] = u - v;
                }
            }
        }
        if (inverse)
            for (auto &x : a) x /= n;
    }
};
```

模板题使用：Luogu P3803 【模板】多项式乘法（FFT）—直接变换接口

题意：给出次数 0≤n,m≤1000000、系数均为 0..9 整数的两个多项式，按升幂读入并输出乘积的 n+m+1 项。补零到不小于 n+m+1 的最小二次幂 L（至多 2^21），显式构造 ComplexFFT，分别正变换、逐点相乘、逆变换后 llroundl 取整。保留本节的二进制至少 53 位、最近舍入、单位根误差界及禁用 fast-math 前提；本题范数乘积至多 81000081。时间 O(L log L)，空间 O(L)。这是底层变换接口的完整用法，卷积包装器和精确 CRT 用法另列。

所需模板：ComplexFFT（第 66 页，4.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3803

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    int size = 1;
    while (size < n + m + 1) size *= 2;
    auto fft = ComplexFFT(size);
    vector<ComplexFFT::C> a(size), b(size);
    for (int i = 0, x; i <= n; i++) cin >> x, a[i] = x;
    for (int i = 0, x; i <= m; i++) cin >> x, b[i] = x;
    fft.transform(a);
    fft.transform(b);
    for (int i = 0; i < size; i++) a[i] *= b[i];
    fft.transform(a, true);
    for (int i = 0; i <= n + m; i++) cout << llroundl(a[i].real()) << ' ';
    cout << '\n';
}
```

4.1.2 拆系数 FFT 任意模卷积

功能：计算两个整数多项式乘积的系数模任意给定整数 M，即 $c_k = \sum_{i+j=k} a_i b_j \bmod M$；$M$ 可以是合数或不支持 NTT 的素数。

核心思路：每个系数先归一化到 $[0, M)$，再拆成三个 11 位块 $v = v_0 + v_1 B + v_2 B^2$（$B = 2^{11}$），这样任意两块卷积的真实系数都足够小，浮点 FFT 后四舍五入即可精确还原。对 a, b 的三个块各做一次正变换（共六次），再对每对块 (i, j) 逐点相乘、逆变换、`llroundl` 取整，乘上 $B^{i+j} \bmod M$（预存于 `weight`）累加进结果，共九次逆变换。

代码依赖：ComplexFFT（第 66 页，4.1.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：convolution_mod_fft(a,b,mod) 接受 vector<int>，返回 [0,mod) 系数；1≤mod≤INT_MAX，允许合数与负输入，mod=1 返回零。空输入返回空，非空结果长度 ≤2^19。依赖 ComplexFFT 及其浮点前提；归一化后拆成三个 11 位块，六次正变换和九次逆变换，分别舍入后模合并。每路误差界小于 0.079；不依赖更宽的 long double。时间 O(N log N)，空间 O(N)，常数高于打包版本以保持数值界与代码清楚；覆盖 P4245。

```
// Three 11-bit limbs; same floating-point assumptions as convolution_fft.
vector<int> convolution_mod_fft(const vector<int> &a, const vector<int> &b, int mod)
{
    assert(mod >= 1);
    if (a.empty() || b.empty()) return {};
    assert(a.size() + b.size() - 1 <= (1 << 19));
    int size = a.size() + b.size() - 1, n = 1;
    while (n < size) n *= 2;
    vector<int> result(size);
    if (mod == 1) return result;
```

```
    ComplexFFT fft(n);
    using C = ComplexFFT::C;
    const int base = 1 << 11;
    vector<vector<C>> x(3, vector<C>(n)), y = x;
    for (int i = 0; i < (int)a.size(); i++)
    {
        unsigned v = (a[i] % mod + (long long)mod) % mod;
        for (int j = 0; j < 3; j++) x[j][i] = (v >> (11 * j)) & (base - 1);
    }
    for (int i = 0; i < (int)b.size(); i++)
    {
        unsigned v = (b[i] % mod + (long long)mod) % mod;
        for (int j = 0; j < 3; j++) y[j][i] = (v >> (11 * j)) & (base - 1);
    }
    for (int j = 0; j < 3; j++)
    {
        fft.transform(x[j]);
        fft.transform(y[j]);
    }
    long long weight[5] = {1};
    for (int i = 1; i < 5; i++) weight[i] = weight[i - 1] * base % mod;
    vector<C> z(n);
    for (int i = 0; i < 3; i++)
    {
        for (int j = 0; j < 3; j++)
        {
            for (int k = 0; k < n; k++) z[k] = x[i][k] * y[j][k];
            fft.transform(z, true);
            for (int k = 0; k < size; k++)
            {
                long long v = llroundl(z[k].real()) % mod;
                result[k] = (result[k] + v * weight[i + j]) % mod;
            }
        }
    }
    return result;
}
```

模板题使用：Luogu P4245

题意：输入两个多项式的次数、任意模数与升幂系数，输出模乘积。题目次数不超过十万、模数最多 1000000009；函数会先归一化再拆系数，不要求模数支持 NTT。这里使用三块浮点 FFT，并遵守其单位根与舍入前提。

所需模板：ComplexFFT（第 66 页，4.1.1 节）、convolution_mod_fft（第 68 页，4.1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4245

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, mod;
    cin >> n >> m >> mod;
    vector<int> a(n + 1), b(m + 1);
    for (int &x : a) cin >> x;
    for (int &x : b) cin >> x;
    auto c = convolution_mod_fft(a, b, mod);
    for (int x : c) cout << x << ' ';
    cout << '\n';
}
```

相关应用：CDQ 分治配合 NTT 的卷积递推，附件标题为分治 FFT；算法不做浮点变换。

4.1.3 CDQ 分治卷积递推

问题描述：求满足半在线卷积递推 $f_i = \text{init}_i + \sum_{j<i} f_j\, g_{i-j}$ 的前 n 项（核满足 $g_0 = 0$，模 NTT 素数）。f_i 依赖自身更早的项，不能直接做一次卷积。

核心思路：在区间 $[l, r)$ 上分治：先递归解出左半 $[l, m)$，此时左半的 f 已全部确定；把它与核 $g_0, \ldots, g_{r-l-1}$ 做一次 NTT 乘法，乘积第 $i - l$ 项就是左半对右半位置 $i \in [m, r)$ 的贡献，累加进 `initial[i]`；再递归右半。区间长度为 1 时该位置的所有贡献都已加完（$g_0 = 0$，无自身项），即为最终值。数组 `initial` 原地存放结果。

代码依赖：ModInt（第 12 页，3.1.2 节）、NttConvolution（第 73 页，4.2.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：cdq_convolution<mod=998244353,primitive=3>(kernel,initial) 求 f[i]=initial[i]+∑(j<i)f[j]kernel[i-j]，返回前 n 项。两数组为等长 vector<ModInt<mod»，kernel[0] 必须为 0；空输入返回空。模数须为 int 范围素数，primitive 是原根；依赖 NttConvolution。非空时要求 n+floor(n/2)-1≤NTT::max_size，保证每层完整卷积可变换。初始项可任意，并非只有 f[0]=1；左半区解定后，将其到右半区的卷积贡献加上，再递归右半区。O(n $\log^2$(n+1)) 时间、O(n) 额外空间，递归深度 O(log n)。本函数是静态线性递推，不提供流式追加或非线性叶子回调。P4721 使用 initial[0]=1、其他项 0。

```
// f[i] = initial[i] + sum_{j<i} f[j] * kernel[i-j]; kernel[0] must be zero.
template <int mod = 998244353, int primitive = 3>
vector<ModInt<mod>> cdq_convolution(const vector<ModInt<mod>> &kernel,
                                    vector<ModInt<mod>> initial)
{
    using Ntt = NttConvolution<mod, primitive>;
    using Poly = typename Ntt::Poly;
    assert(kernel.size() == initial.size());
    if (initial.empty()) return {};
    assert(kernel[0].v == 0);
    assert(initial.size() + initial.size() / 2 - 1 <= Ntt::max_size);
    int n = initial.size();
    auto solve = [&](auto &&self, int l, int r) -> void
    {
        if (r - l == 1) return;
        int m = (l + r) / 2;
        self(self, l, m);
        Poly left(initial.begin() + l, initial.begin() + m);
        Poly right(kernel.begin(), kernel.begin() + r - l);
        Poly product = Ntt::multiply(move(left), move(right));
        for (int i = m; i < r; i++) initial[i] += product[i - l];
        self(self, m, r);
    };
    solve(solve, 0, n);
    return initial;
}
```

模板题使用：Luogu P4721 分治 FFT

题意：输入 g[1..n-1]，计算 f[0]=1、f[i]=∑(j=1..i)f[i-j]g[j]，模 998244353。kernel[0] 留零，initial 仅第零项为 1；调用的是真正 CDQ 分治卷积，依赖 NTT 与模整数。

所需模板：ModInt（第 12 页，3.1.2 节）、NttConvolution（第 73 页，4.2.1 节）、cdq_convolution（第 71 页，4.1.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4721

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    using Z = ModInt<998244353>;
    vector<Z> g(n), b(n);
    for (int i = 1; i < n; i++)
    {
        int x;
        cin >> x;
        g[i] = x;
    }
    b[0] = 1;
    auto f = cdq_convolution(g, b);
    for (int i = 0; i < n; i++) cout << f[i].v << " \n"[i + 1 == n];
}
```

4.1.4 浮点 FFT：舍入前先约束最坏误差

前提：53 位有效精度、最近舍入、禁用 fast-math，单位根绝对误差不超过 $8u$（$u = 2^{-53}$）。长度 $N = 2^{\ell}$、两输入欧氏范数积 S 时卷积误差界

$$S\left((1+u)^{3\ell}(1+\sqrt{5}u)^{3\ell+1}(1+8u)^{3\ell}-1\right).$$

整数入口要求 $N \le 2^{22}$、$\max|a_i|\max|b_i|\sqrt{|a||b|} \le 2\cdot 10^{12}$，误差 < 0.166，用 llroundl 舍入；超限不保证正确。任意模卷积拆三个 11 位块，结果长度 $\le 2^{19}$，误差 < 0.079；模数可为 1 或合数，空输入返回空。

4.2 快速数论变换

4.2.1 可指定素数与原根的 NTT 卷积

功能：给定两个模素数 p 的多项式（系数数组）a, b，求乘积 $c_k = \sum_{i+j=k} a_i b_j \bmod p$；同时提供原地数论变换 `ntt` 供其他多项式算法复用。

核心思路：取 $p-1$ 的二次幂因子中的长度 n 与 n 次单位根 $\omega = g^{(p-1)/n}$，把系数表示变成 n 个点值：位逆序重排后自底向上做蝴蝶合并。两个点值逐项相乘后做逆变换（同一正变换后翻转下标 $1..n-1$ 并乘 n^{-1}）还原系数。单位根表按层缓存，各长度复用。

代码依赖：ModInt（第 12 页，3.1.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：声明 NttConvolution<mod, primitive>，primitive 默认 3。mod 必须为 int 范围内的素数，primitive 是模 mod 原根；不在接口内验证素性或阶。max_size 是 mod-1 的最大二次幂因子。ntt(a,invert) 原地变换，长度须为正二次幂且不超过 max_size；invert=true 为逆变换。multiply(a,b) 返回普通卷积，任一为空则为空；非空时长度和减一不超过 max_size。元素类型 Z，数组类型 Poly。O(n log n) 时间；每种 mod/primitive 实例缓存分层单位根，空间 O(历次最大变换长度)，同一实例调用时不并发扩容。输入输出均为自然顺序，逆变换复用正变换后翻转下标 1..n-1 并乘 n 的逆元。998244353 和 167772161 均可用原根 3。

```
template <int mod, int primitive = 3> struct NttConvolution
{
    static_assert(mod >= 2 && primitive >= 1 && primitive < mod);
    static constexpr int max_size = (mod - 1) & -(mod - 1);
    using Z = ModInt<mod>;
    using Poly = vector<Z>;

    // mod is prime, primitive is a primitive root modulo mod.
    static void ntt(Poly &a, bool invert = false)
    {
        int n = a.size();
        assert(n > 0 && (n & (n - 1)) == 0 && n <= max_size);
        static Poly roots{0, 1};
        for (int half = roots.size(); half < n; half *= 2)
        {
            Z step = Z(primitive).pow((mod - 1) / (2 * half));
            roots.resize(2 * half);
            for (int i = half / 2; i < half; i++)
            {
                roots[2 * i] = roots[i];
                roots[2 * i + 1] = roots[i] * step;
            }
        }
        for (int i = 1, j = 0; i < n; i++)
        {
            int bit = n >> 1;
            for (; j & bit; bit >>= 1)
                j ^= bit;
            j ^= bit;
            if (i < j)
                swap(a[i], a[j]);
        }
```

蝶形变换与卷积（接上页同一结构体与函数）：

```
        for (int half = 1; half < n; half *= 2)
        {
            for (int i = 0; i < n; i += 2 * half)
            {
                for (int j = 0; j < half; j++)
                {
                    Z u = a[i + j];
                    Z v = a[i + j + half] * roots[half + j];
                    a[i + j] = u + v;
                    a[i + j + half] = u - v;
                }
            }
        }
        if (invert)
        {
            reverse(a.begin() + 1, a.end());
            Z inverse = Z(n).inv();
            for (auto &x : a)
                x *= inverse;
        }
    }

    static Poly multiply(Poly a, Poly b)
    {
        if (a.empty() || b.empty())
            return {};
        assert(a.size() + b.size() - 1 <= max_size);
        int size = a.size() + b.size() - 1, n = 1;
        while (n < size)
            n *= 2;
        a.resize(n);
        b.resize(n);
        ntt(a);
        ntt(b);
        for (int i = 0; i < n; i++)
            a[i] *= b[i];
        ntt(a, true);
        a.resize(size);
        return a;
    }
};
```

模板题使用：Library Checker convolution_mod

题意：求两个多项式的卷积，模 998244353。输入为各自系数个数和升幂系数；输出 n+m-1 个系数。题目保证输入系数已经取模。

所需模板：ModInt（第 12 页，3.1.2 节）、NttConvolution（第 73 页，4.2.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/convolution_mod

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    using C = NttConvolution<998244353>;
    int n, m;
    cin >> n >> m;
    C::Poly a(n), b(m);
    for (auto &x : a) cin >> x.v;
    for (auto &x : b) cin >> x.v;
    auto c = C::multiply(move(a), move(b));
    for (int i = 0; i < (int)c.size(); i++)
        cout << c[i].v << (i + 1 == (int)c.size() ? '\n' : ' ');
}
```

4.2.2 精确有符号整数卷积

功能：计算两个 `long long` 整数系数（可为负）多项式乘积的精确系数，要求结果系数本身落在有符号 64 位范围内。

核心思路：用三个 NTT 友好素数 p, q, r（乘积约 2^{86}）分别做模卷积（`convolution_residue`），再用 Garner 算法合并：先由模 p、q 的余数求出 $v \equiv c \pmod{pq}$，再求 u 使 $v + pq\,u \equiv c \pmod{r}$，得到 $[0, pqr)$ 中的唯一代表。它超过 $pqr/2$ 时减去 pqr 取中心代表，从而还原负系数。Garner 所需的两个逆元只计算一次。

代码依赖：ModInt（第 12 页，3.1.2 节）、NttConvolution（第 73 页，4.2.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：convolution_i64(a,b) 返回 signed64 整数卷积，空输入返回空。非空结果长度 ≤2^24，所有真实系数必须在 signed64 范围内。依赖 NttConvolution，三个素数模卷积后用 int128 CRT 重构中心代表。断言不能检测任意越界结果的模数别名。O(L log L) 时间、O(L) 空间，L 为补齐的二次幂长度。负数与极值已通过 cpp_int 参考验证；P3803 最大次数驱动本地通过，线上性能待验证。

```
template <int p>
vector<int> convolution_residue(const vector<long long> &a, const vector<long long> &b)
{
    using N = NttConvolution<p>;
    typename N::Poly x(a.begin(), a.end()), y(b.begin(), b.end());
    auto c = N::multiply(move(x), move(y));
    vector<int> r(c.size());
    for (int i = 0; i < (int)c.size(); i++) r[i] = c[i].v;
    return r;
}

vector<long long> convolution_i64(const vector<long long> &a,
                                  const vector<long long> &b)
{
    if (a.empty() || b.empty()) return {};
    assert(a.size() <= (1 << 24) && b.size() <= (1 << 24));
    assert(a.size() + b.size() - 1 <= (1 << 24));
    const long long p = 167772161, q = 469762049, r = 1224736769;
    const long long pq = p * q;
    const __int128 mod = (__int128)pq * r;
    static const long long ip = ModInt<q>(p).inv().v;
    static const long long ipq = ModInt<r>(pq).inv().v;
    auto x = convolution_residue<p>(a, b);
    auto y = convolution_residue<q>(a, b);
    auto z = convolution_residue<r>(a, b);
    vector<long long> c(x.size());
    for (int i = 0; i < (int)c.size(); i++)
    {
        long long t = (y[i] - (long long)x[i] + q) % q * ip % q;
        long long v = x[i] + p * t;
        long long u = (z[i] - v % r + r) % r * ipq % r;
        __int128 value = v + (__int128)pq * u;
        if (value > mod / 2) value -= mod;
        assert(LLONG_MIN <= value && value <= LLONG_MAX);
        c[i] = (long long)value;
    }
    return c;
}
```

模板题使用：Luogu P3803

题意：求两个整数多项式的乘积，系数按升幂输入。首行 n、m 是次数，系数个数分别为 n+1、m+1；输出精确整数，不能取模。本题系数在 0..9，满足 signed64 结果范围。

所需模板：ModInt（第 12 页，3.1.2 节）、NttConvolution（第 73 页，4.2.1 节）、convolution_i64（第 76 页，4.2.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3803

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    vector<long long> a(n + 1), b(m + 1);
    for (auto &x : a) cin >> x;
    for (auto &x : b) cin >> x;
    auto c = convolution_i64(a, b);
    for (int i = 0; i < (int)c.size(); i++)
        cout << c[i] << (i + 1 == (int)c.size() ? '\n' : ' ');
}
```

4.2.3 参数化 NTT 与模数适配

实现见第 73 页。传入素数 p 与原根 g；长度 n 为二次幂且整除 $p-1$，上限 $L = 2^{v_2(p-1)}$。普通卷积补齐到不小于长度和减一的二次幂。998244353 的 $L = 2^{23}$，167772161 的 $L = 2^{25}$，原根均为 3。接口不检查素性或原根的阶；模数限 int 范围内的素数，乘积用 64 位。逆变换用逆单位根并乘 n^{-1}。

4.3 快速沃尔什变换

4.3.1 按位 OR、AND、XOR 卷积

功能：给定两个长度为 2^m 的数组，求按下标位运算合并的卷积 $c_s = \sum_{i \circ j = s} a_i b_j$，其中 $\circ$ 为按位或、与、异或之一（模 `mod`）；也单独提供原地变换 `transform`。

核心思路：三种运算都能逐位独立处理：每层把下标在该位上为 0、1 的一对元素 u, v 做变换。OR 做子集和 $v \leftarrow v + u$，AND 做超集和 $u \leftarrow u + v$，逆变换改为减法；XOR 做 Walsh 变换 $(u, v) \leftarrow (u + v, u - v)$，逆变换每层再乘 $1/2$（取 $(\mathrm{mod} + 1)/2$，故只要求模数为奇数）。`multiply` 对两数组正变换、逐点相乘，再逆变换。

代码依赖：ModInt（第 12 页，3.1.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：SetConvolution<mod=998244353>；Z 为 ModInt<mod>，Poly 为 vector<Z>。数组长度 N 必须是正的 2 次幂且 ≤INT_MAX，表示固定二进制位数的全集，长度 1 合法。transform(a,op,invert=false) 原地变换，op 只能为 |、&、^；| 为子集和，& 为超集和，^ 为 Walsh 变换。multiply(a,b,op) 返回 $\sum$ a[i]b[j]（满足 i op j=s），a、b 长度必须相同。mod 可为 ≥2 的 int 范围合数；XOR 要求 mod 为奇数，逆 2 用 (mod+1)/2，不使用费马逆元。O(N log N) 时间，原地变换 O(1) 额外空间，卷积 O(N) 空间。

```
template <int mod = 998244353> struct SetConvolution
{
    static_assert(mod >= 2);
    using Z = ModInt<mod>;
    using Poly = vector<Z>;

    static void transform(Poly &a, char op, bool invert = false)
    {
        assert(!a.empty() && a.size() <= INT_MAX);
        int n = a.size();
        assert((n & (n - 1)) == 0);
        assert(op == '|' || op == '&' || op == '^');
        assert(op != '^' || mod % 2 == 1);
        Z half = (mod + 1LL) / 2;
        for (int len = 1; len < n; len *= 2)
            for (int i = 0; i < n; i += len * 2)
                for (int j = 0; j < len; j++)
                {
                    Z &u = a[i + j], &v = a[i + j + len];
                    if (op == '|')
                        v = invert ? v - u : v + u;
                    else if (op == '&')
                        u = invert ? u - v : u + v;
                    else
                    {
                        Z x = u, y = v;
                        u = x + y;
                        v = x - y;
                        if (invert)
                        {
                            u = u * half;
                            v = v * half;
                        }
                    }
                }
    }
```

点乘并逆变换（接上页同一结构体）：

```
    static Poly multiply(Poly a, Poly b, char op)
    {
        assert(a.size() == b.size());
        transform(a, op);
        transform(b, op);
        for (int i = 0; i < int(a.size()); i++) a[i] = a[i] * b[i];
        transform(a, op, true);
        return a;
    }
};
```

模板题使用：Luogu P4717

题意：给两个长度为 2^m 的数组，分别求按下标 OR、AND、XOR 合并的卷积，模 998244353，按此顺序输出三行。multiply 的最后参数选择运算。

所需模板：ModInt（第 12 页，3.1.2 节）、SetConvolution（第 78 页，4.3.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4717

```
int main()
{
    int m;
    scanf("%d", &m);
    int n = 1 << m;
    using F = SetConvolution<998244353>;
    F::Poly a(n), b(n);
    for (auto &x : a)
    {
        int v;
        scanf("%d", &v);
        x = v;
    }
    for (auto &x : b)
    {
        int v;
        scanf("%d", &v);
        x = v;
    }
    for (char op : string("|&^"))
    {
        auto c = F::multiply(a, b, op);
        for (int s = 0; s < n; s++) printf("%d%c", c[s].v, s + 1 == n ? '\n' : ' ');
    }
}
```

4.3.2 不相交子集卷积

功能：把下标视为集合，求不相交子集卷积 $c_S = \sum_{T \subseteq S} a_T\, b_{S \setminus T}$，即只统计 $i \cup j = S$ 且 $i \cap j = \varnothing$ 的乘积对（模 mod）。

核心思路：按集合大小（popcount，存于 rank）分层：f[k]、g[k] 只保留大小为 k 的下标，各自做 OR 变换（子集和）。$i \cup j = S$ 且 $|i| + |j| = |S|$ 恰等价于 i, j 不相交，所以对每个 k 在变换域逐点累加 $\sum_t f_t \cdot g_{k-t}$，逆 OR 变换后只取 $|S| = k$ 的位置写入答案。

代码依赖：ModInt（第 12 页，3.1.2 节）、SetConvolution（第 78 页，4.3.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：subset_convolution<mod=998244353>(a,b) 返回 c[s]=∑ a[t]b[s xor t]，求和枚举 t⊆s，等价于 i|j=s 且 i&j=0。依赖 SetConvolution，只使用 OR 变换；mod 为 ≥2 的 int 范围整数，允许偶数与合数。两个数组长度相同、非空，N=2^m≤INT_MAX；长度 1 的结果为 a[0]b[0]。按 popcount 分层，层间普通卷积再逆 OR 变换，保留总层数等于目标集合大小的项。O(m²2^m) 时间、O(m2^m) 空间；两组分层数组约占 8(m+1)2^m 字节，另有线性辅助数组。m=20 时仅分层数组约 168 MiB，使用前核对内存。

```
// c[s] = sum_{t subset s} a[t] * b[s xor t].
template <int mod = 998244353>
vector<ModInt<mod>> subset_convolution(const vector<ModInt<mod>> &a,
                                       const vector<ModInt<mod>> &b)
{
    using F = SetConvolution<mod>;
    using Poly = typename F::Poly;
    assert(!a.empty() && a.size() == b.size() && a.size() <= INT_MAX);
    int n = a.size(), m = 0;
    assert((n & (n - 1)) == 0);
    while ((1 << m) < n) m++;
    vector<Poly> f(m + 1, Poly(n)), g(m + 1, Poly(n));
    vector<int> rank(n);
    for (int s = 0; s < n; s++)
    {
        rank[s] = __builtin_popcount(unsigned(s));
        f[rank[s]][s] = a[s];
        g[rank[s]][s] = b[s];
    }
    for (int k = 0; k <= m; k++)
    {
        F::transform(f[k], '|');
        F::transform(g[k], '|');
    }
    Poly answer(n), h(n);
    for (int k = 0; k <= m; k++)
    {
        fill(h.begin(), h.end(), 0);
        for (int i = 0; i <= k; i++)
            for (int s = 0; s < n; s++) h[s] = h[s] + f[i][s] * g[k - i][s];
        F::transform(h, '|', true);
        for (int s = 0; s < n; s++)
            if (rank[s] == k) answer[s] = h[s];
    }
    return answer;
}
```

模板题使用：Luogu P6097

题意：把下标看作集合，求 c[s]=∑ a[t]*b[s⁀]，t 遍历 s 的子集，两部分不相交。输入首项为集合大小 m，数组长度 2^m；本题模数为 1000000009。

所需模板：ModInt（第 12 页，3.1.2 节）、SetConvolution（第 78 页，4.3.1 节）、subset_convolution（第 80 页，4.3.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P6097

```
int main()
{
    int m;
    scanf("%d", &m);
    int n = 1 << m;
    using F = SetConvolution<1000000009>;
    F::Poly a(n), b(n);
    for (auto &x : a)
    {
        int v;
        scanf("%d", &v);
        x = v;
    }
    for (auto &x : b)
    {
        int v;
        scanf("%d", &v);
        x = v;
    }
    auto c = subset_convolution<1000000009>(a, b);
    for (int s = 0; s < n; s++) printf("%d%c", c[s].v, s + 1 == n ? '\n' : ' ');
}
```

4.3.3 集合变换与不相交子集卷积

代码见第 78 页与第 80 页。长度 $N = 2^m$，第 s 项对应集合 s。OR 变换 $\widehat{a}_s = \sum_{t \subseteq s} a_t$，AND 变换求超集和，逆变换改加为减。XOR 变换系数 $(-1)^{\mathrm{popcount}(s\&t)}$，蝶形 $(u, v) \to (u + v, u - v)$，逆变换每层除以 2。OR、AND 可用任意模数；XOR 需 2 可逆，本实现限奇数模数并用 $(\mathrm{mod} + 1)/2$。三类卷积均为变换、逐项相乘、逆变换。

$$c_s = \sum_{t \subseteq s} a_t b_{s \setminus t} \qquad \text{（两集合不相交）}$$

按大小分层 $A_{i,s} = [|s| = i]a_s$，各层 OR 变换后 $H_{k,s} = \sum_{i=0}^{k} \widehat{A}_{i,s} \widehat{B}_{k-i,s}$，逆变换后取 $k = |s|$。时间 $O(m^2 2^m)$，空间 $O(m2^m)$；$m = 20$ 时两组分层数组占 168 MiB。

4.3.4 集合变换：逆变换对模数的要求

OR（子集和）单层 $(u,v)\mapsto(u,u+v)$，AND（超集和）单层 $(u+v,v)$，只需加减，任意模数。XOR 单层 $(u+v,u-v)$，两次等于乘 2，要求奇数模数，逆 2 取 $(p+1)/2$。长度为正的二次幂；不同于子集卷积。

4.4 Chirp Z 变换

4.4.1 Chirp Z：等比点求值

功能：给定多项式 f 与 a,r，一次求出 f 在等比点 $a,ar,ar^2,\dots,ar^{m-1}$ 处的 m 个值（模 NTT 素数），比逐点秦九韶求值快。

核心思路：关键恒等式 $ij=\binom{i+j}{2}-\binom{i}{2}-\binom{j}{2}$，于是 $f(ar^i)=r^{-\binom{i}{2}}\sum_j\left(f_ja^jr^{-\binom{j}{2}}\right)r^{\binom{i+j}{2}}$，成为一次差卷积。`inverse[t]` 存 $r^{-\binom{t}{2}}$，三角数指数由连乘递推，不计算平方级指数。把 $f_ja^jr^{-\binom{j}{2}}$ 逆序放入 `x`，$r^{\binom{t}{2}}$ 放入 `y`，循环卷积后第 $n-1+i$ 项乘 $r^{-\binom{i}{2}}$ 即为答案；循环回绕只落在下标小于 $n-1$ 处，不影响结果。$a=0$、$r=0$ 或 $r=1$ 时点值退化为 $f(a)$ 与 $f(0)$，用秦九韶直接填充。

代码依赖：ModInt（第 12 页，3.1.2 节）、NttConvolution（第 73 页，4.2.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：chirp_z(f,a,r,m) 返回 f(a*r^i)，i=0..m-1；f 为升幂系数数组，a、r 为 ModInt<mod>。m≥0、f.size()≤INT_MAX；空 f 表示零多项式，m=0 返回空。约定 r^0=1，因此 r=0 时首项为 f(a)、其余为 f(0)；a=0 或 r=1 也单独处理。普通分支依赖 NTT 素数及原根，非空长度 n 要求 n+m-1≤NTT::max_size。利用三角数指数与循环卷积，避免平方指数的整数溢出；O((n+m) log(n+m)) 时间、O(n+m) 空间，特殊分支 O(n+m)。默认模板参数 mod=998244353、primitive=3。输出顺序允许重复点。

```
template <int mod = 998244353, int primitive = 3>
vector<ModInt<mod>>
chirp_z(const vector<ModInt<mod>> &f, ModInt<mod> a, ModInt<mod> r, int m)
{
    using N = NttConvolution<mod, primitive>;
    using Z = typename N::Z;
    using Poly = typename N::Poly;
    assert(m >= 0 && f.size() <= INT_MAX);
    if (m == 0 || f.empty()) return Poly(m);
    if (a.v == 0 || r.v == 0 || r.v == 1)
    {
        Poly answer(m, f[0]);
        Z value = 0;
        for (int i = int(f.size()) - 1; i >= 0; i--) value = value * a + f[i];
        answer[0] = value;
        if (r.v == 1) fill(answer.begin(), answer.end(), value);
        return answer;
    }
    assert(f.size() + m - 1 <= N::max_size);
    int n = f.size(), size = 1;
    while (size < n + m - 1) size *= 2;
    Poly x(size), y(size), inverse(max(n, m));
    Z power = 1, triangle = 1, ri = r.inv();
    for (auto &v : inverse)
    {
        v = triangle;
        triangle = triangle * power;
        power = power * ri;
    }
```

构造循环卷积并取点值（接上页同一函数）：

```
    power = 1;
    for (int j = 0; j < n; j++)
    {
        x[n - 1 - j] = f[j] * power * inverse[j];
        power = power * a;
    }
    power = 1;
    triangle = 1;
    for (int j = 0; j < n + m - 1; j++)
    {
        y[j] = triangle;
        triangle = triangle * power;
        power = power * r;
    }
    N::ntt(x);
    N::ntt(y);
    for (int j = 0; j < size; j++) x[j] = x[j] * y[j];
    N::ntt(x, true);
    Poly answer(m);
    for (int i = 0; i < m; i++) answer[i] = x[n - 1 + i] * inverse[i];
    return answer;
}
```

模板题使用：Library Checker multipoint_evaluation_on_geometric_sequence

题意：给多项式 f 的升幂系数及 a、r，依次输出 f(a*r^i)，i=0..m-1，模 998244353。r=0 时首项仍为 f(a)，后续各项为 f(0)。

所需模板：ModInt（第 12 页，3.1.2 节）、NttConvolution（第 73 页，4.2.1 节）、chirp_z（第 82 页，4.4.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/multipoint_evaluation_on_geometric_sequence

```
int main()
{
    int n, m, a, r;
    scanf("%d%d%d%d", &n, &m, &a, &r);
    using Z = ModInt<998244353>;
    vector<Z> f(n);
    for (auto &x : f)
    {
        int v;
        scanf("%d", &v);
        x = v;
    }
    auto answer = chirp_z(f, Z(a), Z(r), m);
    for (int i = 0; i < m; i++) printf("%d%c", answer[i].v, i + 1 == m ? '\n' : ' ');
}
```

4.5 多项式牛顿迭代

4.5.1 形式幂级数求逆

功能：给定常数项非零的形式幂级数 $a(x)$，求 $b(x)$ 使 $a(x)b(x) \equiv 1 \pmod{x^n}$，返回前 n 项（模 998244353）。

核心思路：牛顿迭代：已知 $b \bmod x^k$ 时，$b \leftarrow b - b(ab-1)$ 把精度翻倍到 $2k$，初值为 a_0^{-1}。实现只计算新增的高半段：取 a 的前 $2k$ 项 `f` 与当前 b 做长度 $2k$ 的循环卷积，真实乘积低 k 项为 $1, 0, \ldots, 0$，而回绕只污染低 k 项，于是清零低半段后剩下的就是 $ab-1$ 的有效部分；再乘一次 b（复用其频域值 `g`），第 k 到 $m-1$ 项取负即为 b 新增的系数。

代码依赖：NttConvolution（第 73 页，4.2.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：FpsInverse::inverse(a,n) 返回 a 的逆级数前 n 项，模数固定 998244353。要求 $1 \le n \le 2^{23}$、a 非空且 a[0]≠0；a.size() 在 int 范围内，不足项视为零，多余项只取需要前缀。继承 NttConvolution<998244353> 的 Poly、Z、multiply。Newton 公式 b←b(2-ab) 只更新新增高半段：长度 2k 循环卷积的回绕只污染低 k 项，清除低半段后再乘 b，高半段即修正项。每轮五次长度 2k 变换，复用 b 的频域值。O(n log n) 时间、O(n) 工作空间；单位根缓存另按 NTT 约定。

```
struct FpsInverse : NttConvolution<998244353>
{
    static constexpr int mod = 998244353;

    static Poly inverse(const Poly &a, int n)
    {
        assert(n > 0 && n <= max_size && !a.empty() && a[0].v);
        Poly b{a[0].inv()};
        while ((int)b.size() < n)
        {
            int k = b.size(), len = 2 * k;
            int m = min(n, len);
            Poly f(a.begin(), a.begin() + min(m, (int)a.size()));
            Poly g = b;
            f.resize(len);
            g.resize(len);
            ntt(f);
            ntt(g);
            for (int i = 0; i < len; i++)
                f[i] *= g[i];
            ntt(f, true);
            // Only the high half of a*b is free of cyclic wraparound.
            fill(f.begin(), f.begin() + k, Z(0));
            ntt(f);
            for (int i = 0; i < len; i++)
                f[i] *= g[i];
            ntt(f, true);
            b.resize(m);
            for (int i = k; i < m; i++)
                b[i] = Z(0) - f[i];
        }
        return b;
    }
};
```

模板题使用：Library Checker inv_of_formal_power_series

题意：给出常数项非零的形式幂级数，求其乘法逆的前 n 项，模 998244353。系数按升幂输入，题目保证已经取模。

所需模板：ModInt（第 12 页，3.1.2 节）、NttConvolution（第 73 页，4.2.1 节）、FpsInverse（第 84 页，4.5.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/inv_of_formal_power_series

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    using F = FpsInverse;
    int n;
    cin >> n;
    F::Poly a(n);
    for (auto &x : a) cin >> x.v;
    auto b = F::inverse(a, n);
    for (int i = 0; i < n; i++) cout << b[i].v << (i + 1 == n ? '\n' : ' ');
}
```

4.5.2 多项式带余除法

功能：给定多项式 a, b（$b \neq 0$），求商 q 与余式 r，使 $a = qb + r$ 且 $\deg r < \deg b$（模 998244353）；另附去掉高次零系数的 `trim`。

核心思路：设 $\deg a = n$、$\deg b = m$，则商的次数为 $n - m$。记系数反转 $a^R(x) = x^n a(1/x)$，有 $a^R = q^R b^R + x^{n-m+1}(\cdots)$，故 $q^R \equiv a^R (b^R)^{-1} \pmod{x^{n-m+1}}$。代码取 a 反转后的前 $k = n - m + 1$ 项，与 b^R 的逆级数（`inverse`）相乘、截断、再反转得到 q；最后 $r = a - qb$ 只保留低 m 项并去高次零。

代码依赖：FpsInverse（第 84 页，4.5.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：PolynomialDivision::divide(a,b) 返回 pair<Poly,Poly>{商, 余式}，模数固定 998244353，系数从低次到高次。输入按值传入；先删除高次端零系数，零多项式统一返回空 vector；b 不可为零。支持零被除式、常数除数及 deg(a)<deg(b)。返回 a=q*b+r，deg(r)<deg(b)，商和余式都去掉高次零。依赖 FpsInverse 与 NttConvolution；反转后求逆得到商，再计算余式。输入各长度 $\leq 2^{22}$；令 N=max(a.size(),b.size())，时间 O(N log(N+1))，额外空间 O(N)。trim(a) 是可直接使用的去高次零工具。P4512 的固定长度输出需在使用处 resize 补零。

```
struct PolynomialDivision : FpsInverse
{
    static void trim(Poly &a)
    {
        while (!a.empty() && a.back().v == 0) a.pop_back();
    }

    static pair<Poly, Poly> divide(Poly a, Poly b)
    {
        assert(a.size() <= max_size / 2 && b.size() <= max_size / 2);
        trim(a);
        trim(b);
        assert(!b.empty());
        if (a.size() < b.size()) return {{}, a};
        int k = a.size() - b.size() + 1;
        Poly ar(a.rbegin(), a.rbegin() + k);
        Poly br(b.rbegin(), b.rend());
        br.resize(min(k, (int)br.size()));
        auto q = multiply(ar, inverse(br, k));
        q.resize(k);
        reverse(q.begin(), q.end());
        auto product = multiply(q, b);
        for (int i = 0; i < (int)a.size(); i++) a[i] -= product[i];
        a.resize(b.size() - 1);
        trim(a);
        return {q, a};
    }
};
```

模板题使用：Luogu P4512 多项式除法

题意：输入两个多项式的次数与系数，输出商的 n-m+1 项和余式的 m 项；模板返回的商、余式均已去高次零，打印前按题目要求补零。

所需模板：ModInt（第 12 页，3.1.2 节）、NttConvolution（第 73 页，4.2.1 节）、FpsInverse（第 84 页，4.5.1 节）、PolynomialDivision（第 86 页，4.5.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4512

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    using P = PolynomialDivision;
    P::Poly a(n + 1), b(m + 1);
    for (auto &x : a)
    {
        int value;
        cin >> value;
        x = value;
    }
    for (auto &x : b)
    {
        int value;
        cin >> value;
        x = value;
    }
    auto [q, r] = P::divide(a, b);
    q.resize(n - m + 1);
    r.resize(m);
    for (auto x : q) cout << x.v << ' ';
    cout << '\n';
    for (auto x : r) cout << x.v << ' ';
    cout << '\n';
}
```

4.5.3 多项式带余除法：先反转再求逆

`PolynomialDivision::divide(a,b)` 返回规范化的商、余式，零多项式为空向量；除数去高次零后须非零。设 $A = \deg a$、$B = \deg b$、$k = A - B + 1$：

$$\operatorname{rev}(q) = \operatorname{rev}(a)\ \operatorname{rev}(b)^{-1} \pmod{x^k}, \qquad r = a - qb.$$

定长输出需自行补零。各长度 $\le 2^{22}$，$O(N \log(N+1))$ 时间、$O(N)$ 空间。

4.6 多项式多点求值 | 快速插值

4.6.1 任意点多项式多点求值

问题描述：给定模 998244353 的多项式 f 和 m 个任意点 $x_0, \ldots, x_{m-1}$（可以重复），求全部 $f(x_i)$。点集固定后，同一棵树可对多个不同的 f 反复求值。

核心思路：利用 $f(x_i) = f \bmod (X - x_i)$。构造时在线段树形的乘积树上自底向上建出 p[u]$= \prod_{l \le i < r}(X - x_i)$：叶子为 $X - x_l$，内部结点用 multiply 合并左右儿子。eval 自顶向下传递余式：进入结点时，若当前多项式次数不低于 p[u] 就对它取模，再分别递归左右儿子。余式已为零多项式时直接返回（答案数组初值为 0）；只剩常数时把它填进整段点的答案，不再下探。

代码依赖：PolynomialDivision（第 86 页，4.5.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：固定模数 998244353，Poly 为升幂系数 vector<Z>。MultipointEvaluation(x) 建立点集乘积树，evaluate(f) 按输入点顺序返回 f(x[i])；允许重复点、零点、空点集、空或尾零多项式，返回长度始终等于点数。构造复制点值，查询按值接收 f，不改变树或调用者的左值，可多次复用；不再使用 f 时可 move 传入。系数个数 n≤2^22，点数 m<2^22，且内存可用；本地最大规模证据按报告列出，不表示已实测此绝对上限。每结点存首一多项式，查询将余式下传，叶值就是答案；仅求值，不包含快速插值。建树 O(m $\log^2$(m+1))，查询 O(n log(n+1)+m $\log^2$(m+1))，常驻空间 O(m log(m+1))、含临时数组 O(n+m log(m+1))，递归深度 O(log(m+1))。依赖 PolynomialDivision/FpsInverse/NTT 及模整数。

```
struct MultipointEvaluation : PolynomialDivision
{
    int n;
    vector<Poly> p;

    MultipointEvaluation(const Poly &x)
    {
        assert(x.size() < max_size / 2);
        n = x.size();
        p.resize(4 * max(1, n));
        if (n) build(1, 0, n, x);
    }

    void build(int u, int l, int r, const Poly &x)
    {
        if (r - l == 1)
        {
            p[u] = {Z(0) - x[l], 1};
            return;
        }
        int m = (l + r) / 2;
        build(2 * u, l, m, x);
        build(2 * u + 1, m, r, x);
        p[u] = multiply(p[2 * u], p[2 * u + 1]);
    }
```

余式下传与求值入口（接上页同一结构体）：

```
    void eval(int u, int l, int r, Poly f, Poly &a) const
    {
        if (f.size() >= p[u].size()) f = divide(move(f), p[u]).second;
        if (f.empty()) return;
        if (f.size() == 1)
        {
            fill(a.begin() + l, a.begin() + r, f[0]);
            return;
        }
        int m = (l + r) / 2;
        eval(2 * u, l, m, f, a);
        eval(2 * u + 1, m, r, move(f), a);
    }

    Poly evaluate(Poly f) const
    {
        assert(f.size() <= max_size / 2);
        trim(f);
        Poly a(n);
        if (n) eval(1, 0, n, move(f), a);
        return a;
    }
};
```

模板题使用：Library Checker Multipoint Evaluation

题意：给出 n 项升幂系数和 m 个任意横坐标，按输入顺序输出全部 f(x[i])，模 998244353。1≤n,m≤131072，系数和点均已取模，最高次系数非零；横坐标可以重复。构造一次乘积树后调用 evaluate；同一树可以继续求其他多项式的点值。

所需模板：ModInt（第 12 页，3.1.2 节）、NttConvolution（第 73 页，4.2.1 节）、FpsInverse（第 84 页，4.5.1 节）、PolynomialDivision（第 86 页，4.5.2 节）、MultipointEvaluation（第 88 页，4.6.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/multipoint_evaluation

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    using P = MultipointEvaluation;
    P::Poly f(n), x(m);
    for (auto &a : f)
    {
        int v;
        cin >> v;
        a = v;
    }
    for (auto &a : x)
    {
        int v;
        cin >> v;
        a = v;
    }
    auto a = P(x).evaluate(f);
    for (int i = 0; i < m; i++) cout << a[i].v << (i + 1 == m ? '\n' : ' ');
}
```

4.6.2 任意点多项式快速插值

问题描述：给定 n 个横坐标互异的点 (x_i, y_i)，在模 998244353 下求次数小于 n 的唯一多项式 f，使 $f(x_i) = y_i$，返回其系数。

核心思路：Lagrange 插值写成

$$f(X) = \sum_i \frac{y_i}{P'(x_i)} \cdot \frac{P(X)}{X - x_i},$$

其中 $P(X) = \prod_i (X - x_i)$。先用 `MultipointEvaluation` 建乘积树，根 `tree.p[1]` 就是 P；逐项求导得 P'，多点求值得到各 $P'(x_i)$（出现 0 说明横坐标重复），权重 $w_i = y_i / P'(x_i)$。然后在同一棵树上自底向上合并：叶子返回常数 w_l，内部结点返回 “左半结果乘右儿子乘积，加右半结果乘左儿子乘积”，根处即为 f。

代码依赖：MultipointEvaluation（第 88 页，4.6.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：polynomial_interpolation(x,y) 返回模 998244353 下次数 <m 的唯一插值多项式，升幂 vector 恰好 m 项并保留高次零。optional 为空表示长度不符或归一化横坐标重复；两个空数组返回有值的空 vector，不改输入。复用 MultipointEvaluation 乘积树及导数求值，按 y/P'(x) 递归合并；重复点只在独立求值接口合法。m≤$2^{22}-1$ 是既有除法/NTT 的统一充分界，官方最大实测 131072。时间 O(m $\log^2$(m+1)+m log p)，p=998244353；单点逆元沿用模幂，不省略 m log p 项。总空间 O(m log(m+1)+1)，递归 O(log(m+1))。

```
optional<MultipointEvaluation::Poly>
polynomial_interpolation(const MultipointEvaluation::Poly &x,
                         const MultipointEvaluation::Poly &y)
{
    using P = MultipointEvaluation;
    using Poly = P::Poly;
    if (x.size() != y.size()) return nullopt;
    assert(x.size() < P::max_size / 2);
    int n = x.size();
    if (!n) return Poly{};
    const P tree(x);
    Poly d(n);
    for (int i = 1; i <= n; i++) d[i - 1] = tree.p[1][i] * i;
    auto w = tree.evaluate(move(d));
    for (auto v : w)
        if (!v.v) return nullopt;
    for (int i = 0; i < n; i++) w[i] = y[i] * w[i].inv();
    auto combine = [&](auto &&self, int u, int l, int r) -> Poly
    {
        if (r - l == 1) return {w[l]};
        int m = (l + r) / 2;
        auto a = self(self, 2 * u, l, m);
        auto b = self(self, 2 * u + 1, m, r);
        a = P::multiply(move(a), tree.p[2 * u + 1]);
        b = P::multiply(move(b), tree.p[2 * u]);
        for (int i = 0; i < r - l; i++) a[i] += b[i];
        return a;
    };
    return combine(combine, 1, 0, n);
}
```

模板题使用：Library Checker Polynomial Interpolation

题意：给出 n 个互异横坐标及对应点值，输出唯一的次数小于 n 的多项式的 n 项升幂系数，模 998244353。1≤n≤131072，坐标与点值均已取模；先以乘积根多项式的导数求权重，再递归合并。接口返回 optional，长度不符或模意义重复横坐标返回空 optional，空输入返回有值的空向量；本题保证输入合法。

所需模板：ModInt（第 12 页，3.1.2 节）、NttConvolution（第 73 页，4.2.1 节）、FpsInverse（第 84 页，4.5.1 节）、PolynomialDivision（第 86 页，4.5.2 节）、MultipointEvaluation（第 88 页，4.6.1 节）、polynomial_interpolation（第 91 页，4.6.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/polynomial_interpolation

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    using Poly = MultipointEvaluation::Poly;
    Poly x(n), y(n);
    for (auto &a : x)
    {
        int v;
        cin >> v;
        a = v;
    }
    for (auto &a : y)
    {
        int v;
        cin >> v;
        a = v;
    }
    auto f = polynomial_interpolation(x, y);
    assert(f);
    for (int i = 0; i < n; i++) cout << (*f)[i].v << (i + 1 == n ? '\n' : ' ');
}
```

4.7 多项式初等函数

4.7.1 形式幂级数开根（含前导零与无解）

功能： 给定形式幂级数 $a(x)$，求 $b(x)$ 使 $b^2 \equiv a \pmod{x^n}$ 的前 n 项（模 998244353），无解时报告；`inverse_sqrt` 求 $a^{-1/2}$ 的前 n 项。

核心思路： 先求逆平方根：常数项用 `mod_sqrt` 取一个模平方根再求逆，之后按牛顿迭代 $b \leftarrow (3b - ab^3)/2$ 倍增。已有 k 项时，在长度 $4k$ 下一次算出 ab^3，回绕只落到低 k 项；新 b 的低 k 项不变，高半段等于 $-\frac{1}{2}(ab^3)$ 的对应项。`sqrt` 先找首个非零项次数 k：k 为奇数或该系数不是二次剩余则无解；否则除掉 x^k 得 `f`，用 $\sqrt{f} = f \cdot f^{-1/2}$ 求根，最后在前面补 $k/2$ 个零。

代码依赖：FpsInverse（第 84 页，4.5.1 节）、mod_sqrt（第 43 页，3.14.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约： FpsSqrt::sqrt(a,n) 返回 optional<Poly>，模 998244353；无解为 nullopt，有解恰好 n 项。$0 \le n \le 2^{22}$、a.size() 在 int 范围内，仅看前 n 项，不足补零；n=0 返回有值空 vector。全零返回全零，首个非零项次数 k 为奇数或系数非二次剩余时无解。否则提出 x^k，首项选较小模平方根，末尾未确定的 k/2 项取零。inverse_sqrt(a,n) 直接求所选根的逆：n>0 时要求常数项非零且为二次剩余，否则 nullopt；n=0 仍返回空。按 $b \leftarrow (3b - ab^3)/2$ 倍增，长度 4k 的循环卷积只回绕到低 k 项，保留旧低半段、更新高半段；sqrt 再乘原级数并移位。不枚举全部根。依赖 mod_sqrt、FpsInverse、NTT；O(n log(n+1)) 时间加一次标量开根的期望 O(log mod)，O(n) 工作空间，单位根缓存另按 NTT 约定。P5205 的 a[0]=1 得到首项 1。

```
struct FpsSqrt : FpsInverse
{
    static optional<Poly> inverse_sqrt(const Poly &a, int n)
    {
        assert(n >= 0 && n <= max_size / 2);
        if (n == 0)
            return Poly{};
        if (a.empty() || a[0].v == 0)
            return nullopt;
        auto roots = mod_sqrt(a[0].v, mod);
        if (roots.empty())
            return nullopt;
        Poly b{Z(roots[0]).inv()};
        Z half = Z(2).inv();
        while ((int)b.size() < n)
        {
            int k = b.size(), len = 4 * k;
            int m = min(n, 2 * k);
            Poly f(a.begin(), a.begin() + min(m, (int)a.size()));
            Poly g = b;
            f.resize(len);
            g.resize(len);
            ntt(f);
            ntt(g);
            for (int i = 0; i < len; i++)
                f[i] *= g[i] * g[i] * g[i];
            ntt(f, true);
            // b <- (3*b - a*b^3)/2; cyclic wrap only reaches below k.
            b.resize(m);
            for (int i = k; i < m; i++)
                b[i] = Z(0) - f[i] * half;
        }
        return b;
    }
```

平方根与前导零处理（接上页同一结构体）：

```
    static optional<Poly> sqrt(const Poly &a, int n)
    {
        assert(n >= 0 && n <= max_size / 2);
        int k = 0;
        while (k < n && (k >= (int)a.size() || a[k].v == 0))
            k++;
        if (k == n)
            return Poly(n);
        if (k & 1)
            return nullopt;
        int target = n - k;
        Poly f(a.begin() + k, a.begin() + min(n, (int)a.size()));
        auto inverse = inverse_sqrt(f, target);
        if (!inverse)
            return nullopt;
        Poly b = multiply(f, *inverse);
        b.resize(target);
        b.insert(b.begin(), k / 2, Z(0));
        b.resize(n);
        return b;
    }
};
```

模板题使用：Luogu P5205 多项式开根

题意：输入 n 项、保证 a[0]=1，输出平方根的前 n 项，首项取较小的 1。通用接口也可用于无解输入，须先检查 optional。

所需模板：ModInt（第 12 页，3.1.2 节）、NttConvolution（第 73 页，4.2.1 节）、FpsInverse（第 84 页，4.5.1 节）、mod_sqrt（第 43 页，3.14.1 节）、FpsSqrt（第 93 页，4.7.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5205

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    FpsSqrt::Poly a(n);
    for (auto &x : a)
    {
        int value;
        cin >> value;
        x = value;
    }
    auto b = FpsSqrt::sqrt(a, n);
    if (!b)
    {
        cout << -1 << '\n';
        return 0;
    }
    for (auto x : *b) cout << x.v << ' ';
    cout << '\n';
}
```

4.7.2 形式幂级数微积分、对数与指数

功能： 形式幂级数的基本运算：逐项求导 `derivative`、常数项为零的积分 `integral`、常数项为 1 时的对数 $\ln a$，以及常数项为 0 时的指数 $\exp a$，后两者返回前 n 项（模 998244353）。

核心思路： 积分所需的 $1/i$ 用线性递推 $i^{-1} = -\lfloor p/i \rfloor \cdot (p \bmod i)^{-1}$ 求出。$\ln a = \int a'/a$：截取前 n 项，导数乘逆级数（`inverse`）后积分。exp 用牛顿迭代 $b \leftarrow b(1 - \ln b + a)$，从 $b = 1$ 开始每轮精度翻倍：求 $\ln b$ 的前 m 项，构造 $a - \ln b + 1$，再与 b 相乘并截断到 m 项。

代码依赖：FpsInverse（第 84 页，4.5.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约： FpsFunctions 继承 FpsInverse，复用同一 NTT 和求逆。derivative(a) 形式求导；integral(a) 返回常数项为零的积分，要求 a.size()<998244353。log(a,n) 要求 n>0、a 非空、a[0]=1；exp(a,n) 要求 n>0、a 为空或 a[0]=0。返回前 n 项，不足输入项视为零，超过 n 的尾项不参与计算。内部每次卷积结果长度须 $\le 2^{23}$；$n \le 2^{22}$ 是统一充分界，输入数组长度须在 int 范围内。微积分 O(a.size())；log/exp 为 O(n log(n+1)) 时间和 O(n) 辅助空间。

```
struct FpsFunctions : FpsInverse
{
    static Poly derivative(const Poly &a)
    {
        Poly b;
        for (int i = 1; i < (int)a.size(); i++) b.push_back(a[i] * i);
        return b;
    }

    static Poly integral(const Poly &a)
    {
        assert(a.size() < mod);
        Poly b(a.size() + 1);
        vector<Z> inv(a.size() + 1);
        if (!a.empty()) inv[1] = 1;
        for (int i = 2; i <= (int)a.size(); i++)
            inv[i] = Z(mod - mod / i) * inv[mod % i];
        for (int i = 0; i < (int)a.size(); i++) b[i + 1] = a[i] * inv[i + 1];
        return b;
    }

    static Poly log(const Poly &a, int n)
    {
        assert(n > 0 && !a.empty() && a[0].v == 1);
        Poly f(a.begin(), a.begin() + min(n, (int)a.size()));
        auto b = multiply(derivative(f), inverse(f, n));
        b.resize(n - 1);
        return integral(b);
    }
```

```
    static Poly exp(const Poly &a, int n)
    {
        assert(n > 0 && (a.empty() || a[0].v == 0));
        Poly b{1};
        while ((int)b.size() < n)
        {
            int m = min(n, 2 * (int)b.size());
            auto t = log(b, m);
            for (int i = 0; i < m; i++) t[i] = (i < (int)a.size() ? a[i] : Z(0)) - t[i];
            t[0] = t[0] + 1;
            b = multiply(b, t);
            b.resize(m);
        }
        return b;
    }
};
```

模板题使用：Library Checker log_of_formal_power_series

题意：给定常数项为 1 的形式幂级数，求其形式对数的前 n 项，模 998244353。系数按升幂输入且已取模，结果常数项为 0。

所需模板：ModInt（第 12 页，3.1.2 节）、NttConvolution（第 73 页，4.2.1 节）、FpsInverse（第 84 页，4.5.1 节）、FpsFunctions（第 96 页，4.7.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/log_of_formal_power_series

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    using F = FpsFunctions;
    int n;
    cin >> n;
    F::Poly a(n);
    for (auto &x : a) cin >> x.v;
    auto b = F::log(a, n);
    for (int i = 0; i < n; i++) cout << b[i].v << (i + 1 == n ? '\n' : ' ');
}
```

4.7.3 形式幂级数快速幂（十进制大指数）

功能：给定形式幂级数 $a(x)$ 与以十进制字符串给出的非负整数 k（可能极大），求 $a(x)^k \bmod x^n$ 的前 n 项（模 998244353）。

核心思路：设首个非零项为 $c\,x^s$，则 $a = c\,x^s\,b$ 且 b 的常数项为 1，$a^k = c^k x^{sk} \exp(k \ln b)$。读指数时同时保留 $k \bmod p$（乘进 $\ln b$）、$k \bmod (p-1)$（`phi`，由费马小定理用于 c^k）以及截到 n 的真实值 `cap`（判断位移 sk 是否越界）。若 $sk \geq n$ 结果全零；否则只需 b 的前 $n - sk$ 项，做 log、乘 k、exp、乘 c^k 后平移 sk 位。$k = 0$ 时直接返回 1。

思路中的情况	代码中的处理	行
读入大指数	逐位维护 $k \bmod p$、$k \bmod (p-1)$ 及饱和于 n 的 `cap`	14
指数为 0	返回常数 1	18
位移 $sk \geq n$ 或级数全零	直接返回全零	25
提出 $c\,x^s$	截取 b 并整体乘 c^{-1}，使常数项为 1	29
计算 $\exp(k \ln b)$	对数乘 $k \bmod p$ 后取指数	32
乘回 $c^k x^{sk}$	乘 c^{phi} 并写到第 `shift` 项之后	34

代码依赖：FpsFunctions（第 96 页，4.7.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：FpsPower::power(a,exponent,n) 返回 a 的非负整数次幂模 x^n 的前 n 项，固定模数 998244353。exponent 为非空十进制数字 string_view，允许前导零；整数指数可传 to_string(k)。$0 \leq n \leq 2^{22}$，a.size() 须在 int 范围内；短输入补零、长输入只看前 n 项。n=0 返回空，零次幂（含零多项式的零次幂）返回 1。对首个非零项次数 s，若真实 s*k≥n 则返回全零；否则提取常数 c，计算 x^(s*k)*c^k*exp(k*log(a/(c*x^s)))。指数分别保留模 p、模 p-1 和截到 n 的值；不可仅用 k mod p 判零或算位移。依赖 FpsFunctions、FpsInverse、NTT。指数位数 L，时间 O(L+n log(n+1))、额外空间 O(n)，不支持负指数或小数指数。

```
struct FpsPower : FpsFunctions
{
    static Poly power(const Poly &a, string_view exponent, int n)
    {
        assert(n >= 0 && n <= max_size / 2 && !exponent.empty());
        long long k = 0, phi = 0;
        int cap = 0;
        for (char c : exponent)
        {
            assert('0' <= c && c <= '9');
            int digit = c - '0';
            k = (k * 10 + digit) % mod;
            phi = (phi * 10 + digit) % (mod - 1);
            cap = min(n, cap * 10 + digit);
        }
        Poly result(n);
        if (n == 0) return result;
        if (cap == 0)
        {
            result[0] = 1;
            return result;
        }
        int first = 0;
        while (first < n && (first >= (int)a.size() || a[first].v == 0)) first++;
        if (first == n || (first && cap > (n - 1) / first)) return result;
        int shift = first * cap, m = n - shift;
        Poly b(a.begin() + first, a.begin() + min(first + m, (int)a.size()));
        Z leading = a[first], inverse_leading = leading.inv();
        for (auto &x : b) x *= inverse_leading;
        auto f = log(b, m);
        for (auto &x : f) x *= Z(k);
        auto t = exp(f, m);
        Z scale = leading.pow(phi);
        for (int i = 0; i < m; i++) result[i + shift] = t[i] * scale;
        return result;
    }
};
```

模板题使用：Luogu P5273 多项式幂函数（加强版）

题意：输入 n 和十进制大指数 k，输出 a^k 的前 n 项；k 必须用 string 读入，不能只读 k mod p。允许前导零和 k=0。本程序同样适用于 P5245（a[0]=1）及 Library Checker Pow of Formal Power Series，三个入口均已本地验证。

所需模板：ModInt（第 12 页，3.1.2 节）、NttConvolution（第 73 页，4.2.1 节）、FpsInverse（第 84 页，4.5.1 节）、FpsFunctions（第 96 页，4.7.2 节）、FpsPower（第 98 页，4.7.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5273

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    string k;
    cin >> n >> k;
    FpsPower::Poly a(n);
    for (auto &x : a)
    {
        int value;
        cin >> value;
        x = value;
    }
    auto b = FpsPower::power(a, k, n);
    for (auto x : b) cout << x.v << ' ';
    cout << '\n';
}
```

4.7.4 级数开根：前导零和自由尾项

`FpsSqrt::sqrt(a,n)` 只用前 n 项。全零返回全零；首个非零项次数 k 为奇数或其系数非二次剩余则无解。否则 $a = x^k h$，迭代 $b \leftarrow (b + h/b)/2$ 求 h 的根前 $n-k$ 项后左移 $k/2$；末尾 $k/2$ 个自由系数取零，故解一般不止两个。返回 optional，$n = 0$ 返回空向量。$n \le 2^{22}$，$O(n \log(n+1))$ 加一次模开根，空间 $O(n)$。

4.7.5 ln 与 exp：只处理所需前缀

log 要求 $a_0 = 1$，$(\log a)' = a'/a$；exp 要求 $a_0 = 0$，$b \leftarrow b(1 + a - \log b)$ 倍增。对数输入先截到 n 项。积分要求 $|a| < 998244353$。$n \le 2^{22}$，$O(n \log(n+1))$ 时间、$O(n)$ 空间。

4.7.6 幂函数的大指数：保留三份指数信息

`FpsPower::power(a,exponent,n)` 的指数为十进制字符串，求模 x^n 的非负整数次幂；$n = 0$ 返回空，零次幂返回 1（含 0^0）。$k > 0$ 时设 $a = x^s ch$、$h(0) = 1$、$c \neq 0$；全零或 $sk \ge n$ 时全零，否则

$$a^k = x^{sk} c^k \exp(k \log h) \pmod{x^n}.$$

$k \bmod p$ 用于对数系数，$k \bmod (p-1)$ 用于 c^k，$\min(k, n)$ 用于零次幂与位移（要求 $n \le 2^{22} < p$）。判 $sk \ge n$ 用 $k > (n-1)/s$ 防溢出。指数位数 L，$O(L + n \log(n+1))$ 时间、$O(n)$ 空间。

4.8 常系数齐次线性递推

4.8.1 特征多项式求线性递推第 k 项

问题描述：已知 k 阶常系数齐次线性递推 $a_i = \sum_{j=1}^{k} c_{j-1} a_{i-j}$ 及初值 $a_0, \ldots, a_{k-1}$，求第 n 项（n 很大），模数可为合数。

核心思路：设特征多项式 $P(x) = x^k - \sum_{j=1}^{k} c_{j-1} x^{k-j}$。把 a_i 看作单项式 x^i 的线性函数，递推说明 x^i 可以按 P 降次而值不变，故若 $x^n \equiv \sum_{i<k} r_i x^i \pmod P$，则 $a_n = \sum_i r_i a_i$。`multiply` 做朴素多项式乘法后，从最高次往下用 $x^i = \sum_j c_{j-1} x^{i-j}$ 消去次数不小于 k 的项；以此快速幂求 $x^n \bmod P$（$k = 1$ 时 x 已降为常数 c_0），最后与初值点乘。

接口契约：recurrence_nth(init,c,n,p) 使用 0-based 下标，n 为 unsigned long long。init 与 c 长度均为阶数 k；a[i]=∑ c[j-1]a[i-j]。2≤p≤INT_MAX，数值均在 [0,p)，本函数允许合数模数。空系数约定全零序列。计算 x^n 对特征多项式取余再与初值点乘，O(k² log(n+1)) 时间、O(k) 空间；不依赖 BM 或其他模板。较大阶数可使用本库 BostanMori::recurrence，须满足其 NTT 素数模数与容量条件；本函数允许合数模数。

```cpp
// init has exactly k entries; c[j-1] multiplies a[n-j]. Modulus need not be prime.
int recurrence_nth(const vector<int> &init,
                   const vector<int> &c,
                   unsigned long long n,
                   int p = 998244353)
{
    assert(p >= 2 && init.size() == c.size());
    int k = c.size();
    if (!k) return 0;
    for (int x : init) assert(0 <= x && x < p);
    for (int x : c) assert(0 <= x && x < p);
    if (n < (unsigned)k) return init[n];
    auto multiply = [&](const vector<int> &a, const vector<int> &b)
    {
        vector<int> t(2 * k - 1);
        for (int i = 0; i < k; i++)
        {
            for (int j = 0; j < k; j++) t[i + j] = (t[i + j] + 1LL * a[i] * b[j]) % p;
        }
        for (int i = 2 * k - 2; i >= k; i--)
        {
            for (int j = 1; j <= k; j++)
                t[i - j] = (t[i - j] + 1LL * t[i] * c[j - 1]) % p;
        }
        t.resize(k);
        return t;
    };
    vector<int> answer(k), x(k);
    answer[0] = 1;
    if (k == 1)
        x[0] = c[0];
    else
        x[1] = 1;
    while (n)
```

```
    {
        if (n & 1) answer = multiply(answer, x);
        x = multiply(x, x);
        n >>= 1;
    }
    long long value = 0;
    for (int i = 0; i < k; i++) value = (value + 1LL * answer[i] * init[i]) % p;
    return value;
}
```

模板题使用：Luogu P5487

题意：给出 n 个初值，恢复唯一的最短递推并求第 m 项，n≤10000、n<m≤10^9、阶数 ≤5000，模 998244353。BM 返回 c[j-1] 乘 a[i-j]，首行仅输出系数、不带阶数；裁取恰好 c.size() 个初值后调用 recurrence_nth。全零序列空递推仍输出空首行和第二行 0。复杂度 O(nk+k^2log m)，O(n+k) 空间。

所需模板：berlekamp_massey（第 181 页，11.1 节）、recurrence_nth（第 101 页，4.8.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5487

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    unsigned long long m;
    cin >> n >> m;
    vector<int> s(n);
    for (auto &x : s)
    {
        cin >> x;
        x %= 998244353;
        if (x < 0) x += 998244353;
    }
    auto c = berlekamp_massey(s);
    for (int i = 0; i < (int)c.size(); i++)
    {
        if (i) cout << ' ';
        cout << c[i];
    }
    cout << '\n';
    vector<int> init(s.begin(), s.begin() + c.size());
    cout << recurrence_nth(init, c, m) << '\n';
}
```

4.8.2 快速递推与有理生成函数单项

问题描述：求有理生成函数的单项系数 $[x^n]\frac{p(x)}{q(x)}$；以及给定 k 阶常系数齐次线性递推与初值，求第 n 项（模 NTT 素数）。

核心思路：分子分母同乘 $q(-x)$：$\frac{p(x)}{q(x)}=\frac{p(x)q(-x)}{q(x)q(-x)}$，新分母只含偶次项，记为 $V(x^2)$，所以只需分子中次数与 n 同奇偶的项，记为 $x^{n \bmod 2}U(x^2)$，于是 $[x^n]\frac{p}{q}=[x^{\lfloor n/2\rfloor}]\frac{U}{V}$。`coefficient` 每轮两次 NTT 乘法后抽取奇偶项、$n$ 减半，直到 $n=0$ 时答案为 p_0/q_0。`recurrence` 取 $q=1-\sum_i c_i x^{i+1}$、$p=(\text{init}\cdot q) \bmod x^k$，转化为上述问题。

代码依赖：NttConvolution（第 73 页，4.2.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约： BostanMori<mod=998244353,primitive=3>，依赖参数化 NTT；素数模数与原根条件同 NttConvolution。Poly 为升幂系数 vector<Z>。coefficient(p,q,n) 返回 [x^n]p(x)/q(x)，q 非空且 q[0]≠0，p 可空或次数不小于 q；令 d=max(p.size(),q.size())，要求 2d-1≤NTT::max_size。recurrence(init,c,n) 返回 a[n]，从下标 k 起 a[i]=c[0]a[i-1]+⋯+c[k-1]a[i-k]，init、c 长度均为 k；k=0 约定全零。非初值查询要求 2k+1≤NTT::max_size。n 支持完整 unsigned long long。每轮两次卷积后抽取奇偶项，O(d log d log(n+1)) 时间、O(d) 空间；递推接口 d=k+1。

```
template <int mod = 998244353, int primitive = 3> struct BostanMori
{
    using N = NttConvolution<mod, primitive>;
    using Z = typename N::Z;
    using Poly = typename N::Poly;
    using U = unsigned long long;

    // Coefficient of x^n in p(x) / q(x), with q[0] != 0.
    static Z coefficient(Poly p, Poly q, U n)
    {
        assert(!q.empty() && q[0].v != 0);
        assert(max(p.size(), q.size()) <= (N::max_size + 1ULL) / 2);
        while (n && !p.empty())
        {
            Poly r = q;
            for (int i = 1; i < int(r.size()); i += 2) r[i] = Z(0) - r[i];
            Poly a = N::multiply(p, r);
            Poly b = N::multiply(q, r);
            p.clear();
            q.clear();
            for (int i = n & 1; i < int(a.size()); i += 2) p.push_back(a[i]);
            for (int i = 0; i < int(b.size()); i += 2) q.push_back(b[i]);
            n >>= 1;
        }
        return p.empty() ? Z(0) : p[0] / q[0];
    }
```

从初值构造递推生成函数（接上页同一结构体）：

```
    // a[i] = c[0]*a[i-1] + ... + c[k-1]*a[i-k], for i >= k.
    static Z recurrence(const Poly &init, const Poly &c, U n)
    {
        assert(init.size() == c.size());
        int k = c.size();
        if (k == 0) return 0;
        if (n < init.size()) return init[n];
        assert(k + 1ULL <= (N::max_size + 1ULL) / 2);
        Poly q(k + 1);
        q[0] = 1;
        for (int i = 0; i < k; i++) q[i + 1] = Z(0) - c[i];
        Poly p = N::multiply(init, q);
        p.resize(k);
        return coefficient(p, q, n);
    }
};
```

模板题使用：Luogu P4723

题意：给 k 个递推系数和 a[0..k-1]，求 a[n] 模 998244353。c[0] 乘前一项，c[k-1] 乘前 k 项；输入先系数后初值，recurrence 参数先初值后系数。通过 ModInt 赋值归一化负数。

所需模板：ModInt（第 12 页，3.1.2 节）、NttConvolution（第 73 页，4.2.1 节）、BostanMori（第 102 页，4.8.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4723

```
int main()
{
    unsigned long long n;
    int k;
    scanf("%llu%d", &n, &k);
    using B = BostanMori<>;
    B::Poly c(k), init(k);
    for (auto &x : c)
    {
        int v;
        scanf("%d", &v);
        x = v;
    }
    for (auto &x : init)
    {
        int v;
        scanf("%d", &v);
        x = v;
    }
    printf("%d\n", B::recurrence(init, c, n).v);
}
```

4.8.3 线性递推：系数方向与外推条件

约定 $a_i = \sum_{j=1}^{k} c_{j-1} a_{i-j}$，初值 $a_0, \ldots, a_{k-1}$，特征多项式

$$F(x) = x^k - c_0 x^{k-1} - c_1 x^{k-2} - \cdots - c_{k-1}.$$

若 $x^n \bmod F(x) = \sum_{i=0}^{k-1} r_i x^i$，则 $a_n = \sum_{i=0}^{k-1} r_i a_i$。BM 要求素数模数；已知阶数至多 d 时至少给出前 $2d$ 项。系数末尾的零不能删除，否则改变递推开始适用的位置。BM 加特征多项式求余求第 m 项：时间 $O(nk + k^2 \log m)$，空间 $O(n + k)$。

4.8.4 Bostan–Mori 与快速递推单项

代码见第 102 页。对 $a_i = \sum_{j=1}^{k} c_{j-1} a_{i-j}$，令 $Q(x) = 1 - \sum_{j=1}^{k} c_{j-1} x^j$，$P(x) = (Q(x) \sum_{i=0}^{k-1} a_i x^i) \bmod x^k$，则 $a_n = [x^n]P/Q$。系数按前一项、前两项顺序给出，初值从 a_0 开始。

$$\frac{P(x)}{Q(x)} = \frac{P(x)Q(-x)}{Q(x)Q(-x)} = \frac{U_0(x^2) + xU_1(x^2)}{V(x^2)},$$

保留与 n 同奇偶的分子项、分母偶数项，n 右移一位；$n = 0$ 时答案为 $P(0)/Q(0)$。分母常数项须非零；不要求真分式，空分子返回零。初始最大长度 d 须满足 $2d - 1$ 不超过 NTT 容量。时间 $O(d \log d \log(n + 1))$，空间 $O(d)$；下标可达完整无符号 64 位。

4.9 多项式平移 | 连续点值平移

4.9.1 多项式系数平移

功能： 给定多项式 $f(x)$ 的升幂系数和常数 c，求 $f(x+c)$ 的升幂系数（模 NTT 素数）。

核心思路： 二项式展开得 $[x^j]f(x+c) = \frac{1}{j!}\sum_{i\geq j} f_i\, i! \cdot \frac{c^{i-j}}{(i-j)!}$，右边是关于差 $i-j$ 的卷积。把 $f_i\, i!$ 逆序放进 a（存在 a[n-1-i]），把 $c^t/t!$ 放进 b，一次 NTT 乘法后第 $n-1-j$ 项正是上式的求和部分，再乘 $1/j!$ 即得答案。阶乘与逆阶乘先线性预处理。

代码依赖：ModInt（第 12 页，3.1.2 节）、NttConvolution（第 73 页，4.2.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约： polynomial_shift<mod=998244353,primitive=3>(f,c) 返回 f(x+c) 的升幂系数，结果长度始终与 f 相同，保留末尾零；空输入返回空。参数 c 使用 ModInt<mod>。依赖参数化 NTT，mod 为素数且 primitive 为原根；非空长度 n 要求 2n-1≤NTT::max_size，因此阶乘均可逆。逆序的 f[i]i! 与 c^i/i! 卷积后提取对应项，O(n log n) 时间、O(n) 空间。c=0 合法。

```
template <int mod = 998244353, int primitive = 3>
vector<ModInt<mod>> polynomial_shift(const vector<ModInt<mod>> &f, ModInt<mod> c)
{
    using N = NttConvolution<mod, primitive>;
    using Z = typename N::Z;
    using Poly = typename N::Poly;
    if (f.empty()) return {};
    assert(f.size() <= (N::max_size + 1ULL) / 2);
    int n = f.size();
    Poly fact(n), inv(n), a(n), b(n);
    fact[0] = 1;
    for (int i = 1; i < n; i++) fact[i] = fact[i - 1] * i;
    inv[n - 1] = fact[n - 1].inv();
    for (int i = n - 1; i > 0; i--) inv[i - 1] = inv[i] * i;
    Z power = 1;
    for (int i = 0; i < n; i++)
    {
        a[n - 1 - i] = f[i] * fact[i];
        b[i] = power * inv[i];
        power = power * c;
    }
    auto product = N::multiply(a, b);
    for (int i = 0; i < n; i++) a[i] = product[n - 1 - i] * inv[i];
    return a;
}
```

模板题使用：Library Checker polynomial_taylor_shift

题意：给多项式 f 的 n 个升幂系数和常数 c，输出 f(x+c) 的 n 个升幂系数，模 998244353。平移方向为加 c，c=0 时原样返回。

所需模板：ModInt（第 12 页，3.1.2 节）、NttConvolution（第 73 页，4.2.1 节）、polynomial_shift（第 106 页，4.9.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/polynomial_taylor_shift

```
int main()
{
    int n, c;
    scanf("%d%d", &n, &c);
    using Z = ModInt<998244353>;
    vector<Z> f(n);
    for (auto &x : f)
    {
        int v;
        scanf("%d", &v);
        x = v;
    }
    auto answer = polynomial_shift(f, Z(c));
    for (int i = 0; i < n; i++) printf("%d%c", answer[i].v, i + 1 == n ? '\n' : ' ');
}
```

4.9.2 Taylor 平移与等比点求值

代码见第 106 页和第 82 页。升幂系数，长度 n 表示次数小于 n。

$$[x^i]f(x+c)=\frac{1}{i!}\sum_{j=i}^{n-1}f_j j!\frac{c^{j-i}}{(j-i)!}.$$

倒序 $f_j j!$ 与 $c^j/j!$ 卷积，取第 $n-1-i$ 项乘 $1/i!$。要求 NTT 素数且 $2n-1$ 不超过变换容量；空数组返回空数组。时间 $O(n\log n)$、空间 $O(n)$。这不是点值平移。

Chirp Z 返回 $f(ar^i)$，$0\le i<m$。设 $T_j=j(j-1)/2$，

$$f(ar^i)=r^{-T_i}\sum_{j=0}^{n-1}f_j a^j r^{-T_j}r^{T_{i+j}}.$$

倒序第一项数组与 r^{T_j} 卷积，取第 $n-1+i$ 项；循环卷积长度取不小于 $n+m-1$ 的二次幂即可。要求 $n+m-1$ 不超过 NTT 容量，时间 $O((n+m)\log(n+m))$、空间 $O(n+m)$。$r=0$（约定 $r^0=1$）、$a=0$、$r=1$ 单独处理；$m=0$ 返回空，空输入返回 m 个零。

4.10 符号化方法

4.10.1 生成函数建模到模板：三个可核验例子

记 N 为所求最大次数、$L=N+1$、$M(L)=O(L\log L)$ 为 NTT 乘法复杂度。第 73 页 `NttConvolution<p,g>` 可指定素数与原根；`FpsInverse`（第 84 页）、`FpsFunctions`（第 96 页）、`FpsPower`（第 98 页）固定 $p=998244353$。FPS 长度参数是项数，求 x^N 须传 $N+1$。NTT 检查完整乘积长度 $a.size()+b.size()-1$，固定模数最大 2^{23}，FPS 取 $L\le 2^{22}$。

4.10.1.1 例一：不同盒子、相同球、每盒有上下界（OGF 乘积）

$l_i\le a_i\le u_i$、$\sum a_i=N$。令 $c_i=u_i-l_i$、$R=N-\sum l_i$；若 $u_i<l_i$ 或 $R<0$ 答案为 0，否则

$$[x^R]\prod_{i=1}^{k}(1+x+\cdots+x^{c_i}).$$

只保留 $0,\dots,R$ 次：c_i 截至 R，乘后超过 $R+1$ 才截短，不要把中间多项式补零。小规模用滑动窗口

$$d_i(s)=d_i(s-1)+d_{i-1}(s)-[s>c_i]d_{i-1}(s-c_i-1),$$

$d_0(0)=1$、负下标为 0，$O(k(R+1))$ 时间、$O(R+1)$ 空间，任意模数可用。分治 NTT 为 $O(k+M(D+1)\log(k+1))$，$D=\sum\min(c_i,R)$。所有 $c_i=c$ 时可用 `FpsPower::power(P,to_string(k),R+1)`，$P=1+\cdots+x^{\min(c,R)}$，$O(M(R+1)+\log(k+1))$；或在 $k\ge 1$ 时容斥

$$\sum_{j=0}^{\min(k,\lfloor R/(c+1)\rfloor)}(-1)^j\binom{k}{j}\binom{R-j(c+1)+k-1}{k-1},$$

组合数接口见第 113 页。$k=0$ 单独返回 $[N=0]$。

4.10.1.2 例二：步长 1、2 的有序序列（OGF 求逆）

单步 OGF $B(x)=\sum b_sx^s$（$s\ge 1$），有序序列 $F=(1-B)^{-1}\bmod x^{N+1}$，$O(M(N+1))$ 时间、$O(N+1)$ 空间。步长 1、2 时调用：

```
FpsInverse::inverse({1,-1,-1},N+1)
```

要求 $B(0)=0$；只区分各长度使用次数时应改为 $\prod_s(1-x^s)^{-1}$。N 巨大且步长有界时改用远项算法。

4.10.1.3 例三：不同元素分成单点与无序对（EGF 指数）

块的 EGF 为

$$B(x)=x+\frac{x^2}{2!},\qquad F(x)=\exp B(x),\qquad a_n=n![x^n]F(x).$$

在 $p=998244353$、$N<p$ 下用 `FpsFunctions::exp(b,N+1)`，再乘 `Binomial` 的 `fac[N]`；$O(M(N+1))$。也有 $a_0=a_1=1$、$a_n=a_{n-1}+(n-1)a_{n-2}$。恰好 k 块为 $n![x^n]B(x)^k/k!$：用 `FpsPower::power(b,to_string(k),n+1)` 取第 n 项再乘 `fac[n]`、`ifac[k]`，表预处理至 $\max(n,k)<p$；先判 $k\le n\le 2k$，$(n,k)=(0,0)$ 为 1。盒子带标号时不除以 $k!$；允许空盒时单盒因子添 1。

4.11 Lagrange 反演

4.11.1 隐式生成函数与 Lagrange 反演

用于 $T = x\Phi(T)$ 只问 T 或 $H(T)$ 某一项的情形；这里是**复合逆**，不同于第 165 页的点值插值。

4.11.1.1 适用条件与取系数公式

特征零域上形式幂级数，$\Phi(0) \neq 0$（一般 $\mathbb{Q}$-代数中要求 $\Phi(0)$ 可逆），$T(x) = x\Phi(T(x))$、$T(0) = 0$ 唯一确定 T。对 $n \geq 1$：

$$\boxed{[x^n]H(T(x)) = \frac{1}{n}[t^{n-1}]H'(t)\Phi(t)^n,} \qquad \boxed{[x^n]T(x)^k = \frac{k}{n}[t^{n-k}]\Phi(t)^n \ (n \geq k \geq 1).}$$

边界：$[x^0]H(T) = H(0)$；$[x^n]T^0 = [n = 0]$；$k \geq 1$、$n < k$ 时为 0。不要求解析收敛。

4.11.1.2 模型一按内部结点计数的满有序多叉树

每个结点为叶子或恰有 $d \geq 2$ 个有序孩子，按内部结点计数：$A = 1 + xA^d$，$a_0 = 1$，$n \geq 1$ 时

$$a_n = \frac{1}{n}\binom{dn}{n-1} = \frac{1}{(d-1)n+1}\binom{dn}{n}.$$

孩子无序时不能用 A^d。用 `Binomial<p> c(d*n)` 须在宽类型中算 dn 并保证 $dn < p$，否则按第 113 页选组合数算法。

4.11.1.3 模型二带标号有根树的 EGF

根的孩子无序：$T = x\exp T$，$r_n = n![x^n]T = n^{n-1}$（$n \geq 1$）；指定根时为 n^{n-2}（$n \geq 2$）。EGF 系数须乘回 $n!$。

4.11.1.4 落到现有接口及模数边界

单个幂系数。第 98 页的 `FpsPower::power` 固定 $p = 998244353$；在 $1 \leq k \leq n < p$、$L = n - k + 1 \leq 2^{22}$ 下：

```
using F = FpsPower;
using Z = F::Z;
int l = n - k + 1;
auto q = F::power(phi, to_string(n), l);
Z answer = Z(k) * Z(n).inv() * q[n - k];
```

长度是项数（传 $n-k+1$），指数传完整十进制串。单次 $O(L\log(L+1)+\log(n+1))$、空间 $O(L)$；对所有 $n \leq N$ 逐个求为 $O(N^2\log(N+1))$。一般 H：先算 $q = \Phi^n \bmod t^n$，答案为 $n^{-1}\sum_{i=1}^{n} iH_i q_{n-i}$。第 96 页的 `log`（首项 1）、`exp`（首项 0）可构造已知表达式，不能解任意隐式方程；`Binomial<p> c(N)` 要求 $N < p$。第 84 页的 `inverse` 是乘法逆，要求常数项非零，不能求 $f = t/\Phi(t)$ 的复合逆。

模数。在 $\mathbb{F}_p$ 中使用 $1/n$ 公式要求 $p \nmid n$，EGF 的 $1/j!$ 不能跨过 $j = p$；否则改用无除法递推或提升精度后再整除。

4.12 普通生成函数

4.12.1 生成函数与递推

OGF 乘法对应普通卷积；EGF $\sum a_n x^n/n!$ 乘法对应 $c_n = \sum_k \binom{n}{k} a_k b_{n-k}$。求逆要求常数项可逆；$\log A$ 要求 $A(0) = 1$，$\exp A$ 要求 $A(0) = 0$；有限域积分要求 i 不被特征整除。

$$B_{\text{new}} = B(2 - AB) \pmod{x^{2m}}, \qquad (\log A)' = A'/A.$$

BM 外推未来项前须另知序列有不超过相应阶数的线性递推（如由矩阵转移、有限状态得到阶数上界）。

第五章　组合数学

5.1　排列组合

5.1.1　阶乘组合数

功能：在素数模数下预处理阶乘与逆阶乘，快速回答组合数 $\binom{n}{k}$ 与排列数 $n!/(n-k)!$；表的上限可以按需扩大。

核心思路：`fac[i]`$= i!$，`ifac[i]`$= (i!)^{-1}$。`init` 扩表时先顺推新增的阶乘，只对最大的 `fac[n]` 求一次逆元，再用 $((i-1)!)^{-1} = (i!)^{-1} \cdot i$ 倒推回旧上限。查询时 $\binom{n}{k} = n!\,(k!)^{-1}((n-k)!)^{-1}$，$k$ 越界返回 0。

代码依赖：ModInt（第 12 页，3.1.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：Binomial<prime>(n=0) 预处理 0..n，要求 prime 为素数且 0≤n<prime。init(n) 只扩大当前上限，较小或相同的 n 不改变表；choose(n,k) 返回 C(n,k)，permute(n,k) 返回 n!/(n-k)!。查询前显式预处理到 n，查询本身不扩表。n<0、k<0 或 k>n 返回 0，C(0,0)=P(0,0)=1。fac、ifac 为可直接读取的阶乘与逆阶乘表，增长可能使旧引用失效。单次查询 O(1)；从 old 扩到 n 新增 O(n-old+log prime) 次域运算，重分配还可能复制旧表。首次预处理 O(n+log prime)，空间 O(n)。超过模数不能使用阶乘除法：小素数用 Lucas，小合数用 exLucas。

```
template <int mod> struct Binomial
{
    using Z = ModInt<mod>;
    vector<Z> fac{Z(1)}, ifac{Z(1)};

    Binomial(int n = 0) { init(n); }

    void init(int n)
    {
        assert(0 <= n && n < mod);
        int old = (int)fac.size() - 1;
        if (n <= old) return;
        fac.resize(n + 1);
        ifac.resize(n + 1);
        for (int i = old + 1; i <= n; i++) fac[i] = fac[i - 1] * i;
        ifac[n] = fac[n].inv();
        for (int i = n; i > old; i--) ifac[i - 1] = ifac[i] * i;
    }
```

```
    Z choose(int n, int k) const
    {
        if (k < 0 || k > n) return 0;
        assert(n < (int)fac.size());
        return fac[n] * ifac[k] * ifac[n - k];
    }

    Z permute(int n, int k) const
    {
        if (k < 0 || k > n) return 0;
        assert(n < (int)fac.size());
        return fac[n] * ifac[n - k];
    }
};
```

5.1.2 计数建模：先确认哪些对象被区分

先说明**一个方案是什么，什么变化会产生新方案**，再选公式。

1. **物品可区分吗？**相同球只记录数量，不能再乘 $n!$。
2. **容器可区分吗？**无标号盒子记录集合划分，空盒不成块。
3. **次序在哪里？**固定 s 个标签组成非空块：集合 1 种，线性排列 $s!$ 种，圆排列 $(s-1)!$ 种。
4. **哪些数量固定？**“恰好 k 个非空组”与“至多 k 个组”不同；统计块数时每块贡献一个 y。
5. **空对象是否是一种方案？**空序列、空集合各 1 种，常数项为 1；“至少一个块”减去 1。无物品且无盒计 1，非空物品放入零盒计 0，题面另有定义时以题面为准。

仅当恰好 k 个非空块且只区分盒子标号时，才可除以 $k!$。

5.1.2.1 OGF、EGF 的选择来自乘法，而非题目用词

两部分合并恰有 a_ib_j 种时用 OGF 普通卷积；若还需从 $i+j$ 个标签中分配，多出 $\binom{i+j}{i}$，用 EGF。

$$\text{OGF 数组：}A_i=a_i,\qquad \text{EGF 数组：}A_i=a_i/i!,\qquad a_n=n!A_n.$$

卷积模板始终计算普通数组卷积，不会补上 EGF 的阶乘。非空带标号块的 EGF 为 $B(x)$ 时，有序块序列是 $1/(1-B(x))$，无序块集合是 $\exp B(x)$；二者都要求 $B(0)=0$。

5.1.3 球盒问题：八种情形

n 个球放入 m 个盒子，$S(n,k)$ 为第二类斯特林数，$f_m(s)$ 为把 s 拆成至多 m 个部分的分拆数（第 129 页）。

球	盒	允许空盒	不允许空盒
不同	不同	m^n	$m!\,S(n,m)$
不同	相同	$\sum_{k=0}^{m} S(n,k)$	$S(n,m)$
相同	不同	$\binom{n+m-1}{m-1}$	$\binom{n-1}{m-1}$
相同	相同	$f_m(n)$	$f_m(n-m)$ $(n\ge m)$

$S(n,0..m)$ 一次取第 121 页整行；$m\ge n$ 时第二行左列即贝尔数。组合数用第 111 页 `Binomial`，负下标返回 0 正好给出 $n<m$ 时的 0。边界：$m=0$ 时八种都为 $[n=0]$；$n=0$、$m\ge 1$ 时允许空盒为 1、不允许空盒为 0。f_m 小数据用 $O(nm)$ 的 `parts_at_most`，模 998244353 且规模大时用 $O(n\log n)$ 的 `parts_at_most_fps`。

5.1.4 组合数选型：先看模数，再看下标与预算

先用有符号或更宽类型处理负下标、$k > n$ 与 $n+k$ 溢出；`Lucas::choose`、`ExLucas::choose` 接收无符号参数，不要传入容斥中的负数。

素数 p，所有查询 $n \le N < p$： 第 111 页 `Binomial<p> c(N)`，`c.choose(n,k)`、`c.permute(n,k)`。建表 $O(N + \log p)$ 时间、$O(N)$ 空间，查询 $O(1)$。`c.init(M)` 每次扩表都求一次逆，尽量一次扩至最大上界。p 为编译期模数，由调用者保证是素数。

素数 p，n 很大且 p 足够小： 第 38 页 `Lucas c(p)`。预处理 $O(p + \log p)$ 时间、$O(p)$ 空间，单次 $O(1 + \log_p n)$。$p = 10^9 + 7$ 时两张表约 8 GB，不适用。

合数 m，所有素数幂因子的表可以放下： 第 39 页 `ExLucas c(m)`，按 $m = \prod q_i$、$q_i = p_i^{e_i}$ 建表。空间 $O(\sum q_i)$，建表 $O(\sqrt{m} + \sum q_i + \sum \log q_i)$；单次 $O(\sum_i (1 + \log_{p_i}(n+1)) \log(n+2) + \sum_i \log q_i)$。模数 1 返回 0；某个 q_i 很大时不适用。

三者都不合适时： n 小、模数任意用 Pascal 递推，$O(n^2)$ 时间、$O(n)$ 滚动空间（只需 $k \le K$ 列时 $O(nK)$）；素数 p 很大、$k' = \min(k, n-k) < p$ 很小时，连乘 k' 个分子再乘 $(k'!)^{-1}$，$O(k' + \log p)$，$n \ge p$ 也可用。合数模数下分母未必可逆，不能把整数除法直接换成模逆元。

5.2 容斥原理

5.2.1 容斥与二项式反演：恰好与至少

交集和带重数，不能直接当作“至少”的数量；性质之间不要求独立。

5.2.1.1 从交集和恢复恰好数量

全集 Ω，性质集合 $A_1,\dots,A_m$，$q(x)=|\{i:x\in A_i\}|$，

$$N_t=|\{x\in\Omega:q(x)=t\}|,\qquad S_j=\sum_{|J|=j}\left|\bigcap_{i\in J}A_i\right|,\quad S_0=|\Omega|.$$

$$\boxed{S_j=\sum_{t=j}^{m}\binom{t}{j}N_t,\qquad N_r=\sum_{j=r}^{m}(-1)^{j-r}\binom{j}{r}S_j\quad(0\le r\le m).}$$

并集 $|\bigcup_i A_i|=\sum_{j=1}^{m}(-1)^{j-1}S_j$。

5.2.1.2 至少数量与变换方向

对 $1\le r\le m$：

$$\boxed{|\{x:q(x)\ge r\}|=\sum_{j=r}^{m}(-1)^{j-r}\binom{j-1}{r-1}S_j.}$$

$r=0$ 返回 S_0，$r>m$ 返回 0。另一方向（下三角）的二项式反演：

$$b_n=\sum_{k=0}^{n}\binom{n}{k}a_k\quad\Longleftrightarrow\quad a_n=\sum_{k=0}^{n}(-1)^{n-k}\binom{n}{k}b_k.$$

只含整数乘加减，对任意正模数成立。一般 S_j 需枚举 2^m 个子集；仅当交集大小只依赖 $|J|$ 时才可写 $S_j=\binom{m}{j}\cdot$ 代表值。

5.2.1.3 例：有标号球放入有标号盒

n 个互异球放入 m 个互异盒，恰好 r 个非空盒：

$$\boxed{\binom{m}{r}\sum_{j=0}^{r}(-1)^j\binom{r}{j}(r-j)^n=\binom{m}{r}r!\left\{{n\atop r}\right\}.}$$

以“盒为空”为性质时 $S_j=\binom{m}{j}(m-j)^n$，对应 N_{m-r}。$r<0$、$r>m$ 或 $r>n$ 时为 0；$n=0$ 时仅 $r=0$ 为 1；$0^0=1$。

5.2.1.4 直接使用现有组合数与模整数

固定 $p=998244353$，要求 $m<p$；用第 3.1.2 节（第 12 页）和第 5.1.1 节（第 111 页）：

```
using Z = ModInt<998244353>;
Z answer = 0;
if (0 <= r && r <= m && r <= n)
{
    Binomial<998244353> c(m);
    for (int j = 0; j <= r; j++)
    {
        Z term = c.choose(r, j) * Z(r - j).pow(n);
        if (j & 1) answer -= term;
        else answer += term;
    }
```

```
    answer *= c.choose(m, r);
}
```

n 须为非负 signed long long（`pow` 指数类型）。时间 $O(m+\log p+(r+1)\log(n+2))$、空间 $O(m)$。素数限制来自 `Binomial` 的费马逆元；合数模或 $m\geq p$ 改用 Pascal、Lucas 或扩展 Lucas。

5.2.2 组合计数

容斥：$|\bigcup A_i|=\sum_{\emptyset\neq S}(-1)^{|S|+1}|\bigcap_{i\in S}A_i|$。二项式反演：若 $b_n=\sum_{k=0}^{n}\binom{n}{k}a_k$，则 $a_n=\sum_{k=0}^{n}(-1)^{n-k}\binom{n}{k}b_k$。

$$\sum_k\binom{a}{k}\binom{b}{n-k}=\binom{a+b}{n},\qquad C_n=\binom{2n}{n}-\binom{2n}{n+1}.$$

差分形式的 Catalan 公式避免在模数下除以 $n+1$。斯特林数见第 117 页。

球与盒	空盒允许	空盒禁止
不同球，不同盒	k^n	$k!S(n,k)$
相同球，不同盒	$\binom{n+k-1}{k-1}$	$\binom{n-1}{k-1}$
不同球，相同盒	$\sum_{j=0}^{k}S(n,j)$	$S(n,k)$

上表默认 $k\geq 1$；$n=0$、$k=0$ 按原问题单独处理。

5.3 错位排列

5.3.1 错位排列

D_n 为没有不动点的 n 元排列数：

$$D_0=1,\ D_1=0,\quad D_n=(n-1)(D_{n-1}+D_{n-2}),\qquad D_n=n!\sum_{k=0}^{n}\frac{(-1)^k}{k!}=nD_{n-1}+(-1)^n.$$

恰有 k 个不动点：$\binom{n}{k}D_{n-k}$。$n\geq 1$ 时 D_n 是最接近 $n!/e$ 的整数，只用于估计，取模时用递推。$O(n)$ 打表。

5.4 卡特兰数

5.4.1 Catalan 与投票问题：把前缀限制变成反射双射

一种操作记 $+1$、另一种记 -1，高度为两者之差；p 正 q 负共 $\binom{p+q}{q}$ 个次序。约定 $\binom{N}{k}=0$ 当 $k<0$ 或 $k>N$。

5.4.1.1 弱前缀条件：减去首次走到负一的坏路径

每个前缀高度 ≥ 0，$p\geq q\geq 0$：

$$W(p,q)=\binom{p+q}{q}-\binom{p+q}{q-1}.$$

$p<q$ 时为 0（不能用差式），$p=q=0$ 计 1。$p=q=n$ 得

$$C_n=\binom{2n}{n}-\binom{2n}{n-1}=\frac{1}{n+1}\binom{2n}{n},\qquad C_0=1.$$

5.4.1.2 严格领先：先取走必需的第一步

每个非空前缀高度 >0，$p>q$：

$$S(p,q)=\binom{p+q-1}{q}-\binom{p+q-1}{q-1}=\frac{p-q}{p+q}\binom{p+q}{q}.$$

$p+q>0$ 且 $p\leq q$ 时为 0；空路径单独计 1。

5.4.1.3 初始有余额：反射起点，而非只改终点

从 $h \geq 0$ 走 L 步到 $e \geq 0$、全程不低于 0：须 $|e-h| \leq L$ 且 $L+e-h$ 为偶数，令 $k=(L+e-h)/2$，

$$B(L,h,e)=\binom{L}{k}-\binom{L}{k+h+1}.$$

下界为 b 时起终点先减 b；严格大于 b 取下界 $b+1$。只适用单个常数下界与单位 ± 1 步；有上界、禁位、非单位步等需 DP 或重复反射。

5.4.1.4 取模时优先使用差式，除法条件不能省

差式不要求 $n+1$、$p+q$ 可逆；除法形式在分母不可逆时失效。选择见第 113 页（5.1.4 节）：素数模数且最大上标小于模数用 Binomial（第 111 页，5.1.1 节）；跨过素数模数用 Lucas（第 38 页，3.13.1 节）；合数模数考虑 ExLucas（第 39 页，3.13.2 节）。先用有符号类型检查负下标与 $p+q, 2n, L+e-h, k+h+1$ 的溢出，最后规范化两模值之差。

5.5 斯特林数与斯特林反演

5.5.1 定义与性质

设 $n,k\ge 0$。**无符号第一类** $c(n,k)$：n 个带标号元素的排列中，恰有 k 个循环的方案数；循环之间无序，循环内只忽略起点。**第二类** $S(n,k)$：把 n 个带标号元素划分为 k 个无标号非空集合的方案数。**有符号第一类** $s(n,k)=(-1)^{n-k}c(n,k)$。例：$c(3,1)=2$、$c(3,2)=3$；$S(3,1)=1$、$S(3,2)=3$。

递推与边界 考虑第 n 个元素：第一类可自成一个循环，或插在已有 $n-1$ 个元素之一的后面；第二类可自成一块，或加入已有 k 块之一。故

$$c(n,k)=c(n-1,k-1)+(n-1)c(n-1,k),\qquad S(n,k)=S(n-1,k-1)+kS(n-1,k),$$

$$s(n,k)=s(n-1,k-1)-(n-1)s(n-1,k).$$

三者 $(0,0)$ 项为 1；$n>0$ 时第 0 列为 0；$k>n$ 或下标为负时为 0。$c(n,n)=S(n,n)=1$；$n\ge 1$ 时 $c(n,1)=(n-1)!$、$S(n,1)=1$；$\sum_k c(n,k)=n!$，$\sum_k S(n,k)$ 为 Bell 数。

三种幂的转换 记下降幂 $x^{\underline{k}}=x(x-1)\cdots(x-k+1)$、上升幂 $x^{\overline{k}}=x(x+1)\cdots(x+k-1)$，零次幂为 1，且 $x^{\underline{n}}=(-1)^n(-x)^{\overline{n}}$。普通幂 x^n 与两种阶乘幂互相展开：

$$x^{\overline{n}}=\sum_{k=0}^{n}c(n,k)x^k,\qquad x^{\underline{n}}=\sum_{k=0}^{n}s(n,k)x^k,\qquad x^n=\sum_{k=0}^{n}S(n,k)x^{\underline{k}}=\sum_{k=0}^{n}(-1)^{n-k}S(n,k)x^{\overline{k}}.$$

第一式按“每个因子 $x+i$ 取 x 还是取 i”对应排列的循环结构；第三式在 x 为正整数时数 n 元素到 x 种颜色的映射，按同色类划分后给 k 块分配互异颜色。其余两式由 $x\to -x$ 得到。所以 c、s 分别把上升幂、下降幂展开成普通幂；S 反过来把普通幂展开成下降幂，带符号 $(-1)^{n-k}S$ 展开成上升幂。

生成函数 固定列的指数生成函数（按循环/非空块的标号集合构造）为

$$\sum_{n\ge k}c(n,k)\frac{x^n}{n!}=\frac{(-\log(1-x))^k}{k!},\qquad \sum_{n\ge k}s(n,k)\frac{x^n}{n!}=\frac{(\log(1+x))^k}{k!},$$

$$\sum_{n\ge k}S(n,k)\frac{x^n}{n!}=\frac{(e^x-1)^k}{k!}.$$

第二类的普通生成函数与两类的双变量指数生成函数为

$$\sum_{n\ge k}S(n,k)x^n=\frac{x^k}{\prod_{j=1}^{k}(1-jx)},\qquad \sum_{n,k}c(n,k)y^k\frac{x^n}{n!}=(1-x)^{-y},\qquad \sum_{n,k}S(n,k)y^k\frac{x^n}{n!}=e^{y(e^x-1)}.$$

模素数 p 算到 N 次需 $N<p$，使出现的整数与阶乘可逆；快速实现另受 NTT 长度限制。

5.5.1.1 行列快速求解：推导与选择

需求	函数	方法	复杂度
第一类整行 $c(n,0..n)$	`stirling_first_row`	倍增 + 泰勒平移	$O(n\log n)$
第一类整列 $c(0..N,k)$	`stirling_first_column`	$\exp(k\log A)$	$O(N+m\log m)$
第二类整行 $S(n,0..m)$	`stirling_second_row`	容斥一次卷积	$O(m\log m+m\log p)$
第二类整列 $S(0..N,k)$	`stirling_second_column`	$\exp(k\log A)$	$O(N+m\log m)$

第一类整行。$F_n(x)=x^{\overline{n}}$ 的系数即所求。若已知 F_m，则 $F_{2m}(x)=F_m(x)\,F_m(x+m)$，奇数长度再乘 $x+2m$。$F_m(x+m)$ 用泰勒平移：$[x^i]F(x+m)=\frac{1}{i!}\sum_{j\ge i}(j!f_j)\frac{m^{j-i}}{(j-i)!}$，为一次卷积。$T(n)=T(\lfloor n/2\rfloor)+O(n\log n)$，故 $O(n\log n)$ 时间、$O(n)$ 空间。

第二类整行。由容斥（i^n 为 n 元到 i 色的映射数，再要求每色非空并除以 $k!$），$S(n,k)=\sum_{i=0}^{k}\frac{i^n}{i!}\cdot\frac{(-1)^{k-i}}{(k-i)!}$，一次卷积得 $k=0..m$；$0^0=1$ 单独处理，正底数的指数才对 $p-1$ 取模。

两种整列。列生成函数 $(-\log(1-x))^k$、$(e^x-1)^k$ 都以 x^k 开头，常数项为 0，不能直接取对数。先提出 x^k：第一类 $A(x)=-\log(1-x)/x=\sum_{j\ge 0}\frac{x^j}{j+1}$，第二类 $A(x)=(e^x-1)/x=\sum_{j\ge 0}\frac{x^j}{(j+1)!}$，二者常数项都是 1，于是

$$c(n,k)\text{ 或 }S(n,k)=\frac{n!}{k!}[x^{n-k}]A(x)^k,\qquad A^k=\exp(k\log A).$$

只需 A 的前 $m=N-k+1$ 项；倍率 $n!/k!$ 从 $n=k$ 时的 1 起逐次乘 $n+1$。$k=0$ 时列为 $(1,0,\dots,0)$，$k>N$ 时全零，均直接返回。有符号列再乘 $(-1)^{n-k}$。整行模板可传素数模参数（如 167772161）；整列依赖固定 998244353 的 `FpsFunctions`，不能只改调用处换模数。

5.5.2 无符号第一类斯特林数：同一行

问题描述：固定 n，求无符号第一类斯特林数 $c(n,0),\dots,c(n,n)$，即 $x^{\overline{n}}=x(x+1)\cdots(x+n-1)$ 的全部系数。

核心思路：倍增：$x^{\overline{2m}}=x^{\overline{m}}\cdot(x+m)^{\overline{m}}$，后者由前者泰勒平移一次卷积得到，奇数再乘一个一次因子（推导见上文）。

代码依赖：ModInt（第 12 页，3.1.2 节）、NttConvolution（第 73 页，4.2.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：stirling_first_row<mod,primitive>(n) 返回无符号 c(n,0..n)，默认模 998244353、原根 3。c(n,k) 为 n 元排列恰有 k 个循环的个数。n≥0，n+1≤NTT 的 max_size；n=0 返回 {1}。倍增配合多项式平移计算 x(x+1)…(x+n-1)，O(n log n) 时间、O(n) 空间。带符号版本再乘 (-1)^(n-k)。依赖 NttConvolution；模 167772161 可用 <167772161>。

```
template <int mod = 998244353, int primitive = 3>
vector<ModInt<mod>> stirling_first_row(int n)
{
    using N = NttConvolution<mod, primitive>;
    assert(n >= 0 && n < N::max_size);
    using Z = typename N::Z;
    using Poly = typename N::Poly;
    Poly fact(n + 1), inv(n + 1);
    fact[0] = 1;
    for (int i = 1; i <= n; i++) fact[i] = fact[i - 1] * i;
    inv[n] = fact[n].inv();
    for (int i = n; i > 0; i--) inv[i - 1] = inv[i] * i;
    function<Poly(int)> solve = [&](int len) -> Poly
    {
        if (len == 0) return {1};
        int m = len / 2;
        Poly f = solve(m);
        Poly a(m + 1), b(m + 1);
        Z power = 1;
        for (int i = 0; i <= m; i++)
        {
            a[m - i] = f[i] * fact[i];
            b[i] = power * inv[i];
            power = power * m;
        }
        auto c = N::multiply(a, b);
        for (int i = 0; i <= m; i++) a[i] = c[m - i] * inv[i];
        f = N::multiply(f, a);
        if (len % 2)
```

```
        {
            f.push_back(0);
            for (int i = len; i > 0; i--) f[i] = f[i - 1] + f[i] * (len - 1);
            f[0] = f[0] * (len - 1);
        }
        return f;
    };
    return solve(n);
}
```

模板题使用：Luogu P5408

题意：输出无符号第一类斯特林数 c(n,0..n)，即 n 元排列按循环个数计数。本题模数为 167772161，须显式传模板参数；无需乘交替符号。

所需模板：ModInt（第 12 页，3.1.2 节）、NttConvolution（第 73 页，4.2.1 节）、stirling_first_row（第 118 页，5.5.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5408

```
int main()
{
    int n;
    scanf("%d", &n);
    auto row = stirling_first_row<167772161>(n);
    for (int i = 0; i <= n; i++) printf("%d%c", row[i].v, i == n ? '\n' : ' ');
}
```

5.5.3 无符号第一类斯特林数：同一列

问题描述：固定 k，求无符号第一类斯特林数 $c(0,k),\ldots,c(N,k)$。

核心思路：列的指数生成函数为 $(-\log(1-x))^k/k!$；提出 x^k 后对常数项为 1 的 $A(x)$ 用 $\exp(k\log A)$ 求 k 次幂，再乘 $n!/k!$（推导见上文）。

代码依赖：FpsFunctions（第 96 页，4.7.2 节）、ModInt（第 12 页，3.1.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：stirling_first_column(N,k) 返回无符号 c(0..N,k)，固定模 998244353。0≤N<2^22，k≥0；k>N 返回全零，k=0 仅第零项为 1。令 A=-log(1-x)/x=∑x^j/(j+1)（常数项 1），则 c(n,k)=n!/k! · [x^(n-k)]A^k；用 exp(k · log A) 求前 m=N-k+1 项，再从 n=k 起逐项乘 n+1 得 n!/k!。有符号 s(n,k) 另乘 (-1)^(n-k)。时间 O(N+m log m)，空间 O(N)。依赖 FpsFunctions（及 FpsInverse、NttConvolution、ModInt）；分母都小于模数，不能直接改模 167772161。

```
vector<ModInt<998244353>> stirling_first_column(int n, int k)
{
    using F = FpsFunctions;
    using Z = F::Z;
    assert(n >= 0 && n < F::max_size / 2 && k >= 0);
    vector<Z> answer(n + 1);
    if (k > n) return answer;
    if (k == 0)
    {
        answer[0] = 1;
        return answer;
    }
    // A = -log(1-x)/x, c(n,k) = n!/k! [x^(n-k)] A^k.
    int m = n - k + 1;
    F::Poly a(m), inv(m + 1);
    inv[1] = 1;
    for (int i = 2; i <= m; i++) inv[i] = Z(F::mod - F::mod / i) * inv[F::mod % i];
    for (int i = 0; i < m; i++) a[i] = inv[i + 1];
    auto f = F::log(a, m);
    for (auto &x : f) x *= k;
    f = F::exp(f, m);
    Z scale = 1;
    for (int i = k; i <= n; i++)
    {
        answer[i] = f[i - k] * scale;
        scale *= i + 1;
    }
    return answer;
}
```

模板题使用：Library Checker Stirling Number of the First Kind (Fixed K)

题意：输入 N K，0≤K≤N≤500000，模 998244353；只输出 n=K..N 的有符号 s(n,K)。库函数返回无符号 c(n,K)，逐项乘 (-1)^(n-K)；不输出 n<K 的前导零。

所需模板：ModInt（第 12 页，3.1.2 节）、NttConvolution（第 73 页，4.2.1 节）、FpsInverse（第 84 页，4.5.1 节）、FpsFunctions（第 96 页，4.7.2 节）、stirling_first_column（第 120 页，5.5.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/stirling_number_of_the_first_kind_fixed_k

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, k;
    cin >> n >> k;
    auto column = stirling_first_column(n, k);
    for (int i = k; i <= n; i++)
    {
        auto value = column[i];
        if ((i - k) % 2)
            value = decltype(value)(0) - value;
        cout << value.v << (i == n ? '\n' : ' ');
    }
}
```

5.5.4 第二类斯特林数：同一行前缀

问题描述：固定 n（可达 $2^{64}-1$），求第二类斯特林数 $S(n,0),\dots,S(n,m)$。

核心思路：容斥式 $S(n,k)=\sum_i \frac{i^n}{i!}\cdot\frac{(-1)^{k-i}}{(k-i)!}$ 是一次卷积：两个序列分别取 $i^n/i!$ 与 $(-1)^i/i!$。

代码依赖：ModInt（第 12 页，3.1.2 节）、NttConvolution（第 73 页，4.2.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：stirling_second_row<mod,primitive>(n,m) 返回 S(n,0..m)，默认 mod=998244353、primitive=3。模数必须是素数，primitive 是原根。n 支持完整 unsigned long long；2m+1 不能超过 NTT 的 max_size。S(n,k) 为无标号非空集合划分数；n=0 时仅第零项为 1，k>n 为零。一次卷积配合逆阶乘，O(m log m + m log mod) 时间、O(m) 空间。依赖 NttConvolution；模 167772161 可直接使用模板参数 <167772161>。

```
template <int mod = 998244353, int primitive = 3>
vector<ModInt<mod>> stirling_second_row(unsigned long long n, int m)
{
    using N = NttConvolution<mod, primitive>;
    assert(m >= 0 && m <= (N::max_size - 1) / 2);
    using Z = typename N::Z;
    typename N::Poly a(m + 1), b(m + 1);
    b[0] = 1;
    for (int i = 1; i <= m; i++) b[i] = b[i - 1] * i;
    b[m] = b[m].inv();
    for (int i = m; i > 0; i--) b[i - 1] = b[i] * i;
    for (int i = 0; i <= m; i++)
```

```
    {
        Z value = i ? Z(i).pow(n % (mod - 1)) : Z(n == 0);
        a[i] = value * b[i];
        if (i % 2) b[i] = Z(0) - b[i];
    }
    auto answer = N::multiply(a, b);
    answer.resize(m + 1);
    return answer;
}
```

模板题使用：Luogu P5395

题意：输出第二类斯特林数 S(n,0..n)，即把 n 个不同元素分成各个数目的非空无标号集合的方案数。本题模数为 167772161，须显式传模板参数。

所需模板：ModInt（第 12 页，3.1.2 节）、NttConvolution（第 73 页，4.2.1 节）、stirling_second_row（第 121 页，5.5.4 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5395

```
int main()
{
    int n;
    scanf("%d", &n);
    auto row = stirling_second_row<167772161>(n, n);
    for (int i = 0; i <= n; i++) printf("%d%c", row[i].v, i == n ? '\n' : ' ');
}
```

5.5.5 第二类斯特林数：同一列

问题描述：固定 k，求第二类斯特林数 $S(0,k),\ldots,S(N,k)$。

核心思路：列的指数生成函数为 $(e^x-1)^k/k!$；提出 x^k 后对 $A(x)=(e^x-1)/x$ 用 $\exp(k\log A)$ 求幂，再乘 $n!/k!$（推导见上文）。

代码依赖：FpsFunctions（第 96 页，4.7.2 节）、ModInt（第 12 页，3.1.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：stirling_second_column(N,k) 返回 S(0..N,k)，固定模 998244353。0≤N<2^22，k≥0；k>N 返回全零，k=0 仅第零项为 1。令 A=(e^x-1)/x=∑x^j/(j+1)!（常数项 1），则 S(n,k)=n!/k! · [x^(n-k)]A^k；用 exp(k · log A) 求前 m=N-k+1 项，再逐项乘 n!/k!。时间 O(N+m log m)，空间 O(N)。依赖 FpsFunctions（及 FpsInverse、NttConvolution、ModInt）；不能直接改模 167772161。

```
vector<ModInt<998244353>> stirling_second_column(int n, int k)
{
    using F = FpsFunctions;
    using Z = F::Z;
    assert(n >= 0 && n < F::max_size / 2 && k >= 0);
    vector<Z> answer(n + 1);
    if (k > n) return answer;
    if (k == 0)
    {
        answer[0] = 1;
        return answer;
    }
    // A = (e^x-1)/x, S(n,k) = n!/k! [x^(n-k)] A^k.
    int m = n - k + 1;
    F::Poly a(m), inv_fact(m + 1);
    inv_fact[0] = 1;
    for (int i = 1; i <= m; i++) inv_fact[i] = inv_fact[i - 1] * i;
    inv_fact[m] = inv_fact[m].inv();
    for (int i = m; i > 0; i--) inv_fact[i - 1] = inv_fact[i] * i;
    for (int i = 0; i < m; i++) a[i] = inv_fact[i + 1];
    auto f = F::log(a, m);
    for (auto &x : f) x *= k;
    f = F::exp(f, m);
    Z scale = 1;
    for (int i = k; i <= n; i++)
    {
        answer[i] = f[i - k] * scale;
        scale *= i + 1;
    }
    return answer;
}
```

模板题使用：Library Checker Stirling Number of the Second Kind (Fixed K)

题意：输入 N K，0≤K≤N≤500000，模 998244353；只输出 n=K..N 的 S(n,K)。K=0 时输出首项 1、其余 0。

所需模板：ModInt（第 12 页，3.1.2 节）、NttConvolution（第 73 页，4.2.1 节）、FpsInverse（第 84 页，4.5.1 节）、FpsFunctions（第 96 页，4.7.2 节）、stirling_second_column（第 123 页，5.5.5 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/stirling_number_of_the_second_kind_fixed_k

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, k;
    cin >> n >> k;
    auto column = stirling_second_column(n, k);
    for (int i = k; i <= n; i++)
    {
        auto value = column[i];
        cout << value.v << (i == n ? '\n' : ' ');
    }
}
```

5.5.6 斯特林反演与应用

公式 对下标从 0 开始的任意序列 $a_0, a_1, \dots$ 与 $b_0, b_1, \dots$（求和均为 $0 \le k \le n$）：

$$b_n = \sum_{k=0}^{n} S(n,k) a_k \quad \Longleftrightarrow \quad a_n = \sum_{k=0}^{n} s(n,k) b_k = \sum_{k=0}^{n} (-1)^{n-k} c(n,k) b_k;$$

$$b_n = \sum_{k=0}^{n} c(n,k) a_k \quad \Longleftrightarrow \quad a_n = \sum_{k=0}^{n} (-1)^{n-k} S(n,k) b_k.$$

即“第二类正变换 $\leftrightarrow$ 有符号第一类逆变换”“无符号第一类正变换 $\leftrightarrow$ 带 $(-1)^{n-k}$ 的第二类逆变换”。符号不能省略，$S(n,k)$ 的行标 n 与列标 k 不能对调；两边都必须对 k 从 0 求和（$n \ge 1$ 时 $k=0$ 项系数为 0，可省）。

推导 把 $x^n = \sum_k S(n,k) x^{\underline{k}}$ 中的 $x^{\underline{k}}$ 再按 $x^{\underline{k}} = \sum_j s(k,j) x^j$ 展开，比较 x^j 的系数得 $\sum_{k=j}^{n} S(n,k) s(k,j) = [n=j]$。矩阵 $\mathbf{S} = (S(n,k))$ 与 $\mathbf{s} = (s(n,k))$ 都是对角线为 1 的下三角矩阵，右逆即左逆，故 $\mathbf{sS} = \mathbf{Ss} = I$，$\mathbf{b} = \mathbf{Sa} \iff \mathbf{a} = \mathbf{sb}$，得到第一对。令对角矩阵 $D = \mathrm{diag}((-1)^n)$，则 $\mathbf{s} = D\mathbf{c}D$，$\mathbf{c}^{-1} = D\mathbf{S}D$，得到第二对。两对关系是整数恒等式，取任意模数仍成立；只有快速行列算法需要素数模和逆元。在指数生成函数层面（$A = \sum a_n x^n/n!$，$B = \sum b_n x^n/n!$），第一对等价于 $B(x) = A(e^x - 1)$ 与 $A(x) = B(\log(1+x))$，是形式复合，不是逐项乘积或普通卷积。

手算检查 取 $a = (1,2,3,4)$，第二类正变换得 $b_0 = 1, b_1 = 2, b_2 = S(2,1)\cdot 2 + S(2,2)\cdot 3 = 5, b_3 = 1\cdot 2 + 3\cdot 3 + 1\cdot 4 = 15$。逆变换用 $s(2,\cdot) = (0,-1,1)$、$s(3,\cdot) = (0,2,-3,1)$：$a_2 = -b_1 + b_2 = 3$，$a_3 = 2b_1 - 3b_2 + b_3 = 4$，恢复原序列。

建模：“按块数赋权” 若对 n 个带标号元素的每种集合划分，给含 k 块的划分赋权 a_k，则总权 $b_n = \sum_k S(n,k) a_k$；若 b_n 容易直接计数，就用有符号第一类恢复“恰好 k 块”的量 a_n。例如 n 个元素各染 q 色之一共 q^n 种，按同色类划分后各块颜色互异，有 $q^{\underline{k}}$ 种，即 $b_n = q^n$、$a_k = q^{\underline{k}}$；反演得 $q^{\underline{n}} = \sum_k s(n,k) q^k$，正是“下降幂 $\to$ 普通幂”的转换。$a_k \equiv 1$ 时 b_n 为 Bell 数，逆变换应恢复全 1。排列按循环数赋权时同理使用第二对。只需少数几个 a_n 时，用整行模板求 $s(n,\cdot)$ 后做点积；四种行列模板不提供任意序列的整段快速斯特林变换。

应用：自然数幂和（普通幂 $\to$ 下降幂） 因为 $\sum_{i=0}^{n} i^{\underline{j}} = \frac{(n+1)^{\underline{j+1}}}{j+1}$，

$$\sum_{i=0}^{n} i^k = \sum_{j=0}^{k} S(k,j) \frac{(n+1)^{\underline{j+1}}}{j+1}.$$

例：$n=3, k=2$ 时 $S(2,1) = S(2,2) = 1$，得 $\frac{4\cdot 3}{2} + \frac{4\cdot 3\cdot 2}{3} = 14 = 0+1+4+9$。模 998244353 时，用一次整行 $O(k \log k)$ 求出系数，n 可达 10^{18}（需 $k+1 < p$ 以除以 $j+1$）：

```
using Z = ModInt<998244353>;
Z power_sum(unsigned long long n, int k) // sum of i^k for i = 0..n
{
    auto S = stirling_second_row(k, k);
    Z answer = 0, falling = Z((n + 1) % 998244353);
    for (int j = 0; j <= k; j++)
    {
        answer += S[j] * falling / Z(j + 1);
        falling *= Z(n % 998244353) - Z(j);
    }
    return answer;
}
```

`falling` 依次为 $(n+1)^{\underline{j+1}}$；$n < j$ 时乘到因子 0 后保持为 0，与真实值一致。

5.6 贝尔数

5.6.1 贝尔数

B_n 为 n 元集合的划分数，$B_0 = 1$：

$$B_{n+1} = \sum_{k=0}^{n} \binom{n}{k} B_k, \qquad B_n = \sum_{k=0}^{n} S(n,k), \qquad \sum_{n\geq 0} B_n \frac{x^n}{n!} = e^{e^x-1}.$$

小范围用递推 $O(n^2)$，或贝尔三角：首列 $a_{n,0} = a_{n-1,n-1}$，$a_{n,k} = a_{n,k-1} + a_{n-1,k-1}$，$B_n = a_{n,0}$。批量模 998244353 用第 96 页 `FpsFunctions::exp` 对 $e^x - 1$ 取 exp，$O(n \log n)$；单个 B_n 也可由第 121 页的斯特林数整行求和。素数 p 下有 Touchard 同余 $B_{n+p} \equiv B_n + B_{n+1} \pmod p$。

5.7 伯努利数

5.7.1 伯努利数批量计算

问题描述： 批量求伯努利数 $B_0, B_1, \ldots, B_n$ 模 998244353，取 $B_1 = -\frac{1}{2}$ 的约定，常用于自然数幂和公式。

核心思路： 伯努利数的指数生成函数为 $\frac{x}{e^x-1} = \sum_k B_k \frac{x^k}{k!}$，它是 $\frac{e^x-1}{x} = \sum_k \frac{x^k}{(k+1)!}$ 的逆级数。代码预处理阶乘与逆阶乘，构造后者的前 $n+1$ 项 `d`，用 `FpsInverse` 求逆，再把第 k 项乘 $k!$ 得到 B_k。

代码依赖：FpsInverse（第 84 页，4.5.1 节）、ModInt（第 12 页，3.1.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约： bernoulli(n) 返回 B_0..B_n 模 998244353，约定 B_1=-1/2（生成函数 x/(e^x-1)）。对 (e^x-1)/x 的前 n+1 项求逆再乘阶乘，O(n log n) 时间、O(n) 空间，要求 n+2≤2^23；依赖 FpsInverse。自然数幂和公式见知识节。

```
// B_0..B_n mod 998244353 from x/(e^x-1) (B_1 = -1/2). n + 1 <= 2^23. O(n log n).
vector<ModInt<998244353>> bernoulli(int n)
{
    using F = FpsInverse;
    assert(n >= 0 && n + 2 <= F::max_size);
    vector<F::Z> fac(n + 2, 1), ifac(n + 2);
    for (int i = 1; i <= n + 1; i++) fac[i] = fac[i - 1] * F::Z(i);
    ifac[n + 1] = fac[n + 1].inv();
    for (int i = n + 1; i > 0; i--) ifac[i - 1] = ifac[i] * F::Z(i);
    F::Poly d(n + 1); // (e^x - 1)/x = sum x^k/(k+1)!
    for (int k = 0; k <= n; k++) d[k] = ifac[k + 1];
    auto b = F::inverse(d, n + 1);
    for (int k = 0; k <= n; k++) b[k] = b[k] * fac[k];
    return b;
}
```

5.7.2 伯努利数与自然数幂和

约定 $B_1 = -\frac{1}{2}$：$\sum_{k\geq 0} B_k \frac{x^k}{k!} = \frac{x}{e^x-1}$，等价于 $B_0 = 1$，$\sum_{j=0}^{m} \binom{m+1}{j} B_j = 0$（$m \geq 1$）；$k \geq 3$ 为奇数时 $B_k = 0$。

$$\sum_{i=0}^{n-1} i^k = \frac{1}{k+1} \sum_{j=0}^{k} \binom{k+1}{j} B_j \, n^{k+1-j}.$$

批量 $B_0..B_n$（模 998244353）用第 126 页 `bernoulli(n)`，$O(n \log n)$。只求一个 k 的幂和时，用第 168 页连续点拉格朗日插值 $O(k)$ 更简单；需要对很多 k 求、或要多项式系数时用伯努利数。模数需大于 $k+1$ 以便除法。

5.8 Eulerian Number

5.8.1 欧拉数（Eulerian number）

$\left\langle {n \atop k} \right\rangle$ 为恰有 k 个下降（$p_i > p_{i+1}$）的 n 元排列数，$\left\langle {0 \atop 0} \right\rangle = 1$：

$$\left\langle {n \atop k} \right\rangle = (k+1)\left\langle {n-1 \atop k} \right\rangle + (n-k)\left\langle {n-1 \atop k-1} \right\rangle = \sum_{i=0}^{k} (-1)^i \binom{n+1}{i} (k+1-i)^n.$$

Worpitzky 恒等式：$x^n = \sum_k \left\langle {n \atop k} \right\rangle \binom{x+k}{n}$。按行和为 $n!$，且关于 $k \leftrightarrow n-1-k$ 对称。递推 $O(n^2)$，单项用求和式 $O(k \log n)$。

5.9 分拆数

5.9.1 整数分拆与重数上限

问题描述：求 0..N 每个数的无序整数分拆数 $p(n)$（模任意正整数），并能查询“每个正整数至多出现 k 次”的分拆个数。

核心思路：五边形数定理 $\prod_{i\geq 1}(1-x^i)=\sum_j(-1)^j x^{j(3j\mp 1)/2}$ 给出递推 $p(i)=\sum_{j\geq 1}(-1)^{j+1}\left[p(i-\frac{j(3j-1)}{2})+p(i-\frac{j(3j+1)}{2})\right]$，每个 i 只有 $O(\sqrt{i})$ 项。重数不超过 k 的生成函数为 $\prod_i \frac{1-x^{(k+1)i}}{1-x^i}=P(x)\prod_i(1-x^{(k+1)i})$，后者就是把五边形数级数中的 x 换成 x^{k+1}，因此 `limited` 只需在 $p(n)$ 上加减 $p(n-(k+1)\cdot$五边形数$)$。

接口契约：Partitions(N,mod) 求无序正整数分拆，0≤N<INT_MAX、1≤mod≤INT_MAX，p[n] 为不限制重数的分拆数；p[0]=1%mod。预处理 O(N$\sqrt{\text{N}}$) 时间、O(N) 空间；模数不要求素数。limited(n,k) 限制每个正整数最多出现 k 次，0≤n≤N、0≤k≤INT_MAX；k=0 时仅空分拆存在，k=1 为互异分拆。单次 O(1+$\sqrt{(\text{n}/(\text{k}+1))}$)，额外空间 O(1)。p/mod 只读；中间五边形数与累加采用 long long。

```
struct Partitions
{
    int mod;
    vector<int> p;

    Partitions(int n, int mod) : mod(mod), p(n + 1)
    {
        assert(n >= 0 && mod >= 1);
        p[0] = 1 % mod;
        for (int i = 1; i <= n; i++)
        {
            long long value = 0;
            for (long long j = 1; j * (3 * j - 1) / 2 <= i; j++)
            {
                long long a = j * (3 * j - 1) / 2;
                long long b = j * (3 * j + 1) / 2;
                int sign = j % 2 ? 1 : -1;
                value += 1LL * sign * p[i - a];
                if (b <= i) value += 1LL * sign * p[i - b];
            }
            p[i] = (value % mod + mod) % mod;
        }
    }

    int limited(int n, int k) const
    {
        assert(0 <= n && n < (int)p.size() && k >= 0);
        long long step = 1LL + k;
        long long value = p[n];
        for (long long j = 1; j * (3 * j - 1) / 2 <= n / step; j++)
        {
            long long a = step * (j * (3 * j - 1) / 2);
            long long b = step * (j * (3 * j + 1) / 2);
            int sign = j % 2 ? -1 : 1;
            value += 1LL * sign * p[n - a];
            if (b <= n) value += 1LL * sign * p[n - b];
        }
        return (value % mod + mod) % mod;
    }
};
```

模板题使用：Library Checker partition_function

题意：输出 0..N 的全部整数分拆数，模 998244353，N≤500000。构造 Partitions 后直接读 p；p[0]=1 表示空分拆。五边形数递推 O(N$\sqrt{N}$) 时间、O(N) 空间，不是有序拆分。此用法不调用 limited。

所需模板：Partitions（第 128 页，5.9.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/partition_function

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    Partitions part(n, 998244353);
    for (int i = 0; i <= n; i++) cout << part.p[i] << (i == n ? '\n' : ' ');
}
```

5.9.2 至多 m 个部分的分拆表

问题描述：求 0..n 每个数分拆成至多 m 个正整数之和（不计顺序）的方案数，模任意正整数。

核心思路：由 Ferrers 图共轭，“至多 m 个部分”等价于“每部分不超过 m”。于是做完全背包：物品是大小 $1..\min(m,n)$ 的部分，每种可取任意多个，按大小逐个加入，正序遍历 s 执行 `f[s] += f[s - i]`。

接口契约：parts_at_most(n,m,mod) 返回 f[0..n]，f[s] 为把 s 拆成至多 m 个正整数之和（不计顺序）的方案数，等价于每部分不超过 m（共轭）。0≤n、0≤m、1≤mod≤INT_MAX，模数不要求素数；f[0]=1%mod，m=0 时仅 f[0] 非零。恰好 m 个部分：n≥m 时为 f[n-m]，否则为 0（m=0 时为 [n=0]）。完全背包逐个加入部分大小 1..min(m,n)，O(n · min(m,n)) 时间、O(n) 空间；n · m 大时用 parts_at_most_fps。

```
// f[s] = partitions of s into at most m parts (= parts no larger than m), 0<=s<=n.
vector<int> parts_at_most(int n, int m, int mod)
{
    assert(n >= 0 && m >= 0 && mod >= 1);
    vector<int> f(n + 1);
    f[0] = 1 % mod;
    for (int i = 1; i <= min(m, n); i++)
        for (int s = i; s <= n; s++) f[s] = (0LL + f[s] + f[s - i]) % mod;
    return f;
}
```

5.9.3 至多 m 个部分的分拆表（生成函数）

问题描述：与 `parts_at_most` 相同，求 $0..n$ 每个数分拆成至多 m 个部分的方案数，但模数固定为 998244353，适合 n, m 都很大的情形。

核心思路：所求生成函数为 $\prod_{i=1}^{m} \frac{1}{1-x^i}$。取对数：$\ln \frac{1}{1-x^i} = \sum_{j\geq 1} x^{ij}/j$，所以对每个 $i \leq \min(m,n)$ 枚举 j，把 $1/j$ 加到 `g[i*j]`，总共调和级数次加法；$1/j$ 由线性逆元递推表 `inv` 给出。最后调用 `FpsFunctions::exp` 求出前 $n+1$ 项。

代码依赖：FpsFunctions（第 96 页，4.7.2 节）、ModInt（第 12 页，3.1.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：parts_at_most_fps(n,m) 返回与 parts_at_most 相同的表，模 998244353。生成函数为 1/(1-x)(1-x^2)⋯(1-x^m)，其对数等于对所有 i≤m、j≥1 累加 x^(ij)/j，再用 FpsFunctions::exp 求前 n+1 项。要求 0≤n、n+1≤2^22、m≥0；m≥n 时即无限制分拆数。构造对数 O(n log n)，exp 为 O(n log n) 时间、O(n) 空间；依赖 FpsFunctions。

```
// Same table modulo 998244353: exp(sum_{i<=m} sum_j x^{ij}/j), n+1 <= 2^22.
vector<ModInt<998244353>> parts_at_most_fps(int n, int m)
{
    using F = FpsFunctions;
    assert(n >= 0 && m >= 0 && n + 1 <= (1 << 22));
    F::Poly g(n + 1);
    vector<F::Z> inv(n + 1, 1);
    for (int j = 2; j <= n; j++) inv[j] = F::Z(F::mod - F::mod / j) * inv[F::mod % j];
    for (int i = 1; i <= min(m, n); i++)
        for (int j = 1; i * j <= n; j++) g[i * j] = g[i * j] + inv[j];
    return F::exp(g, n + 1);
}
```

5.9.4 整数分拆：重数限制的差一问题

实现见第 128 页。$P(z)=\prod_{i\ge1}(1-z^i)^{-1}$，$p(0)=1$，五边形数定理

$$\prod_{i\ge1}(1-z^i)=1+\sum_{j\ge1}(-1)^j\left(z^{j(3j-1)/2}+z^{j(3j+1)/2}\right)$$

给出 $p(n)$ 递推，符号 $+,+,-,-,\ldots$，每个 n 有 $O(\sqrt{n})$ 项；无需逆元，任意正模数（含 1）。每个值至多出现 k 次：

$$\prod_{i\ge1}(1+z^i+\cdots+z^{ki})=P(z)\prod_{i\ge1}(1-z^{(k+1)i}),$$

偏移放大 $k+1$ 倍，符号 $-,-,+,+,\ldots$。`limited(n,k)` 包含 k；$k=0$ 时非零 n 无解，$k=1$ 为互异分拆。“每种少于 k 次”对应 `limited(n,k-1)`（$k\ge1$）。

5.9.4.1 三个容易混淆的限制

$A_k(n)$：部分均不超过 k 的分拆数，等于至多 k 个部分的分拆数，生成函数 $\prod_{i=1}^{k}(1-z^i)^{-1}$，用面值 $1,\ldots,k$ 完全背包。恰好 k 部分为 $A_k(n-k)$；恰好 k 个互异部分为 $A_k\left(n-\frac{k(k+1)}{2}\right)$（用 `1LL*k*(k+1LL)/2`）。负目标为 0；$A_0(0)=1$，$A_0(n)=0$（$n>0$）。这些都不同于限制每个值出现次数的 `limited`。

5.9.4.2 重数限制与禁止倍数部分

$d=k+1\ge1$ 时

$$\prod_{i\ge1}\frac{1-z^{di}}{1-z^i}=\prod_{\substack{i\ge1\\ d\nmid i}}\frac{1}{1-z^i},$$

即每值至多 k 次等价于禁用 d 的倍数，可用完全背包核对；$k=1$ 时互异分拆数等于奇数部分分拆数；$k\ge n$ 时无限制。

5.10 范德蒙德卷积

5.10.1 范德蒙德卷积及常用变形

$$\sum_i\binom{n}{i}\binom{m}{k-i}=\binom{n+m}{k},\qquad\sum_i\binom{n}{i}^2=\binom{2n}{n},\qquad\sum_i\binom{n}{i}\binom{m}{i}=\binom{n+m}{n},$$

$$\sum_{i=0}^{n}\binom{i}{a}\binom{n-i}{b}=\binom{n+1}{a+b+1}.$$

组合解释：从两组中共选 k 个，按第一组选了几个分类；最后一式按第 $a+1$ 个被选元素的位置分类。遇到“对某个分界点求和的两个组合数之积”时先尝试化成一个组合数，可把 $O(n)$ 的求和变成 $O(1)$。

5.11 Pólya 计数

5.11.1 Burnside 平均：任意模数下的精确除法

功能：用 Burnside 引理求轨道数 $\frac{1}{|G|}\sum_{\sigma\in G}|\mathrm{Fix}(\sigma)|$ 时的累加器：逐个加入不动点数（可带重数），最后得到除以群阶后的结果模 p，即使群阶在模 p 下不可逆也正确。

核心思路：记群阶为 N，真实总和 S 必被 N 整除，设 $S = NT$。在模 Np 下累加，得到 $S \bmod Np = N\,(T \bmod p)$，直接整除 N 就是 $T \bmod p$，无需逆元。因此所有不动点数（如颜色数 循环数）都必须在模 mod= Np 下计算，结构内附带同模的快速幂 power；乘法用 __int128 防溢出。

接口契约：BurnsideAverage(g,p)：群阶 g>0，目标模数 p>0，要求 g*p<=LLONG_MAX/2（先用 int128 检查）。mod=g*p 供不动点计数使用；add(fixed,multiplicity=1) 累加非负 long long 计数与重数，内部 int128 乘法。power(a,k) 求 a^k 模 mod，a,k 非负，0^0=1。result() 返回 sum/g，支持 p=1 与 g 不可逆；sum 必须是对全部群元素的不动点计数之和模 mod，不是先模 p 后的计数。调用方保证确为群作用并完整枚举群元素或共轭类型，重数合计 g；不会生成闭包或验证群公理。字段只读，add 后可继续累计，只有完整合法总和才能 result。每次 add/result 为 O(1)，power 为 O(log(k+1))，空间 O(1)。

```
struct BurnsideAverage
{
    using ll = long long;
    ll order, mod, sum = 0;

    BurnsideAverage(ll g, ll p) : order(g)
    {
        assert(g > 0 && p > 0 && __int128(g) * p <= LLONG_MAX / 2);
        mod = g * p;
    }

    ll power(ll a, long long k) const
    {
        assert(a >= 0 && k >= 0);
        a %= mod;
        ll ans = 1 % mod;
        while (k)
        {
            if (k & 1) ans = __int128(ans) * a % mod;
            a = __int128(a) * a % mod;
            k >>= 1;
        }
        return ans;
    }

    void add(ll fixed, ll multiplicity = 1)
    {
        assert(fixed >= 0 && multiplicity >= 0);
        sum = (sum + __int128(fixed) * multiplicity) % mod;
    }

    ll result() const
    {
        assert(sum % order == 0);
        return sum / order;
    }
};
```

5.11.2 置换的循环长度分解

功能：给定 0-based 置换 p，求它分解成的各个不交循环的长度。Pólya 计数中，一个置换下的不动染色数为颜色数 ^{循环数}。

核心思路：从每个未访问的位置 i 出发，沿 $u \leftarrow p_u$ 一直走到回到 i，途中标记 `seen` 并计数，得到包含 i 的循环长度。每个位置恰被访问一次。

接口契约：permutation_cycles(p) 返回各不交循环长度，按循环最小位置升序。p 是 0-based 的合法置换，n<INT_MAX；空置换返回空 vector。本函数不校验输入，非置换不允许。O(n) 时间与空间；环上循环迭代不是 DFS。固定染色在每个循环内同色，命名颜色不限数量时为 colors^ 循环数；限定各色用量时以循环长度做背包。

```
vector<int> permutation_cycles(const vector<int> &p)
{
    int n = p.size();
    vector<int> seen(n), lengths;
    for (int i = 0; i < n; i++)
    {
        if (seen[i]) continue;
        int u = i, len = 0;
        do
        {
            seen[u] = 1;
            len++;
            u = p[u];
        } while (u != i);
        lengths.push_back(len);
    }
    return lengths;
}
```

5.11.3 Pólya 环染色：旋转与翻转

问题描述： n 个珠子排成环，每颗用 c 种颜色之一染色，旋转（可选再加翻转）后相同的方案视为同一种，求本质不同的方案数模给定模数。例如 $n=4$、$c=2$、只计旋转时答案为 6。

核心思路： Burnside 引理。旋转 s 格的置换有 $\gcd(s,n)$ 个循环；阶为 d（$d \mid n$）的旋转恰有 $\varphi(d)$ 个，各有 n/d 个循环，故累加 $\varphi(d)\,c^{n/d}$。代码先分解 n 的质因数，再枚举约数 d 并用这些质因子求 $\varphi(d)$。计翻转时：n 为奇数有 n 条对称轴，各有 $(n+1)/2$ 个循环；n 为偶数时，过顶点的轴与过边中点的轴各 $n/2$ 条，循环数分别为 $n/2+1$ 与 $n/2$。总和交给 `BurnsideAverage` 精确除以群阶 n 或 $2n$。

代码依赖：BurnsideAverage（第 132 页，5.11.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约： necklace_colorings(n,colors,mod,reflection=false)：n>=1 的环，每位置可选 colors>=0 种命名颜色，返回本质不同染色数模 mod。n 为 int；mod>0，群阶 g 为 n 或 2n，须 g*mod<=LLONG_MAX/2。默认只视旋转相同；reflection=true 也视翻转相同。允许颜色不用完，不能用于相邻异色、固定颜色用量或颜色重命名模型。n=1/2 仍适用，二面体抽象群作用可不忠实。用 BurnsideAverage 模 g*mod 后整除，无需模逆元；因子分解与枚举约数 O(sqrt(n))，每约数 O(omega(n)+log(n))，额外 O(omega(n))。

```
long long necklace_colorings(int n, long long colors, long long mod,
                             bool reflection = false)
{
    assert(n >= 1 && colors >= 0);
    BurnsideAverage avg((reflection ? 2LL : 1LL) * n, mod);
    vector<int> primes;
    int x = n;
    for (int p = 2; p <= x / p; p++)
    {
        if (x % p != 0) continue;
        primes.push_back(p);
        while (x % p == 0) x /= p;
    }
    if (x > 1) primes.push_back(x);
    auto add = [&](int d)
    {
        long long phi = d;
        for (int p : primes)
        {
            if (d % p == 0) phi = phi / p * (p - 1);
        }
        avg.add(avg.power(colors, n / d), phi);
    };
    for (int d = 1; d <= n / d; d++)
    {
        if (n % d != 0) continue;
        add(d);
        if (d != n / d) add(n / d);
    }
    if (reflection)
    {
        if (n & 1) avg.add(avg.power(colors, n / 2 + 1), n);
        else
        {
            avg.add(avg.power(colors, n / 2), n / 2);
            avg.add(avg.power(colors, n / 2 + 1), n / 2);
        }
    }
    return avg.result();
}
```

模板题使用：P4980 【模板】Pólya 定理

题意：t<=1000，1<=n<=10^9；n 个点用 n 种命名颜色，仅旋转等价，模 10^9+7。直接调用旋转环计数。

所需模板：BurnsideAverage（第 132 页，5.11.1 节）、necklace_colorings（第 134 页，5.11.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4980

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t;
    cin >> t;
    while (t--)
    {
        int n;
        cin >> n;
        cout << necklace_colorings(n, n, 1000000007) << '\n';
    }
}
```

5.11.4 群作用计数：先确定什么算同一个方案

合法对象集合 X 有限，等价变换组成有限群 G，且作用把 X 映回 X；一个轨道就是一个方案。项链商去 C_n；手链（允许翻面，$n \geq 3$）商去阶 $2n$ 的二面体群，反射是否等价由题意决定。颜色可重命名、指定位置或障碍等会改变作用，只有保持整个合法集合的变换才能用 Burnside：

$$|Gx| = \frac{|G|}{|G_x|}, \qquad |X/G| = \frac{1}{|G|}\sum_{g\in G} |\operatorname{Fix}_X(g)|.$$

仅当所有稳定子平凡时才可用 $|X|/|G|$。

5.11.5 项链与手链：从置换循环推公式

$n \geq 1$、$k \geq 1$。置换 g 有 $c(g)$ 个循环时不动染色数为 $k^{c(g)}$。旋转 t 步有 $\gcd(n,t)$ 个循环（$\gcd(n,0)=n$），故

$$R(n,k) = \sum_{t=0}^{n-1} k^{\gcd(n,t)} = \sum_{d\mid n} \varphi(d) k^{n/d}, \qquad N(n,k) = R(n,k)/n.$$

反射：n 奇时 n 个，各 $k^{(n+1)/2}$；n 偶时穿顶点 $n/2$ 个各 $k^{n/2+1}$，穿边中点 $n/2$ 个各 $k^{n/2}$。

$$B(n,k) = \frac{R(n,k)+F(n,k)}{2n}, \qquad F(n,k) = \begin{cases} nk^{(n+1)/2}, & n \text{ 奇数}, \\ \dfrac{n}{2}\left(k^{n/2+1}+k^{n/2}\right), & n \text{ 偶数}. \end{cases}$$

$n = 1, 2$ 时公式仍成立（$2n$ 个形式元素连同重复置换保留）；若对置换去重，分母同步改为不同置换数。不定义 $n = 0$；$k = 0$ 时答案为 0。

5.11.6 指定颜色库存：给每个循环分配整份颜色

颜色数量 $a_1, \ldots, a_k$，和为 n。若 g 的循环长度为 $\ell_1, \ldots, \ell_c$，

$$|\operatorname{Fix}_{\boldsymbol{a}}(g)| = [x_1^{a_1}\cdots x_k^{a_k}] \prod_{j=1}^{c} \left(x_1^{\ell_j} + \cdots + x_k^{\ell_j}\right),$$

相加后除以 $|G|$（同长循环不再除阶乘）。全部循环长 d 时：若有 $d \nmid a_i$ 则为 0，否则为 $\frac{(n/d)!}{\prod_i (a_i/d)!}$，可依次累乘 $\binom{r}{a_i/d}$，素数模数下用第 111 页的 `Binomial<p>`；若对合数模数 M 下计算，按第 113 页选组合数算法或用下

面的 DP。二色恰 r 个红：取 $[x^r]\prod_j(1+x^{\ell_j})$，$f(0)=1$ 起每个循环做 $f(s)\leftarrow f(s)+[s\ge\ell]f(s-\ell)$，$s$ 从 r 降到 ℓ，时间 $O(c(r+1))$、空间 $O(r+1)$，只用加法，任意正模数。多色状态数至多 $\prod_{i<k}(a_i+1)$。“相邻异色” 等不变约束须把循环缩点后在约束图上数不动点；不对群封闭的约束（如指定位置颜色）不能直接平均。

5.11.7 群阶不可逆：先提升模数，再作整数除法

求 $(S/h) \bmod M$，$h=|G|$。仅当 $\gcd(h,M)=1$ 才能乘逆元；否则在模 $L=Mh$ 下求 $s=S \bmod L$，则

$$h \mid s, \qquad S/h \equiv s/h \pmod{M}.$$

须先合计全部项再除。用第 7 页的 `Mod64::mul`、`Mod64::power` 时要求 $Mh \le 2^{64}-1$（先转宽类型再乘，求和也用 128 位）；超出则需多精度。无库存旋转/反射计数时间 $O(n\log(n+1))$、额外空间 $O(1)$。

第六章　线性代数

6.1　矩阵

6.1.1　模矩阵乘法与幂

功能：模 mod 下的矩阵乘法 multiply 与方阵快速幂 power，常用于加速常系数线性递推或计数转移。

核心思路：乘法按 i,t,j 的顺序三重循环累加 $c_{ij}\mathrel{+}=a_{it}b_{tj}$，内层连续访问行，缓存友好。快速幂从单位阵 r 出发按二进制拆分指数：当前位为 1 时乘上 a，每轮把 a 平方。只用加法和乘法，所以模数不必是素数。

代码依赖：ModInt（第 12 页，3.1.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：ModMatrix<p>::multiply(a,b) 计算 n×k 与 k×m 的矩阵积，要求非空且每行尺寸一致。power(a,e) 接收非空 n×n 方阵和 unsigned long long 指数；e=0 返回单位阵。只使用加乘，p 为 ≥2 的 int 模数，允许合数。矩阵乘法 O(nkm)，矩阵幂 O(n^3 log(e+1))，额外空间 O(n^2)；复用 ModInt。

```
template <int mod> struct ModMatrix
{
    using Z = ModInt<mod>;
    using Matrix = vector<vector<Z>>;

    static Matrix multiply(const Matrix &a, const Matrix &b)
    {
        assert(!a.empty() && !b.empty());
        int n = a.size(), m = b[0].size(), k = b.size();
        assert((int)a[0].size() == k);
        Matrix c(n, vector<Z>(m));
        for (int i = 0; i < n; i++)
            for (int t = 0; t < k; t++)
                for (int j = 0; j < m; j++) c[i][j] = c[i][j] + a[i][t] * b[t][j];
        return c;
    }

    static Matrix power(Matrix a, unsigned long long e)
    {
        int n = a.size();
        assert(n > 0 && (int)a[0].size() == n);
        Matrix r(n, vector<Z>(n));
        for (int i = 0; i < n; i++) r[i][i] = 1;
        for (; e; e >>= 1, a = multiply(a, a))
            if (e & 1) r = multiply(r, a);
        return r;
    }
};
```

模板题使用：Library Checker matrix_product

题意：计算 n×m 矩阵与 m×k 矩阵的乘积，模 998244353。输入元素已取模，按行输出 n×k 个结果。

所需模板：ModInt（第 12 页，3.1.2 节）、ModMatrix（第 137 页，6.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/matrix_product

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    using M = ModMatrix<998244353>;
    int n, m, k;
    cin >> n >> m >> k;
    M::Matrix a(n, vector<M::Z>(m)), b(m, vector<M::Z>(k));
    for (auto &row : a)
        for (auto &x : row) cin >> x.v;
    for (auto &row : b)
        for (auto &x : row) cin >> x.v;
    auto c = M::multiply(a, b);
    for (auto &row : c)
        for (int j = 0; j < k; j++) cout << row[j].v << (j + 1 == k ? '\n' : ' ');
}
```

6.1.2 素数模矩阵求逆

功能：给定模素数的 n 阶方阵 A，判断是否可逆，可逆时求 A^{-1}。

核心思路：对增广矩阵 $[A \mid I]$ 做高斯—若尔当消元：第 i 步从第 i 行往下找第 i 列的非零主元，找不到说明 A 奇异；否则两边同时换行、主元行乘逆元归一，并用它消去其他所有行的第 i 列。左边化为单位阵时，右边的 b 就是 A^{-1}。

代码依赖：ModInt（第 12 页，3.1.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：matrix_inverse<mod>(a) 返回 optional 方阵；a 为 n×n 的 ModInt<mod> 矩阵，mod 必须是素数。可逆时返回逆矩阵，奇异时返回 nullopt；按值传入不改原矩阵，空方阵返回有值的空矩阵。每次同时对原矩阵和单位阵进行换行、倍乘、消元；必须同步交换两边的行。依赖 ModInt；$O(n^3 + n \log mod)$ 时间、$O(n^2)$ 额外空间。

```
template <int mod>
optional<vector<vector<ModInt<mod>>>> matrix_inverse(vector<vector<ModInt<mod>>> a)
{
    using Z = ModInt<mod>;
    int n = a.size();
    vector<vector<Z>> b(n, vector<Z>(n));
    for (int i = 0; i < n; i++)
    {
        assert((int)a[i].size() == n);
        b[i][i] = 1;
    }
    for (int i = 0; i < n; i++)
    {
        int p = i;
        while (p < n && !a[p][i].v) p++;
        if (p == n) return nullopt;
        swap(a[p], a[i]);
        swap(b[p], b[i]);
        Z inv = a[i][i].inv();
        for (int k = i; k < n; k++) a[i][k] = a[i][k] * inv;
        for (int k = 0; k < n; k++) b[i][k] = b[i][k] * inv;
        for (int j = 0; j < n; j++)
            if (j != i && a[j][i].v)
            {
                Z f = a[j][i];
                for (int k = i; k < n; k++) a[j][k] = a[j][k] - f * a[i][k];
                for (int k = 0; k < n; k++) b[j][k] = b[j][k] - f * b[i][k];
            }
    }
    return b;
}
```

模板题使用：Luogu P4783

题意：给定 n 阶方阵，求模 1000000007 的逆矩阵；不可逆输出 No Solution。输入按行读取，输出按行给出逆矩阵，不对原矩阵进行覆盖。

所需模板：ModInt（第 12 页，3.1.2 节）、matrix_inverse（第 139 页，6.1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4783

```
int main()
{
    using Z = ModInt<1000000007>;
    int n;
    scanf("%d", &n);
    vector<vector<Z>> a(n, vector<Z>(n));
    for (auto &row : a)
        for (auto &x : row)
        {
            int v;
            scanf("%d", &v);
            x = v;
        }
    auto b = matrix_inverse<1000000007>(a);
    if (!b)
        puts("No Solution");
    else
        for (auto &row : *b)
        {
            for (auto x : row) printf("%d ", x.v);
            puts("");
        }
}
```

6.1.3 Max-plus 矩阵幂

功能： $(\max,+)$ 半环上的 $n\times n$ 矩阵乘法与快速幂：a_{ij} 是从状态 i 一步转移到 j 的收益（无此转移为 `neg`），`power(k)` 的 (i,j) 元即从 i 出发恰好走 k 步到达 j 的最大总收益。

核心思路： 把普通矩阵乘法中的乘换成加、加换成取最大：$c_{ij}=\max_t(a_{it}+b_{tj})$。该运算满足结合律，故可按二进制拆分指数做快速幂。`neg` 充当 $-\infty$，乘法中跳过等于 `neg` 的项，避免把不可达状态加成有限值；单位阵对角为 0、其余为 `neg`。数值用 `__int128` 存储。

接口契约： a[i][j] 为转移收益，不存在的边为 neg；power(k) 求恰好 k 步的最优转移，单位矩阵对角为 0。所有有限中间值绝对值小于 2^119。O(n^3 log k)；2025 成都 B 的转移类型，具体题目的建模仍需调用者完成。

```
struct MaxPlusMatrix
{
    using I = __int128_t;
    static constexpr I neg = -(I(1) << 120);
    int n;
    vector<vector<I>> a;

    MaxPlusMatrix(int n, bool identity = false) : n(n), a(n, vector<I>(n, neg))
    {
        if (identity)
            for (int i = 0; i < n; i++) a[i][i] = 0;
    }

    MaxPlusMatrix operator*(const MaxPlusMatrix &b) const
    {
        assert(n == b.n);
        MaxPlusMatrix c(n);
        for (int i = 0; i < n; i++)
            for (int k = 0; k < n; k++)
                if (a[i][k] != neg)
                    for (int j = 0; j < n; j++)
                        if (b.a[k][j] != neg)
                            c.a[i][j] = max(c.a[i][j], a[i][k] + b.a[k][j]);
        return c;
    }

    MaxPlusMatrix power(unsigned long long e) const
    {
        MaxPlusMatrix b = *this, r(n, true);
        for (; e; e >>= 1, b = b * b)
            if (e & 1) r = r * b;
        return r;
    }
};
```

6.1.4 线性代数与生成树

域上一致方程组的解集为特解加零空间，零空间维数为未知数个数减秩。浮点秩依赖阈值；模素数消元精确；合数模数下不可随意除以非零元。无向图 Laplacian 对角为度数、非对角为负的边重数，删去同一行列的行列式为生成树数；自环不计，重边累加，单点为 1。有向图须另定入度/出度 Laplacian 与树的方向。

6.2 行列式

6.2.1 任意正模数行列式：欧几里得消元

问题描述：给定整数方阵 A 与任意正整数模数 M（可为合数），求 $\det A \bmod M$。

核心思路：合数模下主元未必可逆，因此用辗转相除代替除法：处理第 i 列时，对下方每一行 j，只要 $a_{ji} \neq 0$，就让第 i 行减去 $\lfloor a_{ii}/a_{ji} \rfloor$ 倍的第 j 行，再交换两行，相当于对 (a_{ii}, a_{ji}) 做欧几里得算法，直到 $a_{ji} = 0$。行相减不改变行列式，每次交换把答案取负。一列消完后若 $a_{ii} = 0$ 则行列式为 0，否则乘进答案，最终得到上三角矩阵对角元的乘积。

接口契约：determinant_mod(a,mod) 返回 det(a) 的最小非负余数。a 为 long long 方阵，元素支持完整 signed 64 位；1≤mod≤LLONG_MAX，可以是合数。不使用模逆元，允许不可逆主元。矩阵按值传入，调用后原矩阵不变；空矩阵返回 1%mod，mod=1 时返回 0。换行改变符号，最终乘上三角矩阵的对角元素。所有乘法先转 int128。O(n^3 log mod) 时间、O(n^2) 空间；素数模通常优先用已有高斯行列式。

```
long long determinant_mod(vector<vector<long long>> a, long long mod)
{
    assert(mod >= 1);
    int n = a.size();
    for (auto &row : a)
    {
        assert((int)row.size() == n);
        for (auto &x : row)
        {
            x %= mod;
            if (x < 0) x += mod;
        }
    }
    long long answer = 1 % mod;
    for (int i = 0; i < n; i++)
    {
        for (int j = i + 1; j < n; j++)
            while (a[j][i])
            {
                long long quotient = a[i][i] / a[j][i];
                for (int k = i; k < n; k++)
                {
                    __int128 value = (__int128)a[i][k] - (__int128)quotient * a[j][k];
                    value %= mod;
                    if (value < 0) value += mod;
                    a[i][k] = (long long)value;
                }
                swap(a[i], a[j]);
                answer = answer ? mod - answer : 0;
            }
        if (a[i][i] == 0) return 0;
        answer = (__int128)answer * a[i][i] % mod;
    }
    return answer;
}
```

模板题使用：Library Checker matrix_det_arbitrary_mod

题意：给方阵和模数，求行列式的最小非负余数，模数可以为合数。函数无需模逆元，按值接收矩阵，示例用 move 交出数据。0 阶方阵的行列式为 1，再对模数取余。

所需模板：determinant_mod（第 142 页，6.2.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/matrix_det_arbitrary_mod

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    long long mod;
    cin >> n >> mod;
    vector<vector<long long>> a(n, vector<long long>(n));
    for (auto &row : a)
        for (auto &x : row) cin >> x;
    cout << determinant_mod(move(a), mod) << '\n';
}
```

6.2.2 任意正模数行列式

实现见第 142 页。合数模不能除以任意非零主元；只用倍加行与交换行：对第 i 列两元素 a, b，$R_i \leftarrow R_i - \lfloor a/b \rfloor R_j$ 后交换，按欧几里得终止；交换改变符号。消成上三角后乘对角元与符号；某列剩余全零直接返回零。支持任意正 signed 64 位模数与输入，中间量用有符号 128 位。空矩阵返回 $1 \bmod m$；模 1 返回零。时间 $O(n^3 \log m)$，空间 $O(n^2)$。合数模的矩阵树使用第 D.1 节（第 196 页）的接口。

6.3 线性基

6.3.1 64 位异或线性基

功能：维护一组 64 位无符号整数的异或线性基，支持插入、判断某数能否由若干元素异或得到、求 x 与任意子集异或的最大值、第 k 小的异或值与最小值，以及合并两组线性基。

核心思路：`a[i]` 保存最高位为 i 的基向量。插入时从高位到低位，该位已有基就异或消去，遇到空位就落户；消成 0 说明线性相关，置 `dependent`（存在异或为 0 的非空子集）。最大值从高到低贪心取 $\max(x, x \oplus a_i)$。第 k 小先由 `rebuild` 化成简化阶梯形（每个主元位只在自己的向量中为 1），按主元升序存入 `p`，则把序号的各二进制位对应的 `p[i]` 异或起来就是答案；能得到 0 时 0 占第 1 名，序号先减一。

接口契约：值类型 U 为 unsigned long long，完整支持 64 位。默认空基；重新赋值 XorBasis() 清空。insert/Insert 返回是否增秩；contains/Contains 判断可表示性（允许空子集，所以 0 总可表示）。query/Query(x=0) 返回 x 与所张成空间异或后的最大无符号值。kth/Kth(k,nonempty=true) 返回第 k 小的不同异或值，1-based，参数 k 为 ___uint128_t，非法序号返回 nullopt。默认只允许非空输入子集，0 仅在 dependent 为真时出现；传 false 则包括空子集所得的 0。minimum/Minimum 同此约定，空输入的非空最小值不存在。merge/Merge 将另一基所代表的输入多重集并入；重复元素和自合并可新增异或为 0 的非空方案。rank 是维数，dependent 表示存在非空零异或子集。插入、成员判断、最大值为 O(64)；合并 O(64^2)。kth 会在需要时 rebuild/Rebuild 规范化缓存，首次有效查询 O(64^2)，后续 O(64)；仅新增独立向量使缓存失效。所有成员仅供只读，不要直接改数组或标志。

```
struct XorBasis
{
    using U = unsigned long long;
    using Wide = __uint128_t;
    U a[64]{};
    vector<U> p;
    int rank = 0;
    bool dependent = false, dirty = false;

    bool insert(U x)
    {
        for (int i = 63; i >= 0; i--)
            if (x >> i & 1)
            {
                if (!a[i])
                {
                    a[i] = x;
                    rank++;
                    dirty = true;
                    return true;
                }
                x ^= a[i];
            }
        dependent = true;
        return false;
    }
```

```
28      bool contains(U x) const
29      {
30          for (int i = 63; i >= 0; i--)
31              if (x >> i & 1) x ^= a[i];
32          return x == 0;
33      }
34
35      U query(U x = 0) const
36      {
37          for (int i = 63; i >= 0; i--) x = max(x, x ^ a[i]);
38          return x;
39      }
40
41      void rebuild()
42      {
43          vector<U> b(a, a + 64);
44          for (int i = 0; i < 64; i++)
45              if (b[i])
46                  for (int j = i + 1; j < 64; j++)
47                      if (b[j] >> i & 1) b[j] ^= b[i];
48          p.clear();
49          for (U x : b)
50              if (x) p.push_back(x);
51          dirty = false;
52      }
```

```
    // k is 1-based; nonempty excludes the empty input subset, not value zero.
    optional<U> kth(Wide k, bool nonempty = true)
    {
        if (!k) return nullopt;
        Wide index = k;
        if (!nonempty || dependent) index--;
        if (index >= (Wide(1) << rank)) return nullopt;
        if (dirty) rebuild();
        U answer = 0;
        for (int i = 0; i < rank; i++)
            if (index >> i & 1) answer ^= p[i];
        return answer;
    }

    optional<U> minimum(bool nonempty = true) { return kth(1, nonempty); }

    void merge(const XorBasis &other)
    {
        dependent |= other.dependent;
        for (U x : other.a)
            if (x) insert(x);
    }
};
```

模板题使用：Luogu P3812

题意：给定一组可能重复的非负整数，选任意子集使异或和最大。逐个 insert 后 query()；使用 unsigned long long 读取和输出。

所需模板：XorBasis（第 144 页，6.3.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3812

```
int main()
{
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    int n;
    std::cin >> n;
    XorBasis basis;
    for (int i = 0; i < n; i++)
    {
        unsigned long long x;
        std::cin >> x;
        basis.insert(x);
    }
    std::cout << basis.query() << '\n';
}
```

6.3.2 带位置限制的异或线性基

功能：按标签（如数组下标）插入 64 位整数；查询时给定下界 `lower`，只允许使用标签不小于它的元素，判断能否异或出 x、求最大异或值或张成空间的维数。常用于按右端点扫描，回答区间内选数异或的问题。

核心思路：仍是按最高位存放的线性基，另用 `pos[i]` 记录 `a[i]` 的标签。插入时若当前向量的标签大于该位已有基的标签，就交换二者，让标签大的留在该位，换出来的旧向量继续往低位消元。这样每一位都尽量保留标签大的向量，标签不小于 `lower` 的那些基恰好张成标签不小于 `lower` 的输入所张成的空间；查询时跳过 `pos[i]` 小于下界的位，其余与普通线性基相同。

接口契约：默认构造为空，重新赋值 PositionBasis() 可清空。insert/Insert(x,at) 插入 unsigned long long 值及非负 int 标签，支持完整 64 位，标签可重复、乱序。contains/Contains(x,lower=0) 判断 x 能否由标签至少为 lower 的输入异或得到；query/Query(lower=0,x=0) 返回 x 与这些输入子集异或后的最大值；rank/Rank(lower=0) 返回对应空间维数。lower 为非负 long long，可大于所有 int 标签，空集仍可产生 0。全部操作 O(64)，空间 O(64)。消元时同一主元优先保留标签较大的向量，较旧向量继续消元；按阈值筛选后仍是正确线性基。标签是组合可用范围的界，不是选点方案；数组仅供只读。区间 [l,r] 应按右端点排序询问，扫描并仅插入到 r，再查询 lower=l；接口本身没有上界，插入超过 r 后不能只用 lower 恢复旧区间。若需保留历史，可复制每个前缀的对象，但空间为 O(64n)。不提供非空零异或判定或第 k 小；普通线性基的 dependent 不能直接移植到所有阈值。

```
struct PositionBasis
{
    using U = unsigned long long;
    U a[64]{};
    int pos[64];

    PositionBasis() { fill(pos, pos + 64, -1); }
```

```
    void insert(U x, int at)
    {
        assert(at >= 0);
        for (int i = 63; i >= 0; i--)
            if (x >> i & 1)
            {
                if (!a[i])
                {
                    a[i] = x;
                    pos[i] = at;
                    return;
                }
                if (at > pos[i])
                {
                    swap(a[i], x);
                    swap(pos[i], at);
                }
                x ^= a[i];
            }
    }

    bool contains(U x, long long lower = 0) const
    {
        assert(lower >= 0);
        for (int i = 63; i >= 0; i--)
            if ((x >> i & 1) && pos[i] >= lower) x ^= a[i];
        return x == 0;
    }

    U query(long long lower = 0, U x = 0) const
    {
        assert(lower >= 0);
        for (int i = 63; i >= 0; i--)
            if (pos[i] >= lower) x = max(x, x ^ a[i]);
        return x;
    }

    int rank(long long lower = 0) const
    {
        assert(lower >= 0);
        int answer = 0;
        for (int i = 0; i < 64; i++) answer += pos[i] >= lower;
        return answer;
    }
};
```

6.3.3 异或线性空间求交

问题描述：给定两个 $\mathbb{F}_2^{64}$ 子空间 A, B 的线性基（XorBasis），求交空间 $A \cap B$ 的一组基。

核心思路：以 A 的基作为初始主元表 pivot，逐个插入 B 的基向量。每个主元附带 from_b，记录它来自 B 的部分（A 的主元为 0）；当前向量 x 异或 pivot[i] 时，part 同步异或 from_b[i]，始终保持 part $\in B$ 且 $x \oplus \text{part} \in A$。若 x 被消成 0，则 part 同属 A 和 B，插入结果；否则 x 成为新主元，并记下它的 part。

代码依赖：XorBasis（第 144 页，6.3.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：basis_intersection(a,b) 返回 span(a) 与 span(b) 的交空间的一组独立生成元，结果类型 XorBasis。支持完整 64 位无符号值，不修改输入，也不要求先 rebuild。使用第一个空间的主元，消去第二个空间的向量，同时记录来自第二个空间的部分；消元为零时，该部分属于交空间。时间 $O(64^2)$，额外空间 O(64)。输出按独立生成元解释：dependent 为 false，输入的重复次数及非空零异或关系不传播；表示的是子空间，不是两组原始输入元素的集合交。枚举整个交空间应调用 result.kth(k,false)，包含零；零空间返回空基，rank=0、contains(0)=true。

```
XorBasis basis_intersection(const XorBasis &a, const XorBasis &b)
{
    using U = unsigned long long;
    U pivot[64], from_b[64]{};
    copy(a.a, a.a + 64, pivot);
    XorBasis result;
    for (U value : b.a)
    {
        U x = value, part = value;
        for (int i = 63; i >= 0 && x; i--)
            if (x >> i & 1)
            {
                if (!pivot[i])
                {
                    pivot[i] = x;
                    from_b[i] = part;
                    break;
                }
                x ^= pivot[i];
                part ^= from_b[i];
            }
        if (!x && part) result.insert(part);
    }
    return result;
}
```

模板题使用：LC Intersection of F2 Vector Spaces

题意：多组给两个 F2 向量空间的基，输出交空间的一组基。T≤100000，n,m≤30，输入独立且小于 2^30。把输入插入两个 XorBasis，调用求交，输出 rank 和 a 中的非零主元；答案不唯一，不能逐字比较不同实现输出。模板本身支持 64 位和相关生成元，属于题外扩展。零维输出 0。

所需模板：XorBasis（第 144 页，6.3.1 节）、basis_intersection（第 149 页，6.3.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/intersection_of_f2_vector_spaces

```
int main()
{
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    int tests;
    std::cin >> tests;
    while (tests--)
    {
        XorBasis a, b;
        int n, m;
        std::cin >> n;
        for (int i = 0; i < n; i++)
        {
            unsigned long long x;
            std::cin >> x;
            a.insert(x);
        }
        std::cin >> m;
        for (int i = 0; i < m; i++)
        {
            unsigned long long x;
            std::cin >> x;
            b.insert(x);
        }
        auto result = basis_intersection(a, b);
        std::cout << result.rank;
        for (auto x : result.a)
            if (x) std::cout << ' ' << x;
        std::cout << '\n';
    }
}
```

6.3.4 Zassenhaus：同时求和空间与交空间

问题描述：给定两个 $\mathbb{F}_2^{64}$ 子空间 A, B 的线性基，一次求出和空间 $A+B$ 与交空间 $A\cap B$ 各自的一组基。

核心思路：Zassenhaus 算法：A 的基向量 x 写成 128 位的 (x,x)，B 的基向量 y 写成 $(y,0)$（高 64 位在前），全部插入一个按最高位存放的 128 位线性基。任一行形如 $(a\oplus b,a)$，$a\in A$、$b\in B$。主元在高半部的行，其高半部组成 $A+B$ 的基；主元在低 64 位的行高半部为 0，即 $a=b$，其低半部同属两个空间，组成 $A\cap B$ 的基。

代码依赖：XorBasis（第 144 页，6.3.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：basis_sum_intersection(a,b) 返回和空间基、交空间基这一对结果。和空间由两组基共同张成，一般不是两个空间的集合并；两结果都是独立生成元，其 dependent 均为 false。将 a 的向量 x 编成 (x,x)，b 的向量 y 编成 (y,0)，用 ___uint128_t 储存高低各 64 位并按高位到低位消元。高半部非零的行投影构成和空间的基，高半部为零的行取低半部构成交空间的基。必须先提升到 128 位再左移 64；不能用 64 位变量移位 64。时间 $O(128^2)$，额外空间 O(128)。结果满足 dim(A+B)+dim(A 交 B)=dim(A)+dim(B)，但维数恒等式只是检查条件，不能单独证明返回向量正确。包含空空间、相同空间和输入别名。

```
// Return independent generators of (sum space, intersection space).
pair<XorBasis, XorBasis> basis_sum_intersection(const XorBasis &a, const XorBasis &b)
{
    using W = __uint128_t;
    W pivot[128]{};
    auto insert = [&](W x)
    {
        for (int i = 127; i >= 0 && x; i--)
            if (x >> i & 1)
            {
                if (!pivot[i])
                {
                    pivot[i] = x;
                    return;
                }
                x ^= pivot[i];
            }
    };
    for (auto x : a.a) insert((W(x) << 64) | x);
    for (auto x : b.a) insert(W(x) << 64);
    XorBasis sum, intersection;
    for (int i = 0; i < 128; i++)
        if (pivot[i])
        {
            if (i < 64)
                intersection.insert((unsigned long long)pivot[i]);
            else
                sum.insert((unsigned long long)(pivot[i] >> 64));
        }
    return {sum, intersection};
}
```

模板题使用：LC Intersection / Zassenhaus

题意：同一线性空间求交题，约束与前例相同。Zassenhaus 返回 {和空间, 交空间}，本题仅输出 second 的 rank 和非零主元，不能据此声称 first 也获线上验证。和空间由独立本地测试检查。两部分均为独立生成元；无需对输入枚举全部异或值。

所需模板：XorBasis（第 144 页，6.3.1 节）、basis_sum_intersection（第 151 页，6.3.4 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/intersection_of_f2_vector_spaces

```
int main()
{
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    int tests;
    std::cin >> tests;
    while (tests--)
    {
        XorBasis a, b;
        int n, m;
        std::cin >> n;
        for (int i = 0; i < n; i++)
        {
            unsigned long long x;
            std::cin >> x;
            a.insert(x);
        }
        std::cin >> m;
        for (int i = 0; i < m; i++)
        {
            unsigned long long x;
            std::cin >> x;
            b.insert(x);
        }
        auto result = basis_sum_intersection(a, b).second;
        std::cout << result.rank;
        for (auto x : result.a)
            if (x) std::cout << ' ' << x;
        std::cout << '\n';
    }
}
```

6.4 特征多项式

6.4.1 有限状态计数到线性递推：先证明阶数，再用 BM

只问极大长度 N 的一个计数时，用有限状态 DP 生成少量项，再用 BM 求标量递推外推；须先有阶数上界。

6.4.1.1 先把状态、方向和输出写清楚

$v_n \in R^d$ 为长度 n 时各状态计数的列向量，A_{ij} 为从状态 j 一步到状态 i 的方案数（重边须累加）：

$$v_{n+1} = Av_n, \qquad v_n = A^n v_0, \qquad a_n = u^{\mathsf{T}} v_n.$$

R 可为整数环或模 m 的交换环；按行向量建模时整体转置。**前提：**状态维数 d 有限且固定，转移矩阵固定，更新与输出均为线性（min-plus 不算）。固定周期可加相位状态，仿射项可加恒为 1 的坐标，阶数按扩充后的维数计。

6.4.1.2 Cayley–Hamilton 给出阶数至多 d

$\chi_A(t) = \det(tI - A) = t^d + q_{d-1}t^{d-1} + \cdots + q_0$，$\chi_A(A) = 0$ 对交换环成立，故

$$\boxed{a_{n+d} = -\sum_{i=0}^{d-1} q_i a_{n+i} \quad (n \geq 0).}$$

按 $a_n = \sum_{j=1}^{k} c_{j-1}a_{n-j}$ 约定取 $k = d$、$c_j = -q_{d-1-j}$。调用 BM 只需上界，不必算出特征多项式。

6.4.1.3 为什么前 2d 项足以恢复这个标量序列

在素数域 $\mathbb{F}_p$ 中，精确的前 $2d$ 项 $a_0, \ldots, a_{2d-1}$ 经 BM 得到的最短递推（阶 $k \leq d$）对整个序列成立；$2d-1$ 项不够。恢复的只是该 u, v_0 下的标量递推，不是矩阵恒等式。BM 返回的末尾零系数不能删除。额外项交叉检查不代替阶数证明。

6.4.1.4 模型：不含连续子串 101 的二进制串

状态为与禁串前缀的最长匹配 $\varepsilon, 1, 10$：

$$A = \begin{pmatrix} 1 & 0 & 1 \\ 1 & 1 & 0 \\ 0 & 1 & 0 \end{pmatrix}, \quad v_0 = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}, \quad u = \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix}, \qquad a_n = 2a_{n-1} - a_{n-2} + a_{n-3}.$$

调用链：`c=berlekamp_massey(s,p)`，`init` 取前 `c.size()` 项，`recurrence_nth(init,c,N,p)`，N 为 `unsigned long long` 零基下标。样本与系数须先正规化到 $[0, p)$ 并在目标素数域内恢复；模一个素数得到的递推不保证在别的模数下有效。

6.4.1.5 按目标选算法，并把采样成本算进去

e 为非零转移数：采样前 $2d$ 项 $O(d(e+d))$ 时间、$O(e+d)$ 空间（稠密 $O(d^3)$）；BM $O(d^2 + d\log p)$ 时间、$O(d)$ 空间。

- 直接 DP：$O(N(e+d))$，适合 N 不大或要整个前缀。
- 矩阵幂：第 137 页 `ModMatrix<p>::power(A,N)`，$O(d^3 \log(N+1))$ 时间、$O(d^2)$ 空间，可复用于不同初值/输出。
- 小阶：第 101 页 `recurrence_nth(init,c,N,p)`，$O(k^2 \log(N+1))$ 时间、$O(k)$ 空间。
- 大阶：第 102 页 `BostanMori<p,g>::recurrence(init,c,N)`，$O(k \log(k+1) \log(N+1))$ 时间、$O(k)$ 空间；参数转换为其 `Poly`，p 素数、g 原根，非初值查询要求 $2k+1 \leq$ `NttConvolution<p,g>::max_size`。

BM 见第 181 页。两种远项接口要求初值**恰好** k 项，下标为 `unsigned long long`；$k = 0$ 返回 0，$N < k$ 返回初值。

合数模数： BM 用 x^{p-2} 求逆，只支持素数模数。已知正确递推时，`recurrence_nth` 支持 $2 \le m \le$ `INT_MAX` 的合数模数（初值和系数在 $[0, m)$）；`ModMatrix` 乘法与非负次幂也无需逆元。求逆矩阵与 NTT 版 Bostan–Mori 不适用。

第七章　线性规划

7.1　单纯形法

7.1.1　线性规划：两阶段单纯形

问题描述：线性规划：在约束 $Ax \le b$、$x \ge 0$ 下最大化 $c^{\mathsf{T}}x$，判断有最优解、不可行或无界；有最优解时给出最优值与一组最优解。

核心思路：单纯形表 `d` 的前 m 行是约束（初始以松弛变量为基，最后一列为右端），第 m 行是目标 $-c$，第 $m+1$ 行是第一阶段目标；第 n 列是唯一的人工变量（编号 $n+m$），系数为 -1。若有 $b_i < 0$，先把人工变量与右端最小的行换基，使所有右端非负，再让人工变量尽量取 0：做不到就不可行，否则若它仍在基中则换出。第二阶段在原目标行上 `improve`。`improve` 选编号最小的负检验数列入基（Bland 规则），按最小比值、平局取基变量编号最小的行出基，找不到出基行即无界。

思路中的情况	代码中的处理	行
建表	约束行写入 A、人工变量列 -1 与右端 b	26
存在负右端	人工变量与右端最小的行换基	86
第一阶段结束	人工变量无法降为 0，判不可行	90
人工变量仍在基中	在该行找非零列换出	93
第二阶段	优化原目标，失败即无界	101
选入基列	负检验数中取变量编号最小者	60
选出基行	最小比值，平局取基变量编号最小	74

接口契约：Simplex(A,b,c) 求 max c · x，约束 Ax≤b、x≥0；A 为 m×n，b 长 m，c 长 n，变量/约束均 0 基。构造后只调用一次 solve()，返回 status、value、x；optimal 时 value 与 x 有效，infeasible/unbounded 时只读 status。允许负 b、冗余约束及零维；内部表与 pivot/improve 仅供实现使用。

long double、绝对 eps=1e-12。两阶段人工变量法；按变量编号选入基、最小比值并按基变量编号解平局。精确运算下 Bland 规则避免循环，浮点容差下不保证任意病态输入的正确分类或终止；勿把 AC 视为精确 LP 保证。先缩放数量级，求解后复核原约束和目标值。每次换基 O((m+1)(n+1))，空间同阶；总换基次数最坏指数级。

```
struct Simplex
{
    using Real = long double;
    static constexpr Real eps = 1e-12L;
    enum Status { optimal, infeasible, unbounded };
    struct Result
    {
        Status status;
        Real value;
        vector<Real> x;
    };
    int n, m;
    vector<int> basic, nonbasic;
    vector<vector<Real>> d;

    Simplex(const vector<vector<Real>> &a, const vector<Real> &b,
            const vector<Real> &c) : n(c.size()), m(b.size()),
            basic(m), nonbasic(n + 1), d(m + 2, vector<Real>(n + 2))
    {
        assert((int)a.size() == m);
        for (int i = 0; i < m; i++)
        {
            assert((int)a[i].size() == n);
            for (int j = 0; j < n; j++) d[i][j] = a[i][j];
            basic[i] = n + i;
            d[i][n] = -1;
            d[i][n + 1] = b[i];
        }
        iota(nonbasic.begin(), nonbasic.end(), 0);
        nonbasic[n] = n + m;
        for (int j = 0; j < n; j++) d[m][j] = -c[j];
        d[m + 1][n] = 1;
    }
```

换基（接上页结构体）：

```
    void pivot(int r, int s)
    {
        Real inv = 1 / d[r][s];
        for (int i = 0; i < m + 2; i++)
        {
            if (i == r) continue;
            Real scale = d[i][s] * inv;
            for (int j = 0; j < n + 2; j++)
                if (j != s) d[i][j] -= scale * d[r][j];
        }
        for (int j = 0; j < n + 2; j++)
            if (j != s) d[r][j] *= inv;
        for (int i = 0; i < m + 2; i++)
            if (i != r) d[i][s] *= -inv;
        d[r][s] = inv;
        swap(basic[r], nonbasic[s]);
    }
```

表的含义 第 i 行表示基本变量等于右端值减去非基本变量的线性组合。目标行同样表示当前目标常数减去约化系数乘变量。负约化系数表示增加该变量可改善目标；仅系数为正的约束限制增加量，故用右端除系数的最小值选择离基。若没有限制行则沿该方向无界。

人工变量与两阶段 对每条约束引入同一个人工变量 t，系数为-1。从最负的右端行让 t 入基即得到可行起点，第一阶段最大化-t。最优值小于零说明原问题无解；为零时把仍在基内的人工变量换出，零冗余行可以保留。原目标行始终参加换基，第二阶段禁止人工变量入基，直接优化原目标。

选择入基与出基变量（接上页结构体）：

```
51  bool improve(int objective)
52  {
53      while (true)
54      {
55          int s = -1;
56          for (int j = 0; j <= n; j++)
57          {
58              if (nonbasic[j] == n + m) continue;
59              if (d[objective][j] >= -eps) continue;
60              if (s == -1 || nonbasic[j] < nonbasic[s]) s = j;
61          }
62          if (s == -1) return true;
63          int r = -1;
64          for (int i = 0; i < m; i++)
65          {
66              if (d[i][s] <= eps) continue;
67              if (r == -1)
68              {
69                  r = i;
70                  continue;
71              }
72              Real x = d[i][n + 1] / d[i][s];
73              Real y = d[r][n + 1] / d[r][s];
74              if (x < y - eps || (abs(x - y) <= eps && basic[i] < basic[r]))
75                  r = i;
76          }
77          if (r == -1) return false;
78          pivot(r, s);
79      }
80  }
```

两阶段求解与方案恢复（接上页结构体）：

```
    Result solve()
    {
        int r = -1;
        for (int i = 0; i < m; i++)
            if (r == -1 || d[i][n + 1] < d[r][n + 1]) r = i;
        if (r != -1 && d[r][n + 1] < -eps)
        {
            pivot(r, n);
            improve(m + 1);
            if (d[m + 1][n + 1] < -eps) return {infeasible, 0, {}};
            for (int i = 0; i < m; i++)
            {
                if (basic[i] != n + m) continue;
                int s = -1;
                for (int j = 0; j <= n; j++)
                    if (abs(d[i][j]) > eps &&
                        (s == -1 || nonbasic[j] < nonbasic[s])) s = j;
                if (s != -1) pivot(i, s);
            }
        }
        if (!improve(m)) return {unbounded, 0, {}};
        vector<Real> x(n);
        for (int i = 0; i < m; i++)
            if (basic[i] < n) x[basic[i]] = d[i][n + 1];
        return {optimal, d[m][n + 1], x};
    }
};
```

退化与数值 在精确运算下，入基选最小变量编号，最小比值并列时选最小基本变量编号，是 Bland 防循环规则。这里按 eps 判断约化系数和比值并列，所以不承诺浮点下严格防循环。极小系数、严重消去或尺度悬殊都可能改变分类；先对模型缩放，并用原输入复核返回方案。需要精确判定时应使用有理数算法，不能仅缩小 eps。

模板题使用：P13337 线性规划

题意：n,m≤100，整数系数绝对值 ≤100，题目保证随机数据。最大化 c · x，Ax≤b、x≥0。输出最优值及任一最优解，或 Infeasible/Unbounded；允许绝对或相对误差 1e-6。

所需模板：Simplex（第 155 页，7.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P13337

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    vector<long double> c(n), b(m);
    vector<vector<long double>> a(m, vector<long double>(n));
    for (auto &v : c) cin >> v;
    for (int i = 0; i < m; i++)
    {
        for (auto &v : a[i]) cin >> v;
        cin >> b[i];
    }
    Simplex lp(a, b, c);
    auto result = lp.solve();
    if (result.status == Simplex::infeasible) cout << "Infeasible\n";
    else if (result.status == Simplex::unbounded) cout << "Unbounded\n";
    else
    {
        cout << fixed << setprecision(12) << result.value << '\n';
        for (auto x : result.x) cout << x << ' ';
        cout << '\n';
    }
}
```

第八章　概率论

8.1　随机变量的数字特征

8.1.1　期望选型：计数拆指标，等待拆尾和

先确定随机对象：均匀随机排列、逐步等概率转移和等概率选择最终方案是不同模型，不能用路径条数代替路径概率。

计数型： $X=\sum_i I_i$，$I_i=[A_i]$，则 $\mathbb{E}X=\sum_i \Pr(A_i)$，不要求独立。但一般 $\mathbb{E}(XY)\neq\mathbb{E}X\,\mathbb{E}Y$；方差为

$$\operatorname{Var}\left(\sum_i X_i\right)=\sum_i \operatorname{Var}(X_i)+2\sum_{i<j}\operatorname{Cov}(X_i,X_j),$$

两两独立才可消去协方差。

等待时间或最大值：非负整数随机变量有

$$\mathbb{E}X=\sum_{k\geq 1}\Pr(X\geq k)=\sum_{k\geq 0}\Pr(X>k).$$

覆盖过程：均匀抽 n 种券，已有 i 种时等待期望 $n/(n-i)$，收齐期望 $n\sum_{j=1}^{n}1/j$；非均匀抽券时“已见种类数”不是充分状态。

8.1.2　条件期望：先定义状态，再解自环

状态须保存决定未来分布的信息。终点 $E_t=0$，第一步分析：

$$E_i=\sum_j p_{ij}(c_{ij}+E_j)=b_i+\sum_j p_{ij}E_j,\qquad b_i=\sum_j p_{ij}c_{ij}.$$

命中概率：好终点 1、坏终点 0，$h_i=\sum_j p_{ij}h_j$；可能不终止时取最小非负解。只有自环且 $p_{ii}<1$ 时 $E_i=\big(b_i+\sum_{j\neq i}p_{ij}E_j\big)/(1-p_{ii})$，其余按依赖序 DP；存在 $i\to j\to i$ 时不是 DAG。成功率 $q>0$ 的重复试验期望 $1/q$。

有环时：取起点可达的非终点状态，转移矩阵 Q，解 $(I-Q)E=b$，稠密消元 $O(s^3)$ 时间、$O(s^2)$ 空间；DAG、链、带状或可按 SCC 分块时先用结构。实数接口见GaussReal（第 177 页，10.3.3 节），秩按阈值判定，概率极小时可能病态。有限状态、固定转移概率下，期望有限的条件：可达区域（仅正概率边）中没有封闭的非吸收强连通分量。无限状态时“几乎必然终止”不推出期望有限。

8.1.3　模概率：可逆的不只是输入分母

既约分数 a/b 仅当 $p\nmid b$ 时映射为 $a\,b^{-1}\bmod p$；模值不能比较大小，模值为零不表示概率为零。直接转模方程的充分条件：所有转移概率与费用的分母可逆，且 $I-Q$（含自环的 $1-p_{ii}$）模 p 可逆。实数下可逆不保证模下可逆；未约分表达式应先在有理数层面约分。合数模 m 要求 $\gcd(b,m)=1$，不能用费马小定理求逆。

接口：第 3.1.2 节（第 12 页）`ModInt<p>` 的 `try_inv()` 用欧几里得，不可逆返回空值；`inv()` 与除法要求可逆。第 10.3.2 节（第 174 页）`GaussMod<p>::solve(a,n)` 要求 p 为素数，传 s 行 $s+1$ 列增广矩阵 $[I-Q\mid b]$ 及变量数 s；先查 `consistent`，再查 `rank==s`，才读 `particular`。

第九章　博弈论

9.1　公平组合游戏

9.1.1　SG 的前提、递推与异或策略

前提：双人轮流、无偏（同一状态双方合法操作相同）、无随机与隐藏信息、不能行动者输、状态图为有限 DAG。

$$g(u) = \operatorname{mex}\{g(v) : u \to v\}, \qquad \operatorname{mex} \varnothing = 0.$$

按逆拓扑序计算；d 个后继时 $g(u) \le d$，用时间戳标记，$O(|V| + |E|)$ 时间与空间。$g(u) = 0$ 当且仅当先手必败；SG 值不是胜率或步数。

不交和（一步只动一个子游戏且互不影响）：$G = g_1 \oplus \cdots \oplus g_k$，$G = 0$ 先手必败。$G \neq 0$ 时取 G 最高位上为 1 的 g_i，在该子游戏中找 SG 值为 $g_i \oplus G$ 的后继。可同时改多堆或相互禁止操作时不是不交和。

9.1.2　减法游戏小例与三类失效边界

一堆 n 颗，每次取 S 中数量：$g(0) = 0$，

$$g(n) = \operatorname{mex}\{g(n - s) : s \in S,\ s \le n\}.$$

$g(n) \le |S|$；长度为 $\max S$ 的整个窗口重复后才保证最终周期，几项相同不是周期证明。

- **反常玩法（misère）**：普通 SG 终点规则与异或结论不可直接搬用。
- **有环游戏（loopy）**：mex 可能无解且可能有和局；须按题目处理重复局面或加入步数使之成为 DAG。
- **有偏游戏（partizan）**：按完整状态（含行动方）做胜负搜索，一般不能异或普通 SG 值。

9.1.3　巴什博弈

内容：一堆 n 颗，每次取 $1 \sim k$ 颗，取走最后一颗者胜。
结论：$(k+1) \mid n$ 时先手必败，否则先手取 $n \bmod (k+1)$ 颗必胜。
思路：对方取 t 颗，自己就取 $k+1-t$ 颗，使每轮共取 $k+1$ 颗，局面始终保持 $(k+1) \mid n$。
适用：单堆、每次取数量为连续区间 $[1, k]$；若允许集合不是区间，按减法游戏求 SG（第 162 页）。

9.1.4　Nim 游戏

内容：若干堆，每次从一堆取任意正数颗，取走最后一颗者胜。
结论：各堆异或为 0 时先手必败。
思路：异或非零时，取异或最高位为 1 的一堆 a_i，改成 $a_i \oplus X < a_i$ 即可使异或变 0；异或为 0 时任何一步都会使其非零。
适用：多个独立子游戏且每个子游戏的 SG 值等于堆大小；一般的不交和先求各自 SG 再异或（第 162 页）。

9.1.5　威佐夫博弈

内容：两堆 $a \le b$，每次从一堆取任意正数颗，或从两堆取相同正数颗，取走最后一颗者胜。
结论：先手必败当且仅当 $a = \lfloor (b-a)\varphi \rfloor$，$\varphi = \frac{1+\sqrt{5}}{2}$。整数计算用 $\lfloor d\varphi \rfloor = \left(d + \lfloor \sqrt{5d^2} \rfloor\right)/2$（整除），$\lfloor \sqrt{\cdot} \rfloor$ 用整数开方，避免浮点误差。
思路：必败态 $(a_d, a_d + d)$ 每个差 d 恰有一个，且所有 a_d 与 $a_d + d$ 恰好不重不漏地覆盖全体正整数（Beatty 序

列），所以从必败态出发的单堆或双堆操作都到达不了另一个必败态。
适用：两堆且允许“同时等量取”；三堆及以上或其他组合操作没有此闭式，需打表或求 SG。

9.1.6 斐波那契博弈

内容：一堆 $n \geq 2$ 颗，先手第一次取 $1 \sim n-1$ 颗，之后每次至少取 1 颗、至多取上一次对手所取数量的 2 倍，取走最后一颗者胜。
结论：n 是斐波那契数时先手必败。否则把 n 写成不相邻斐波那契数之和（Zeckendorf 表示），先手取其中最小的一项必胜。
思路：最小项 F_i 与次小项至少差一个下标，有 $2F_i < F_{i+2}$，对手取不完下一项。剩余部分可以看成若干段独立的斐波那契数局面，于是总能把“剩下斐波那契数”的局面留给对手。
适用：每次可取量依赖上一次取量的倍数限制；限制换成其他倍数时没有此结论，按状态 $(n, \text{上限})$ 记忆化搜索。

9.1.7 阶梯博弈

内容：第 $1..k$ 级台阶上各有若干石子，每次把某一级（$i \geq 1$）的任意正数颗移到第 $i-1$ 级，移到第 0 级（地面）后不再参与，不能操作者负。
结论：等价于只看奇数级台阶的 Nim，即各奇数级石子数的异或为 0 时先手必败。
思路：从奇数级往下移，相当于在 Nim 中取石子。对手若把偶数级的石子移到奇数级，自己就把同样数量再往下移一级，使奇数级恢复原状。
适用：石子只能单向移动、且终点不计分的模型，如树上把子结点的石子移给父亲（按深度奇偶看）、数轴上的硬币只能左移（相邻间隔就是“台阶”）。

9.1.8 反 Nim（取走最后一颗者负）

内容：Nim 规则不变，但取走最后一颗者负。
结论：先手必胜当且仅当以下二者之一成立：所有堆都 ≤ 1 且异或为 0（即 1 的堆数为偶数）；或至少有一堆 > 1 且异或不为 0。
思路：全为 1 时只看奇偶。存在大于 1 的堆时，先手按普通 Nim 取胜，只在最后一个大于 1 的堆处改为留 0 或 1，使剩下的 1 的堆数为奇数。
适用：反常规则的 Nim；一般反常游戏不能套 SG 异或，需要 SJ 定理的前提或直接搜索（第 162 页）。

9.1.9 Nim-k（每次至多动 k 堆）

内容：多堆，每次选 $1 \sim k$ 堆，每堆各取任意正数颗，取走最后一颗者胜。
结论：把各堆写成二进制，若每一位上为 1 的堆数都是 $k+1$ 的倍数，则先手必败。$k = 1$ 即 Nim。
思路：与 Nim 相同，只是把“按位异或”换成“按位求和模 $k+1$”：从这种局面一步最多改动 k 堆，不可能让每一位都保持模 $k+1$ 为 0；反之总能从高位到低位调整至多 k 堆，恢复为 0。
适用：一次可同时操作多堆、且各堆操作相同的模型。

9.1.10 树上删边（Green Hackenbush）

内容：有根树，每次删一条边，与根不连通的部分一起删除，不能操作者负。
结论：$g(v) = \bigoplus_{c \in \text{son}(v)} \big(g(c)+1\big)$，叶子为 0，$g(\text{根}) = 0$ 时先手必败。多棵树把各根的值异或。
思路：一条长为 ℓ 的链等价于大小为 ℓ 的 Nim 堆。同一结点下的几条分支互不影响，可以合并成一条长度为各自 SG 异或的链（Colon 原理）；在子树上方加一条边，SG 值加 1。
适用：删边或删子树的树上博弈。图中有环时需要“融合原理”，把环缩成链或自环后再算，不能直接套用此公式。

9.1.11 二分图博弈（不重复走点）

内容：无向图中一枚棋子放在 s，双方轮流把它移到一个未访问过的相邻点（s 视为已访问），不能移动者负。
结论：先手必胜当且仅当 s 在每一个最大匹配中都被匹配，即删去 s 后最大匹配变小。对一般图也成立：二分图用第 189 页 `BipartiteMatching`，一般图用第 192 页 `Blossom`，求一次最大匹配，删去 s 后再求一次比较。
思路：若 s 在每个最大匹配中，任取一个最大匹配 M，先手总沿 M 中当前点的匹配边走。后手若走到 M 的未匹配点，从 s 到该点是一条交错路，沿它翻转得到同样大小、却不含 s 的最大匹配，矛盾；所以后手每走一步，先手都还有匹配边可走。反之若有不含 s 的最大匹配 M，后手总沿 M 的匹配边应对；先手若走到 M 的未匹配点，从 s 到该点两端都未匹配，是增广路，与 M 最大矛盾，所以先手先无路可走。
适用：棋盘上走马、单词接龙等“走过的格子不能再走”的两人移动博弈；多个起点或询问时，逐个判断“该点是否一定被匹配”。

9.1.12 对称策略：环形取石子

内容：n 枚硬币围成一圈，每次取 1 枚或相邻 2 枚，取走最后一枚者胜。
结论：$n \le 2$ 时先手胜，$n \ge 3$ 时后手胜。
思路：先手第一步把环断成一条链，后手在链的正中取 1 或 2 枚，把链分成长度相等的两段，之后先手在一段中怎么取，后手就在另一段的对称位置照做。
适用：局面可以被对手的一步变成两个完全对称的部分（棋盘中心对称、环上断成两条等长链等）；对称位置必须总是可走，且自己的一步不能同时破坏两边。

9.1.13 策略窃取：Chomp

内容：$r \times c$ 的巧克力，左上角一格有毒。每次选一块，吃掉它以及右下方所有剩余块，吃到有毒那块者负。
结论：只要 $rc > 1$，先手必胜（不给出具体走法）。
思路：设先手先吃右下角一块。若后手对此有必胜应对，这个应对局面先手第一步就能直接走出来，因为吃掉的部分包含右下角。于是先手可以窃取后手的策略，所以后手不可能有必胜策略；有限且无和局时，先手必胜。
适用：只判断谁胜、且存在“先走一步不会更坏”的空步论证（如棋盘对称、多走一步无害）时；需要具体方案时仍需搜索。

第十章　数值算法

10.1　插值

10.1.1　拉格朗日插值与连续点优化

问题描述：已知 n 个横坐标两两不同的点 (x_i, y_i)，求唯一的次数小于 n 的多项式 f 在给定 k 处的值（模素数 p）。预处理一次后可多次查询；横坐标恰为 $0, 1, \ldots, n-1$ 时有更快的专用构造。

核心思路：拉格朗日公式写成 $f(k) = \sum_i w_i \prod_{j \neq i}(k - x_j)$，其中 $w_i = y_i / \prod_{j \neq i}(x_i - x_j)$ 只与样本有关，构造时存入 `w`：任意横坐标逐对相乘后求一次逆；横坐标连续时分母为 $i!\,(n-1-i)!$ 并带符号 $(-1)^{n-1-i}$，用逆阶乘表直接得到。`query` 先求前缀积 `pre`，再从右往左维护后缀积 `suffix`，两者相乘即去掉第 i 个因子的乘积。全程不对 $k - x_i$ 求逆，所以 k 恰为样本点时也正确。

接口契约：Lagrange(xs,ys,p) 使用任意横坐标，xs、ys 为等长 0-based vector<long long>；Lagrange(ys,p) 专用于横坐标 0..n-1；单次查询直接用 lagrange_consecutive。重新构造赋值可更换数据。要求 p 为素数，2≤p≤INT_MAX，点数 n≤p，任意横坐标取模后两两不同；不是仅要求原整数不同。query(k) 返回次数小于 n 的插值多项式在 k 处的值，允许 k、xs、ys 为完整 long long 范围，内部先归一化；空数据约定零多项式。通用预处理 O(n^2+n log p)，连续点预处理 O(n+log p)，每次查询 O(n)，空间 O(n)。查询利用前缀积与滚动后缀积，无需求逆，因此 k 与样本点模 p 相等时也正确。若求 1^d+⋯+k^d，要求 0≤d≤p-2，先计算 0..d+1 的前缀和作为 ys，再构造 Lagrange(ys,p)；用快速幂生成这些样本另花 O(d log(d+1))。仅处理素数域中的精确插值，不适用于浮点曲线拟合或重复节点的导数约束。

```
struct Lagrange
{
    using ll = long long;
    int n, mod;
    vector<ll> x, w;

    ll norm(ll a) const
    {
        a %= mod;
        if (a < 0) a += mod;
        return a;
    }

    ll power(ll a, int b) const
    {
        ll result = 1;
        while (b)
        {
            if (b & 1) result = result * a % mod;
            a = a * a % mod;
            b >>= 1;
        }
        return result;
    }

    Lagrange(const vector<ll> &xs, const vector<ll> &ys, int p)
    {
        n = ys.size();
        mod = p;
```

```
        assert(p >= 2 && n <= p && xs.size() == ys.size());
        x.resize(n);
        w.resize(n);
        for (int i = 0; i < n; i++) x[i] = norm(xs[i]);
        for (int i = 0; i < n; i++)
        {
            ll denominator = 1;
            for (int j = 0; j < n; j++)
                if (i != j) denominator = denominator * norm(x[i] - x[j]) % mod;
            assert(denominator != 0);
            w[i] = norm(ys[i]) * power(denominator, mod - 2) % mod;
        }
    }

    Lagrange(const vector<ll> &ys, int p)
    {
        n = ys.size();
        mod = p;
        assert(p >= 2 && n <= p);
        x.resize(n);
        w.resize(n);
        if (!n) return;
        vector<ll> invfac(n, 1);
        ll factorial = 1;
        for (int i = 1; i < n; i++) factorial = factorial * i % mod;
        invfac[n - 1] = power(factorial, mod - 2);
        for (int i = n - 1; i > 0; i--) invfac[i - 1] = invfac[i] * i % mod;
        for (int i = 0; i < n; i++)
        {
            x[i] = i;
            w[i] = norm(ys[i]) * invfac[i] % mod * invfac[n - 1 - i] % mod;
            if ((n - 1 - i) & 1) w[i] = norm(-w[i]);
        }
    }

    ll query(ll k) const
    {
        k = norm(k);
        vector<ll> pre(n + 1, 1);
        for (int i = 0; i < n; i++) pre[i + 1] = pre[i] * norm(k - x[i]) % mod;
        ll answer = 0, suffix = 1;
        for (int i = n - 1; i >= 0; i--)
        {
            answer = (answer + w[i] * pre[i] % mod * suffix) % mod;
            suffix = suffix * norm(k - x[i]) % mod;
        }
        return answer;
    }
};
```

模板题使用：Luogu P4781

题意：给出 n 个横坐标模素数后互不相同的点，求次数小于 n 的插值多项式在 k 处的值，模 998244353。使用任意点构造函数。

所需模板：Lagrange（第 165 页，10.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4781

```
int main()
{
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    int n;
    long long k;
    std::cin >> n >> k;
    std::vector<long long> x(n), y(n);
    for (int i = 0; i < n; i++) std::cin >> x[i] >> y[i];
    Lagrange poly(x, y, 998244353);
    std::cout << poly.query(k) << '\n';
}
```

10.1.2 横坐标连续的 O(n) 拉格朗日插值

问题描述：已知次数小于 n 的多项式 f 在 $0,1,\ldots,n-1$ 处的值，求单点值 $f(k) \bmod p$。常用于 $\sum_{i=1}^{k} i^d$ 这类关于 k 是多项式的量：先暴力算出前若干项，再插值到很大的 k。

核心思路：$f(k)=\sum_i y_i \prod_{j\neq i} \frac{k-j}{i-j}$。分子用前缀积 `pre` 与后缀积 `suf` 拼出 $\prod_{j\neq i}(k-j)$；分母 $\prod_{j\neq i}(i-j) = i!\,(n-1-i)!\,(-1)^{n-1-i}$，只用一次快速幂求 $(n-1)!$ 的逆，再递推出逆阶乘表 `ifac`，按 $n-1-i$ 的奇偶决定加或减该项。分子不做除法，因此 k 落在插值点上无需特判。

接口契约：lagrange_consecutive(ys,k,p) 已知 ys[i]=f(i)（i=0..n-1，deg f<n），返回 f(k) mod p。p 为素数，2≤p≤INT_MAX，n≤p；ys、k 可为任意 long long，内部取模；空 ys 返回 0。用前缀积、后缀积得所有 j≠i 的 (k-j) 之积，分母为 i!(n-1-i)! 并带符号 (-1)^(n-1-i)，只求一次逆，k 落在插值点上也正确。O(n+log p) 时间、O(n) 空间。横坐标为 a,a+1,…时把 k 换成 k-a；求 1^d+…+k^d 时取 0..d+1 的前缀和共 d+2 个样本。需要对同一组样本多次查询时用 Lagrange(ys,p)。

```cpp
// ys[i] = f(i) for i = 0..n-1 (deg f < n); returns f(k) mod prime p, n <= p. O(n + log p).
long long lagrange_consecutive(const vector<long long> &ys, long long k, int p)
{
    int n = ys.size();
    assert(p >= 2 && n <= p);
    if (!n) return 0;
    auto norm = [&](long long a) { return (a % p + p) % p; };
    auto power = [&](long long a, long long e)
    {
        long long r = 1;
        for (a = norm(a); e; e >>= 1, a = a * a % p)
            if (e & 1) r = r * a % p;
        return r;
    };
    k = norm(k);
    vector<long long> pre(n + 1, 1), suf(n + 1, 1), ifac(n, 1);
    for (int i = 0; i < n; i++) pre[i + 1] = pre[i] * norm(k - i) % p;
    for (int i = n - 1; i >= 0; i--) suf[i] = suf[i + 1] * norm(k - i) % p;
    long long fact = 1;
    for (int i = 1; i < n; i++) fact = fact * i % p;
    ifac[n - 1] = power(fact, p - 2);
    for (int i = n - 1; i > 0; i--) ifac[i - 1] = ifac[i] * i % p;
    long long answer = 0;
    for (int i = 0; i < n; i++)
    {
        // prod_{j != i} (k - j) / prod_{j != i} (i - j)
        long long term = norm(ys[i]) * pre[i] % p * suf[i + 1] % p * ifac[i] % p * ifac[n - 1 - i] % p;
        answer = ((n - 1 - i) & 1 ? answer - term + p : answer + term) % p;
    }
    return answer;
}
```

10.2 数值积分

10.2.1 有工作上限的自适应 Simpson 积分

问题描述：数值计算定积分 $\int_a^b f(x)\,\mathrm{d}x$：在绝对误差目标 `eps` 下自适应地加密采样，同时限制递归深度和总求值次数；返回近似值、误差估计、求值次数，以及是否满足局部误差条件。

核心思路：Simpson 公式 $S(l,r)=\frac{r-l}{6}\big(f(l)+4f(m)+f(r)\big)$。递归函数 `rec` 拿到区间、已算好的三个函数值与整体估计 s，只新算两个四分点，得到左右两半的估计 s_l, s_r；令 $\delta = s_l + s_r - s$，若 $|\delta|/15$ 不超过容差，就接受修正值 $s_l + s_r + \delta/15$；否则两半各分一半容差、深度减一继续递归。中点无法再分、求值预算将用完或出现非有限值时立即停止并标记未满足。反向区间先交换端点，最后取负。

思路中的情况	代码中的处理	行
零长度区间	不调用 f，直接返回 0	20
整体初值	求端点与中点的值和整区间估计	30
无法细分或预算不足	停止，返回粗估计且标记未满足	38
局部误差达标	接受修正值	48
深度用尽或容差下溢	返回当前修正值但标记未满足	49
继续细分	左右两半各用一半容差递归后相加	50

接口契约：integrate(f,a,b,eps,depth=20,limit=100000)，long double 绝对容差 eps>0，有限端点，0≤depth≤60、limit≥0。返回 value、error 估计、evaluations 与 met；met 仅指局部估计条件满足，不是实际精度保证。反向区间变号，零长度不调用 f；预算/深度/中点停滞/非有限计算会停止且 met=false。正常有限样本下保留粗或修正估计；无初始估计时 value 为 NaN；不可把失败值当答案。f 须返回确定值，异常原样传播，不能启用 fast-math。复用样本，O(求值数) 次固定字长运算、O(depth+1) 栈；需 numeric 的 midpoint、cmath、limits、utility。

```
struct AdaptiveSimpson
{
    using ld = long double;
    struct Result
    {
        ld value, error;
        int evaluations;
        bool met;
    };

    template<class F>
    static Result integrate(F &&f, ld a, ld b, ld eps,
                            int depth = 20, int limit = 100000)
    {
        const ld inf = numeric_limits<ld>::infinity();
        const ld nan = numeric_limits<ld>::quiet_NaN();
        if (!isfinite(a) || !isfinite(b) || !isfinite(eps) || eps <= 0 ||
            depth < 0 || depth > 60 || limit < 0)
            return {nan, inf, 0, false};
        if (a == b) return {0, 0, 0, true};
        if (limit < 3) return {nan, inf, 0, false};
        bool reverse = a > b;
        if (reverse) swap(a, b);
        auto area = [](ld l, ld r, ld u, ld v, ld w)
        {
            return (r - l) / 6 * (u + 4 * v + w);
        };
        ld fa = f(a), fm = f(midpoint(a, b)), fb = f(b);
        int used = 3;
        ld whole = area(a, b, fa, fm, fb);
        if (!isfinite(fa) || !isfinite(fm) || !isfinite(fb) || !isfinite(whole))
            return {nan, inf, used, false};
```

采样细分与预算合并（接上页同一 integrate 函数）：

```
        auto rec = [&](auto &&self, ld l, ld r, ld u, ld v, ld w,
                       ld s, ld tol, int left) -> Result
        {
            ld m = midpoint(l, r), x = midpoint(l, m), y = midpoint(m, r);
            if (m == l || m == r || x == l || x == m || y == m || y == r ||
                used > limit - 2)
                return {s, inf, 0, false};
            ld fx = f(x), fy = f(y);
            used += 2;
            ld sl = area(l, m, u, fx, v), sr = area(m, r, v, fy, w);
            ld sum = sl + sr, delta = sum - s;
            ld value = sum + delta / 15, error = abs(delta) / 15;
            if (!isfinite(fx) || !isfinite(fy) || !isfinite(sl) ||
                !isfinite(sr) || !isfinite(value) || !isfinite(error))
                return {s, inf, 0, false};
            if (error <= tol) return {value, error, 0, true};
            if (!left || tol / 2 == 0) return {value, error, 0, false};
            auto p = self(self, l, m, u, fx, v, sl, tol / 2, left - 1);
            auto q = self(self, m, r, v, fy, w, sr, tol / 2, left - 1);
            value = p.value + q.value;
            error = p.error + q.error;
            if (!isfinite(value)) return {s, inf, 0, false};
            return {value, error, 0, p.met && q.met && error <= tol};
        };
        auto answer = rec(rec, a, b, fa, fm, fb, whole, eps, depth);
        if (reverse) answer.value = -answer.value;
        answer.evaluations = used;
        return answer;
    }
};
```

10.2.2 数值积分：局部估计、误差预算与采样盲区

接口见第 10.2.1 节（第 169 页）。前提：定积分存在，采样点有定义；先按间断、尖峰、振荡分段，端点奇异须变量变换，不能只减小 eps。

$$S(a,b)=\frac{b-a}{6}\big(f(a)+4f(m)+f(b)\big),\qquad \int_a^b f-S(a,b)=-\frac{(b-a)^5}{2880}f^{(4)}(\xi),$$

对至多三次多项式精确。$S_2=S(a,m)+S(m,b)$，$\delta=S_2-S(a,b)$，误差指标 $|\delta|/15$，修正值 $S_2+\delta/15$；递归时误差预算对半分。error 是指标之和，`met` 不是实际误差证书；光滑函数也可能被初始五点漏采样。

eps 为正的绝对容差；depth=0 仍做根的五点比较；limit 为回调次数上限。深度耗尽、容差下溢、预算不足、四分点与端点重合或非有限值都令 `met=false`，此时 value 不可直接输出（可能为 NaN）。零长度不调用函数，反向区间取负。

10.2.3 数值算法与几何

排序必须是严格弱序，不能以 “差小于 eps 则相等” 作比较器。叉积与行列式易抵消；整数题优先精确整数判定。自适应积分要求函数足够规则，奇点、间断、强振荡先分段。三分要求单峰（整数域最后枚举剩余区间）；二分答案要求判定单调。

10.3 高斯消元

10.3.1 压位模 2 高斯消元与全部解

问题描述：给定 $\mathbb{F}_2$ 上 m 个方程、n 个未知数的线性方程组（每行一个 01 串，末位为右端项），判断是否有解；有解时求出一个特解和齐次方程组解空间的一组基，全部解就是特解异或任意一组基向量。

核心思路：每行按 64 位压成 `unsigned long long` 数组后做 Gauss–Jordan 消元：逐列寻找主元行换到当前行 `row`，并用它消去**所有**其他行在该列的 1，`pos[col]` 记下主元所在行。消元后每个主元变量只出现在自己那一行。秩以下的行若右端为 1 则无解。特解令自由变量全为 0，主元变量取所在行的右端值；对每个自由列 j，令 $x_j=1$、其余自由变量为 0，主元变量取所在行第 j 列的值，得到一个零空间基向量。

思路中的情况	代码中的处理	行
压位读入	第 j 个字符写入第 `j/64` 个字的第 `j%64` 位	23
找主元	在当前行及以下找该列为 1 的行，找不到则该列是自由列	34
消去其他行	主元行异或进所有该列为 1 的其他行，只需从第 `col/64` 个字开始	38
判无解	零行的右端为 1	43
特解	主元变量取其行右端值	46
零空间基	每个自由列生成一个基向量	51

接口契约：GaussXor::solve(a,n)，a 为 m 行 n+1 列的 01 字符串增广矩阵，0≤n<INT_MAX，最末字符为右端项。返回 consistent、系数秩 rank、01 字符串特解 particular 与零空间基 kernel；所有解为特解异或任意基子集。无解时解数组为空；rank 仍为系数秩。有解时 kernel.size()=n-rank，唯一解的 kernel 为空。允许空方程组或零变量，原输入不变。自由列按原列号递增，基在各自自由列为 1、其他自由列为 0；没有最少 1 个数保证。时间 O(mn+mn min(m,n)/64+n^2)，除输入外空间 O(m ceil((n+1)/64)+n^2)；输出本身可能有 n^2 字符。全程精确异或，不用 eps。

```
struct GaussXor
{
    struct Solution
    {
        bool consistent;
        int rank;
        string particular;
        vector<string> kernel;
    };

    // m rows of n+1 characters: coefficients followed by the right-hand side.
    static Solution solve(const vector<string> &a, int n)
    {
        assert(n >= 0);
        int m = a.size(), w = n / 64 + 1, row = 0;
        vector<vector<unsigned long long>> b(m, vector<unsigned long long>(w));
        for (int i = 0; i < m; i++)
        {
            assert(a[i].size() == (size_t)n + 1);
            for (int j = 0; j <= n; j++)
            {
                assert(a[i][j] == '0' || a[i][j] == '1');
                if (a[i][j] == '1') b[i][j / 64] |= 1ULL << (j % 64);
            }
        }
        auto get = [&](int i, int j)
        {
            return b[i][j / 64] >> (j % 64) & 1;
        };
        vector<int> pos(n, -1);
        for (int col = 0; col < n && row < m; col++)
        {
            int p = row;
            while (p < m && !get(p, col)) p++;
            if (p == m) continue;
            swap(b[p], b[row]);
            for (int i = 0; i < m; i++)
                if (i != row && get(i, col))
                    for (int j = col / 64; j < w; j++) b[i][j] ^= b[row][j];
            pos[col] = row++;
        }
        for (int i = row; i < m; i++)
            if (get(i, n)) return {false, row, {}, {}};
```

构造特解与零空间基（接上页 solve）（接上页同一结构体）：

```
        Solution ans{true, row, string(n, '0'), {}};
        for (int j = 0; j < n; j++)
            if (pos[j] != -1) ans.particular[j] += get(pos[j], n);
        for (int j = 0; j < n; j++)
            if (pos[j] == -1)
            {
                string v(n, '0');
                v[j] = '1';
                for (int k = 0; k < n; k++)
                    if (pos[k] != -1) v[k] += get(pos[k], j);
                ans.kernel.push_back(v);
            }
        return ans;
    }
};
```

模板题使用：Library Checker system_of_linear_equations_mod_2

题意：给定模 2 矩阵 A 和右端 b，输出无解-1，或解空间维数、一个特解和全部零空间基。输出维数是变量数减系数秩，不是 rank。输入每行是连续 01 字符，先把 b 追加到对应行；行列数各至 4096。

所需模板：GaussXor（第 172 页，10.3.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/system_of_linear_equations_mod_2

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int m, n;
    cin >> m >> n;
    vector<string> a(m);
    for (auto &s : a) cin >> s;
    string b;
    cin >> b;
    for (int i = 0; i < m; i++) a[i] += b[i];
    auto sol = GaussXor::solve(a, n);
    if (!sol.consistent)
    {
        cout << -1 << '\n';
        return 0;
    }
    cout << sol.kernel.size() << '\n';
    cout << sol.particular << '\n';
    for (auto &v : sol.kernel) cout << v << '\n';
}
```

10.3.2 素数模高斯消元与全部解

问题描述：求解模素数 p 的线性方程组 $Ax = b$（给出 m 行增广矩阵、n 个未知数）：判断是否有解，有解时给出系数矩阵的秩、一个特解和齐次方程组解空间的一组基，从而描述全部解。

核心思路：高斯—若尔当消元化为简化行阶梯形：逐列找本列非零的行作主元，换到当前行 `row`，整行乘主元的逆归一，再消去其他所有行的该列。`where[col]` 记录主元列所在的行，没有主元的列是自由变量。消元后秩以

下各行若右端非零则无解。特解令自由变量为 0，主元变量取所在行的右端；每个自由变量 j 生成一个基向量：第 j 维为 1，各主元变量取 $-a_{\text{where}[k],j}$。

代码依赖：ModInt（第 12 页，3.1.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约： GaussMod<p>::solve(a,n) 接收 m 行 n+1 列增广矩阵，p 必须是素数，n≥0。返回 consistent、系数矩阵秩 rank、特解 particular 和零空间基 kernel；解为特解加基向量的任意线性组合。无解时解数组为空；空方程组需显式给出变量数。O(m n min(m,n)+n^2) 时间，矩阵及解空间输出 O(mn+n^2) 空间。只依赖 ModInt，不含行列式或矩阵树。

```
template <int mod> struct GaussMod
{
    using Z = ModInt<mod>;
    using Matrix = vector<vector<Z>>;

    struct Solution
    {
        bool consistent;
        int rank;
        vector<Z> particular;
        Matrix kernel;
    };

    // Augmented m x (n+1), prime modulus. Empty system needs explicit n.
    static Solution solve(Matrix a, int n)
    {
        int m = (int)a.size(), row = 0;
        vector<int> where(n, -1);
        for (const auto &v : a) assert((int)v.size() == n + 1);
        for (int col = 0; col < n && row < m; col++)
        {
            int p = row;
            while (p < m && !a[p][col].v) ++p;
            if (p == m) continue;
            swap(a[p], a[row]);
            Z inv = a[row][col].inv();
            for (int j = col; j <= n; j++) a[row][j] = a[row][j] * inv;
            for (int i = 0; i < m; i++)
                if (i != row && a[i][col].v)
                {
                    Z f = a[i][col];
                    for (int j = col; j <= n; j++) a[i][j] = a[i][j] - f * a[row][j];
                }
            where[col] = row++;
        }
        for (int i = row; i < m; i++)
            if (a[i][n].v) return {false, row, {}, {}};
```

```
        Solution ans{true, row, vector<Z>(n), {}};
        for (int j = 0; j < n; j++)
            if (where[j] != -1) ans.particular[j] = a[where[j]][n];
        for (int j = 0; j < n; j++)
            if (where[j] == -1)
            {
                vector<Z> v(n);
                v[j] = 1;
                for (int k = 0; k < n; k++)
                    if (where[k] != -1) v[k] = Z(0) - a[where[k]][j];
                ans.kernel.push_back(v);
            }
        return ans;
    }
};
```

模板题使用：Library Checker system_of_linear_equations

题意：模 998244353 下解 n 个方程、m 个未知数的线性方程组。输入先给系数矩阵，再给常数列。无解输出 -1；否则输出解空间维数、一个特解和齐次解基。

所需模板：ModInt（第 12 页，3.1.2 节）、GaussMod（第 174 页，10.3.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/system_of_linear_equations

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    using G = GaussMod<998244353>;
    int n, m;
    cin >> n >> m;
    G::Matrix a(n, vector<G::Z>(m + 1));
    for (auto &row : a)
        for (int j = 0; j < m; j++) cin >> row[j].v;
    for (auto &row : a) cin >> row[m].v;
    auto s = G::solve(move(a), m);
    if (!s.consistent)
    {
        cout << -1 << '\n';
        return 0;
    }
    cout << s.kernel.size() << '\n';
    auto print = [&](const vector<G::Z> &row)
    {
        for (int j = 0; j < m; j++) cout << row[j].v << (j + 1 == m ? '\n' : ' ');
    };
    print(s.particular);
    for (auto &row : s.kernel) print(row);
}
```

10.3.3 实数高斯消元与数值解空间

问题描述：用 `long double` 浮点数求解实系数线性方程组 $Ax = b$：在容差 `eps` 下判断是否相容，给出数值秩、一个特解和齐次解空间的一组基。

核心思路：先把每行按其系数最大绝对值缩放，使各方程量级一致。然后逐列做部分选主元的高斯—若尔当消元：在未用行中取本列绝对值最大者为主元；若它不超过 `eps`，视该列为零，把活动行的这一列清零后跳过（成为自由变量）；否则换行、主元行归一，再消去其他所有行的该列，`where` 记录主元所在行。剩余行右端绝对值超过 `eps` 判为不相容；特解令自由变量为 0，每个自由变量生成一个基向量，主元变量从对应行读出。

接口契约：solve(a, n, eps) 接收 m×(n+1) 增广矩阵，0≤n<INT_MAX、m≤INT_MAX，输入/中间值须有限可表示，0<eps<1。long double 精度依平台；先按每行系数最大绝对值归一化（不含右端项），再部分选主元；绝对值≤eps 的候选列在活动行中明确置零。对所有非零消元因子更新，不以 eps 跳过。返回 consistent、rank、particular 和 kernel：均描述最终数值消元模型，不是原系统精确可解性/秩的证书；剩余右端项绝对值 ≤eps 按零处理。原全零系数行保留原始右端尺度，n=0 也用此约定。相容时自由变量取零给特解，kernel 列出自由变量方向；任意大线性组合的原方程误差不受 eps 统一保证。非相容时两个解数组为空；输入按值传入不变。主消元通常 O(mn min(m,n)) 时间，另有初始化；O(m(n+1)+n) 工作空间，另有 O(n(n−rank)) 解基输出。近阈值、病态或非有限过程不能当成数学无解；须结合原方程残差与问题条件判断。正常浮点语义，禁 fast-math；assert 仅调试诊断，非发布模式运行时校验。

```
struct GaussReal
{
    using R = long double;
    using Matrix = vector<vector<R>>;

    struct Solution
    {
        bool consistent;
        int rank;
        vector<R> particular;
        Matrix kernel;
    };

    // Numerical classification after coefficient-row scaling, not exact rank.
    static Solution solve(Matrix a, int n, R eps)
    {
        assert(0 <= n && n < INT_MAX && a.size() <= (size_t)INT_MAX);
        assert(isfinite(eps) && 0 < eps && eps < 1);
        int m = a.size(), row = 0;
        vector<int> where(n, -1);
        for (auto &v : a)
        {
            assert(v.size() == (size_t)n + 1);
            R s = 0;
            for (int j = 0; j <= n; j++)
            {
                assert(isfinite(v[j]));
                if (j < n) s = max(s, abs(v[j]));
            }
            if (s > 0)
                for (R &x : v)
                {
                    x /= s;
                    assert(isfinite(x));
                }
        }
```

主元消元与数值解空间（接上页同一 solve 函数）：

```
        for (int col = 0; col < n && row < m; col++)
        {
            int p = row;
            for (int i = row + 1; i < m; i++)
                if (abs(a[i][col]) > abs(a[p][col])) p = i;
            if (abs(a[p][col]) <= eps)
            {
                for (int i = row; i < m; i++) a[i][col] = 0;
                continue;
            }
            swap(a[p], a[row]);
            R pivot = a[row][col];
            for (int j = col + 1; j <= n; j++)
            {
                a[row][j] /= pivot;
                assert(isfinite(a[row][j]));
            }
            a[row][col] = 1;
            for (int i = 0; i < m; i++)
                if (i != row && a[i][col] != 0)
                {
                    R f = a[i][col];
                    for (int j = col + 1; j <= n; j++)
                    {
                        a[i][j] -= f * a[row][j];
                        assert(isfinite(a[i][j]));
                    }
                    a[i][col] = 0;
                }
            where[col] = row++;
        }
        for (int i = row; i < m; i++)
            if (abs(a[i][n]) > eps) return {false, row, {}, {}};
        Solution ans{true, row, vector<R>(n), {}};
        for (int j = 0; j < n; j++)
            if (where[j] != -1) ans.particular[j] = a[where[j]][n];
        for (int j = 0; j < n; j++)
            if (where[j] == -1)
            {
                vector<R> v(n);
                v[j] = 1;
                for (int k = 0; k < n; k++)
                    if (where[k] != -1) v[k] = -a[where[k]][j];
                ans.kernel.push_back(v);
            }
        return ans;
    }
};
```

10.3.4 模线性方程组：工厂工期

POJ2947：星期一为 0、星期日为 6，产品 j 在记录 i 中出现 a_{ij} 次（重复累加），

$$\sum_j a_{ij}x_j \equiv \text{finish}_i - \text{start}_i + 1 \pmod 7.$$

用第 174 页的模高斯消元，模 7，增广矩阵 $m \times (n+1)$。先判 `consistent`（假则无解），再看秩小于 n 为多解，否则特解中小于 3 的分量加 7。合数模数不能直接套用此消元。

10.3.5 模 2 压位消元与完整解空间

`GaussXor::solve(a,n)` 输入 m 行 $n+1$ 列 01 串，末列为右端项。自由变量全设零得特解；每次一个自由变量设一得零空间基。全部解为特解异或任意基子集，维数 $n - rank$。每行 $\lfloor n/64 \rfloor + 1$ 个字；无解与零变量有解都可能返回空特解，须查 `consistent`。输出完整基最坏 n^2 个字符；特解不保证一的个数最少。

10.3.6 模 2 消元后的最少开关用法

每个方格一个变量，系数为自身及四邻，右端项为初始颜色。解集为特解异或零空间基任意子集，须枚举取一最少者；无解输出 `inf`。自由维数 d 时代价 $O(n^2 2^d)$；对此四邻棋盘 $d \le n$，不可推广到任意方程组。

第十一章　Berlekamp–Massey 算法

11.1　Berlekamp–Massey：最短线性递推

问题描述：给定模素数的有限序列 $s_0, \ldots, s_{N-1}$，求阶数最小的线性递推 $s_i = \sum_{j=1}^{k} c_{j-1} s_{i-j}$（对所有 $i \geq k$ 成立）。常与求线性递推第 n 项配合，用于“打表找递推”。

核心思路：维护当前连接多项式 `c`（$c_0 = 1$，满足 $\sum_{j=0}^{L} c_j s_{i-j} = 0$，$L$ 即 `len`），以及上次长度变化前的多项式 `b`、它的失配值 `last` 和相对位移 `shift`。读入 s_i 时算失配 d；$d = 0$ 只增加位移。否则令 $c \leftarrow c - \frac{d}{\text{last}} x^{\text{shift}} b$ 消去本次失配；若 $2L \leq i$，长度必须增长为 $i + 1 - L$，修改前的 `c` 成为新的 `b`。最后把 $c_1, \ldots, c_L$ 取负即为递推系数。

接口契约：berlekamp_massey(s,p) 要求 p 为素数，2≤p≤INT_MAX，输入均在 [0,p)；返回 c，使 s[i]=$\sum$ c[j-1]s[i-j] (j=1..k，i≥k)。空序列或全零序列返回空系数。有限前缀可能对应多种未来，不能无条件外推；若已知原序列满足阶数至多 d 的齐次线性递推，提供至少 2d 项才能恢复可外推的递推。模运算次数 O(Nk+N log p)，后项来自逐次求逆；空间 O(N)。不适用于合数模数或浮点拟合。

```
// Prime p; normalized input. s[i] = sum(c[j-1] * s[i-j]), i >= c.size().
vector<int> berlekamp_massey(const vector<int> &s, int p = 998244353)
{
    assert(p >= 2);
    vector<int> c{1}, b{1};
    int len = 0, shift = 1, last = 1;
    for (int i = 0; i < (int)s.size(); i++)
    {
        assert(0 <= s[i] && s[i] < p);
        long long d = s[i];
        for (int j = 1; j <= len; j++) d = (d + 1LL * c[j] * s[i - j]) % p;
        if (d == 0)
        {
            shift++;
            continue;
        }
        long long inv = 1, x = last;
        for (int e = p - 2; e; e >>= 1)
        {
            if (e & 1) inv = inv * x % p;
            x = x * x % p;
        }
        long long factor = d * inv % p;
        auto old = c;
        if (c.size() < b.size() + shift) c.resize(b.size() + shift);
        for (int j = 0; j < (int)b.size(); j++)
            c[j + shift] = (c[j + shift] - factor * b[j] % p + p) % p;
        if (2 * len <= i)
        {
            len = i + 1 - len;
            b = old;
            last = d;
            shift = 1;
```

```
        }
        else
            shift++;
    }
    c.resize(len + 1);
    c.erase(c.begin());
    for (int &x : c) x = x == 0 ? 0 : p - x;
    return c;
}
```

模板题使用：Library Checker find_linear_recurrence

题意：给模 998244353 下的有限序列，求最短线性递推。c[0] 乘前一项，输出阶数及系数；空序列或全零序列可以是 0 阶。递推只保证对已给序列成立，不能仅凭前缀推定未知后续。

所需模板：berlekamp_massey（第 181 页，11.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/find_linear_recurrence

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<int> a(n);
    for (int &x : a) cin >> x;
    auto c = berlekamp_massey(a);
    cout << c.size() << '\n';
    for (int x : c) cout << x << ' ';
    cout << '\n';
}
```

联合使用示例见第 102 页（Luogu P5487），其中直接调用本模板。

第一部分

跨类依赖

附录 A　树状数组

A.1　树状数组第 k 小与组合依赖

功能：维护长度为 n 的数组，支持单点加、前缀和与区间和；元素表示频数（均非负）时还能求“前缀和首次达到 k 的位置”，即可重集合的第 k 小。

核心思路：a[x] 存原数组在 $(x-\mathrm{lowbit}(x),x]$ 上的和。前缀和沿 $x \leftarrow x-\mathrm{lowbit}(x)$ 累加，单点修改沿 $x \leftarrow x+\mathrm{lowbit}(x)$ 更新所有覆盖它的节点，各 $O(\log n)$。第 k 小从高位到低位倍增：若跳过下一段后累计仍小于 k 就跳过去。

接口契约：保留频数第 k 小和泛型组合用途；P3374 基础单点加/区间和题不再独立收录或安排提交。add(x,v)、sum(x)、query(l,r)，下标 1..n，闭区间；kth(k) 要求所有频率非负且 k>0，超过总和返回 n+1。单次 O(log n)。T 须能从 0 构造并支持 += 与减法；可直接使用 ModInt 维护模数下的区间和。kth 只适用于有序累计频数，不适用于取模后的区间和。

```
template <class T = long long> struct Fenwick
{
    int n;
    vector<T> a;

    Fenwick(int n = 0) : n(n), a(n + 1) {}

    void add(int x, T v)
    {
        assert(x >= 1);
        for (; x <= n; x += x & -x) a[x] += v;
    }

    T sum(int x) const
    {
        T ans = 0;
        for (; x; x -= x & -x) ans += a[x];
        return ans;
    }

    T query(int l, int r) const { return sum(r) - sum(l - 1); }

    // Nonnegative frequencies; returns n+1 if k exceeds total.
    int kth(T k) const
    {
        assert(k > 0);
        int x = 0, b = 1;
        while (b <= n / 2) b *= 2;
        for (; b; b >>= 1)
            if (x + b <= n && a[x + b] < k)
            {
                k -= a[x + b];
                x += b;
            }
        return x + 1;
    }
};
```

模板题使用：Luogu P3369

题意：维护可重集合的插入、删除一个、排名、第 k 小和严格前驱/后继。预读全部操作并离散化非第 k 小查询的值域；频数下标为 1-based，kth 的返回值减 1 后映射回原值。排名查询允许 x 不在集合中；第 k 小与前驱/后继按题目保证存在。n≤100000，|x|≤10000000；总复杂度 O(n log n)，空间 O(n)。

所需模板：Fenwick（第 184 页，A.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3369

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    if (!(cin >> n)) return 0;
    vector<pair<int, int>> q(n);
    vector<int> a;
    for (auto &[op, x] : q)
    {
        cin >> op >> x;
        if (op != 4) a.push_back(x);
    }
    sort(a.begin(), a.end());
    a.erase(unique(a.begin(), a.end()), a.end());
    Fenwick<int> f(a.size());
    for (auto [op, x] : q)
    {
        int p = lower_bound(a.begin(), a.end(), x) - a.begin();
        if (op == 1)
            f.add(p + 1, 1);
        else if (op == 2)
        {
            if (f.query(p + 1, p + 1) > 0) f.add(p + 1, -1);
        }
        else if (op == 3)
            cout << f.sum(p) + 1 << '\n';
        else if (op == 4)
            cout << a[f.kth(x) - 1] << '\n';
        else if (op == 5)
            cout << a[f.kth(f.sum(p)) - 1] << '\n';
        else
        {
            p = upper_bound(a.begin(), a.end(), x) - a.begin();
            cout << a[f.kth(f.sum(p) + 1) - 1] << '\n';
        }
    }
}
```

附录 B　最短路问题

B.1　最短路

B.1.1　Floyd 全源最短路与原边方案

问题描述：给定可含负权、重边的有向图，求所有点对之间的最短距离，并能输出任一点对的一条最短路（原边序列）；若图中有负环则报告失败。

核心思路：`dis[u][v]` 初值取 $u \to v$ 重边中的最小权，`nxt[u][v]` 记最短路的第一条原边。按 $k = 1..n$ 逐个允许 k 作中转点，用 $\mathrm{dis}[u][k] + \mathrm{dis}[k][v]$ 更新 $\mathrm{dis}[u][v]$，更新时第一条边继承 `nxt[u][k]`；只有两段都可达才相加。处理 k 前若 $\mathrm{dis}[k][k] < 0$ 说明存在经过 k 的负环，立即返回。输出路径时从 s 反复走 `nxt[s][t]` 这条边，直到到达 t。

接口契约：Floyd(n)，点号 1..n，0<n<INT_MAX；add(u,v,w) 返回 0-based 原边号，有向边，允许重边、自环及完整 signed64 权值，边数 ≤INT_MAX。无向图需分别加入双向边。run() 重新建立矩阵；存在任何负环返回 false，valid=false，全部结果不可读，不能只取未受负环影响的点对。

成功时 dis[u][v]==inf 为不可达，其他为 int128 最短距离，dis[u][u]=0。path(u,v) 返回正向原边号，并列任选；不可达和 u=v 均为空，结合 dis 区分。nxt 为恢复路径的首条原边号矩阵，通常只需调用 path；e/dis/nxt/valid 只读。add 清除 valid，支持重复运行、加边后重跑与独立值拷贝。

run 时间 $O(n^3+m)$，空间 $O(n^2+m)$，path 时间与输出空间 O(路径长)。每次处理 k 之前检查 dis[k][k]<0，立即拒绝，不能延后到所有松弛结束，否则负环下中间量可指数增长。通过的枢轴不含负环；有限标签取简单路径或闭合路径，拼接候选绝对值 $<2n \cdot 2\hat{}63<2\hat{}95$，int128 与 inf=2^126 足够。两段都有限才相加；不要无条件转 long long。矩阵占用约 $20(n+1)^2$ 字节，另计 vector 和原边表，n 应按时间/内存选择。

```
struct Floyd
{
    static constexpr long long inf = LLONG_MAX / 4;
    struct Edge
    {
        int u, v;
        long long w;
    };
    int n;
    vector<Edge> e;
    vector<vector<long long>> dis;
    vector<vector<int>> nxt;

    Floyd(int n) : n(n) {}

    int add(int u, int v, long long w)
    {
        e.push_back({u, v, w});
        return int(e.size()) - 1;
    }

    // 1-based. False on a negative cycle; then every result is invalid.
    bool run()
    {
        dis.assign(n + 1, vector<long long>(n + 1, inf));
        nxt.assign(n + 1, vector<int>(n + 1, -1));
        for (int u = 1; u <= n; u++) dis[u][u] = 0;
```

```
        for (int id = 0; id < (int)e.size(); id++)
        {
            auto [u, v, w] = e[id];
            if (w < dis[u][v]) dis[u][v] = w, nxt[u][v] = id;
        }
        for (int k = 1; k <= n; k++)
        {
            if (dis[k][k] < 0) return false;
            for (int u = 1; u <= n; u++)
                if (dis[u][k] != inf)
                    for (int v = 1; v <= n; v++)
                        if (dis[k][v] != inf && dis[u][k] + dis[k][v] < dis[u][v])
                            dis[u][v] = dis[u][k] + dis[k][v], nxt[u][v] = nxt[u][k];
        }
        return true;
    }

    // Edge ids from s to t; empty if unreachable or s=t.
    vector<int> path(int s, int t) const
    {
        vector<int> ids;
        if (dis[s][t] == inf) return ids;
        for (; s != t; s = e[ids.back()].v) ids.push_back(nxt[s][t]);
        return ids;
    }
};
```

模板题使用：AOJ GRL_1_C / All Pairs Shortest Path

题意：0-based 有向图输入转换为 1-based；整图有负环输出 NEGATIVE CYCLE，否则逐行输出距离矩阵，不可达为 INF。n≤100、m≤9900、|w|≤20000000，有限结果可转 long long。题面无自环重边，模板额外支持。

所需模板：Floyd（第 186 页，B.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.u-aizu.ac.jp/onlinejudge/description.jsp?id=GRL_1_C&lang=en

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    Floyd t(n);
    while (m--)
    {
        int u, v;
        long long w;
        cin >> u >> v >> w;
        t.add(u + 1, v + 1, w);
    }
    if (!t.run())
    {
        cout << "NEGATIVE CYCLE\n";
        return 0;
    }
    for (int u = 1; u <= n; u++)
    {
        for (int v = 1; v <= n; v++)
        {
            if (v > 1) cout << ' ';
            if (t.dis[u][v] == Floyd::inf) cout << "INF";
            else cout << (long long)t.dis[u][v];
        }
        cout << '\n';
    }
}
```

附录 C　图的匹配

C.1　二分图最大匹配

C.1.1　Hopcroft–Karp 最大匹配与最小点覆盖

问题描述：给定二分图，用 Hopcroft–Karp 求最大匹配并给出每个点的匹配对象；求完后还能构造最小点覆盖（其补集为最大独立集）。

核心思路：每个阶段 `bfs` 从所有未匹配左点出发，按“左点、右点、该右点的匹配左点”分层，未匹配的右点用虚点 0 表示，`dep[0]` 即最短增广路的层数，更深的层不再扩展。随后对每个未匹配左点 `dfs`，只走层数加一的方向并使用当前弧，找到增广路就沿途翻转匹配；搜索失败的点把 `dep` 设为无效，不再进入。由 König 定理，最后一次 BFS 后，未被交替路径到达的左点与已匹配且其匹配左点被到达的右点构成最小点覆盖。

接口契约：左右两侧分别编号 1..n、1..m，可有空侧及重边；add(u,v) 加边，solve() 返回最大匹配总数，l[u] 与 r[v] 为匹配端点，0 表示未匹配。保留已有合法匹配，可加边后再次 solve；不支持删边或手动更改匹配数组。BFS 用 dep[0] 记录最短终点层，递归 DFS 只沿最短层增广，并使用当前弧。V=n+m，单次 solve 最坏 $O((V+E)\sqrt{V})$，空间 O(V+E)。cover() 在 solve 后、下一次改图前调用，返回 (左侧点列表, 右侧点列表)，为最小点覆盖；其补集是最大独立集。覆盖与匹配方案均可直接输出。

```
struct BipartiteMatching
{
    int n, m;
    vector<vector<int>> g;
    vector<int> l, r, dep, cur;

    BipartiteMatching(int n, int m)
        : n(n), m(m), g(n + 1), l(n + 1), r(m + 1), dep(n + 1), cur(n + 1)
    {
    }

    void add(int u, int v)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= m);
        g[u].push_back(v);
    }

    bool bfs()
    {
        queue<int> q;
        fill(dep.begin(), dep.end(), n + 1);
        fill(cur.begin(), cur.end(), 0);
        for (int u = 1; u <= n; u++)
        {
            if (!l[u])
            {
                dep[u] = 0;
                q.push(u);
            }
        }
        while (!q.empty())
        {
            int u = q.front();
```

```
34              q.pop();
35              if (dep[u] >= dep[0]) continue;
36              for (int v : g[u])
37              {
38                  int next = r[v];
39                  if (dep[next] != n + 1) continue;
40                  dep[next] = dep[u] + 1;
41                  if (next) q.push(next);
42              }
43          }
44          return dep[0] != n + 1;
45      }
```

增广、求解与最小点覆盖（接上页结构体）：

```
    bool dfs(int u)
    {
        if (!u) return true;
        for (int &i = cur[u]; i < (int)g[u].size(); i++)
        {
            int v = g[u][i];
            if (dep[r[v]] == dep[u] + 1 && dfs(r[v]))
            {
                l[u] = v;
                r[v] = u;
                return true;
            }
        }
        dep[u] = n + 1;
        return false;
    }

    // Retains an existing matching; edges may be added between calls.
    int solve()
    {
        int answer = 0;
        for (int u = 1; u <= n; u++) answer += (l[u] != 0);
        while (bfs())
        {
            for (int u = 1; u <= n; u++)
            {
                if (!l[u] && dfs(u)) answer++;
            }
        }
        return answer;
    }

    // Call after solve(), before adding more edges. Returns left/right vertex IDs.
    pair<vector<int>, vector<int>> cover() const
    {
        vector<int> left, right;
        for (int u = 1; u <= n; u++)
        {
            if (dep[u] == n + 1) left.push_back(u);
        }
        for (int v = 1; v <= m; v++)
        {
            if (r[v] && dep[r[v]] != n + 1) right.push_back(v);
        }
        return {left, right};
    }
};
```

模板题使用：Luogu P3386

题意：给定二分图，求最大匹配数。左右两侧分别编号 1..n、1..m，add(u,v) 的 u 属于左侧，v 属于右侧；solve 返回匹配数。此题不输出匹配或最小点覆盖方案。

所需模板：BipartiteMatching（第 189 页，C.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3386

```
int main()
{
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    int n, m, e;
    std::cin >> n >> m >> e;
    BipartiteMatching graph(n, m);
    while (e--)
    {
        int u, v;
        std::cin >> u >> v;
        graph.add(u, v);
    }
    std::cout << graph.solve() << '\n';
}
```

C.2 一般图最大匹配

C.2.1 一般图最大基数匹配：带花树

问题描述：一般图（不要求二分）最大匹配：给定无向图，选出尽量多两两不共端点的边，返回匹配边数及每个点的配对点。

核心思路：Edmonds 带花树。依次以每个未匹配点为根做 BFS 建交错树：队列中的点是外点（偶层），`pre[v]` 记奇层点 v 是从哪个外点走到的，`base` 记点所在花的花根。从外点 u 看边 u–v：若 v 也是外点，两条到根的交错路径加这条边构成奇环，用 `lca` 找花根，`mark` 沿环补齐 `pre`，把环上各点的 `base` 改为花根，环上原内点变为外点入队；若 v 未访问且未匹配，找到增广路，沿 `pre`/`mate` 交替翻转；若 v 已匹配，v 成为内点，其配偶成为外点入队。某点一次找不到增广路，以后也不会再有，故每点只试一次。

思路中的情况	代码中的处理	行
逐点尝试增广	对每个仍未匹配的点做一次 BFS	108
无用的边	同一朵花内部的边或已是匹配边，跳过	59
外点连外点	出现奇环：求花根、补 `pre`、缩花，新外点入队	60
到达未匹配点	沿 `pre` 与 `mate` 回溯翻转整条增广路	80
到达已匹配点	v 作内点，其配偶作外点入队	92

接口契约：Blossom(n)，点号 0..n-1；add(u,v) 加无向边，忽略自环，允许重边。solve() 从空匹配重新计算，返回匹配边数；mate[u] 为配对点，未匹配为 -1。结果支持直接输出方案。缩花维护交错树，单次增广 $O(n^2+m)$，总计 $O(n^3+nm)$，简单图为 $O(n^3)$；空间 $O(n+m)$。重复调用 solve 或加边后重算均可。不处理边权，不能代替最大权匹配。

```
struct Blossom
{
    int n;
    vector<vector<int>> g;
    vector<int> mate, pre, base, q;
```

```
    vector<bool> outer, flower, seen;

    Blossom(int n)
        : n(n), g(n), mate(n, -1), pre(n), base(n), outer(n), flower(n), seen(n)
    {
    }

    void add(int u, int v)
    {
        assert(0 <= u && u < n && 0 <= v && v < n);
        if (u == v) return;
        g[u].push_back(v);
        g[v].push_back(u);
    }

    int lca(int u, int v)
    {
        fill(seen.begin(), seen.end(), false);
        while (true)
        {
            u = base[u];
            seen[u] = true;
            if (mate[u] == -1) break;
            u = pre[mate[u]];
        }
        while (!seen[base[v]]) v = pre[mate[base[v]]];
        return base[v];
    }

    void mark(int u, int b, int v)
    {
        while (base[u] != b)
        {
            flower[base[u]] = true;
            flower[base[mate[u]]] = true;
            pre[u] = v;
            v = mate[u];
            u = pre[v];
        }
    }

    bool augment(int root)
    {
        fill(outer.begin(), outer.end(), false);
        fill(pre.begin(), pre.end(), -1);
        iota(base.begin(), base.end(), 0);
        q.assign(1, root);
        outer[root] = true;
        for (int head = 0; head < (int)q.size(); head++)
        {
            int u = q[head];
            for (int v : g[u])
            {
                if (base[u] == base[v] || mate[u] == v) continue;
                if (v == root || (mate[v] != -1 && pre[mate[v]] != -1))
                {
```

```
                    int b = lca(u, v);
                    fill(flower.begin(), flower.end(), false);
                    mark(u, b, v);
                    mark(v, b, u);
                    for (int x = 0; x < n; x++)
                    {
                        if (!flower[base[x]]) continue;
                        base[x] = b;
                        if (!outer[x])
                        {
                            outer[x] = true;
                            q.push_back(x);
                        }
                    }
                }
                else if (pre[v] == -1)
                {
                    pre[v] = u;
                    if (mate[v] == -1)
                    {
                        while (v != -1)
                        {
                            int p = pre[v];
                            int next = mate[p];
                            mate[v] = p;
                            mate[p] = v;
                            v = next;
                        }
                        return true;
                    }
                    int x = mate[v];
                    outer[x] = true;
                    q.push_back(x);
                }
            }
        }
        return false;
    }

    // Rebuild the maximum matching; mate[u] is -1 for an unmatched vertex.
    int solve()
    {
        fill(mate.begin(), mate.end(), -1);
        int answer = 0;
        for (int u = 0; u < n; u++)
        {
            if (mate[u] == -1 && augment(u)) answer++;
        }
        return answer;
    }
};
```

模板题使用：Library Checker general_matching

题意：给简单无向图，输出最大匹配的边数及匹配边。点号 0..n-1；solve 后 mate[u]=-1 表示未匹配，只在 u<mate[u] 时输出，避免同一条边打印两次。

所需模板：Blossom（第 192 页，C.2.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/general_matching

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    Blossom graph(n);
    while (m--)
    {
        int u, v;
        cin >> u >> v;
        graph.add(u, v);
    }
    cout << graph.solve() << '\n';
    for (int u = 0; u < n; u++)
        if (u < graph.mate[u]) cout << u << ' ' << graph.mate[u] << '\n';
}
```

附录 D　矩阵树定理

D.1　任意正模数带权矩阵树

问题描述：给定带权多重图，求所有生成树（无向）或以指定点为根的内向/外向生成树的边权乘积之和，对任意正整数取模。边权为 1 时即生成树个数。

核心思路：矩阵树定理：构造拉普拉斯矩阵 $L = D - A$（有向时 D 取入度或出度），删去根所在的行和列，余子式的行列式就是答案。合数模数下没有逆元，行列式交给按欧几里得辗转相减消元的 `determinant_mod`。

代码依赖：determinant_mod（第 142 页，6.2.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：count(n, edges, root, kind, mod) 返回树的边权乘积之和对 mod 的最小非负余数。mod 为任意正 signed 64 位数，边权支持完整 signed 64 位。Kind 的 undirected、toward_root、away_from_root 分别表示无向、内向和外向树；点号 0..n-1。自环忽略，重边按不同边计数，单点返回 1%mod，mod=1 返回零。先在 128 位中构造模拉普拉斯余子式，再调用 determinant_mod；$O(n^3 \log \mathrm{mod} + m)$ 时间、$O(n^2)$ 额外空间。不使用模逆元，素数与合数模数通用。

```
struct MatrixTreeMod
{
    using ll = long long;
    using Edge = tuple<int, int, ll>;
    enum class Kind
    {
        undirected,
        toward_root,
        away_from_root
    };

    static ll count(int n, const vector<Edge> &edges, int root, Kind kind, ll mod)
    {
        assert(n >= 1 && root >= 0 && root < n && mod >= 1);
        vector<vector<ll>> lap(n - 1, vector<ll>(n - 1));
        auto add = [&](int u, int v, ll w)
        {
            if (u == root) return;
            int i = u - (u > root), j = v - (v > root);
            lap[i][i] = ((__int128)lap[i][i] + w) % mod;
            if (v != root) lap[i][j] = ((__int128)lap[i][j] + mod - w) % mod;
        };
        for (auto [u, v, w] : edges)
        {
            assert(u >= 0 && u < n && v >= 0 && v < n);
            if (u == v) continue;
            w %= mod;
            if (w < 0) w += mod;
            if (kind == Kind::away_from_root) swap(u, v);
            add(u, v, w);
            if (kind == Kind::undirected) add(v, u, w);
        }
        return determinant_mod(move(lap), mod);
    }
};
```

模板题使用：Library Checker Counting Spanning Trees (Undirected)

题意：无向多重图生成树计数，模 998244353；点号 0..n-1，1≤n≤500、0≤m≤500000，允许自环和重边。每条原边权重设 1，自环不参与树，重边是不同选择。删去任意根的拉普拉斯行列，这里取根 0；n=1 的空树计 1，断连计 0。此接口也支持合数模数，复杂度 $O(n^3 \log \mathrm{mod})$，本题固定质数亦可选更快的 MatrixTree 版本。

所需模板：determinant_mod（第 142 页，6.2.1 节）、MatrixTreeMod（第 196 页，D.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/counting_spanning_tree_undirected

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    vector<MatrixTreeMod::Edge> e;
    e.reserve(m);
    for (int i = 0; i < m; i++)
    {
        int u, v;
        cin >> u >> v;
        e.emplace_back(u, v, 1);
    }
    cout << MatrixTreeMod::count(n, e, 0, MatrixTreeMod::Kind::undirected, 998244353)
         << '\n';
}
```

算法索引

AdaptiveSimpson, 169

ballot theorem, 115
basis_intersection, 149
basis_sum_intersection, 151
batch_inverse, 27
Berlekamp–Massey, 153
Berlekamp-Massey, 110
berlekamp_massey, 181
bernoulli, 126
Bezout identity, 24
BigInteger, 1
Binomial, 111
BipartiteMatching, 189
Blossom, 192
BostanMori, 102
Burnside, 135
BurnsideAverage, 132

Carmichael, 26
Catalan, 115
Cayley–Hamilton, 153
cdq_convolution, 71
chirp_z, 82
Chomp, 164
Cipolla, 45
ComplexFFT, 66
conditional expectation, 161
convolution_i64, 76
convolution_mod_fft, 68
coprime pairs, 54
CRT, 15
crt_merge, 31

determinant_mod, 142
Dirichlet convolution, 53
DiscreteLog, 49
divisor sum, 53
Du Jiao sieve, 55
DuJiao, 54

EGF, 112
Euler theorem, 15
Euler totient, 18
euler_phi, 18

euler_power, 25
exCRT, 35
ExLucas, 39
exLucas, 42
extended_gcd, 23

Fenwick, 184
floor moments, 65
floor quotient grouping, 52
floor sum, 64
floor_moments, 62
floor_sum, 60
Floyd, 186
FpsFunctions, 96
FpsInverse, 84
FpsPower, 98
FpsSqrt, 93

Garner, 35
garner, 33
GaussMod, 174
GaussReal, 177
GaussXor, 172
GCD 加权求和, 53
generating functions, 110
geometric series, 53

inclusion-exclusion, 115
indicator variable, 161
integer partitions, 131
inverse_table, 29

Korselt criterion, 26
KthResidue, 51
Kummer theorem, 37

Lagrange, 165
Lagrange interpolation, 42
Lagrange inversion, 109
lagrange_consecutive, 168
Legendre formula, 37
lifting the exponent, 36
linear congruence, 30
linear sieve, 20
linear_congruence, 30
LinearSieve, 20
LTE, 36
Lucas, 38, 42
Lucas theorem, 15

Matrix Tree theorem, 141
matrix_inverse, 139
MatrixTreeMod, 196
MaxPlusMatrix, 141
mex, 162
Miller–Rabin, 17
Min25, 56
mint, 8
Mobius inversion, 53
Mod64, 7
mod_inverse, 26
mod_sqrt, 43
ModInt, 12
ModMatrix, 137
multiplicative order, 48
MultipointEvaluation, 88

necklace_colorings, 134
Nim, 162
Nim-k, 163
NttConvolution, 73
numerical stability, 172

OGF, 112

p-adic valuation, 36
Partitions, 128
parts_at_most, 129
parts_at_most_fps, 130
pentagonal number theorem, 131
permutation_cycles, 133
Pollard–Rho, 17
PollardRho, 21
Polya, 135
polynomial_interpolation, 91
polynomial_shift, 106
PolynomialDivision, 86
PositionBasis, 146
Prime64, 16
PrimitiveRoot, 45

quadratic residue, 45

rank, 141
recurrence_nth, 101
reflection principle, 115
Richardson extrapolation, 172

segmented sieve, 21
segmented_primes, 19

SetConvolution, 78
Simplex, 155
Simpson integration, 172
SJ 定理, 163
Sprague-Grundy, 162
Stirling numbers, 115, 117
stirling_first_column, 120
stirling_first_row, 118
stirling_second_column, 123
stirling_second_row, 121
subset_convolution, 80

tail sum, 161

Widget Factory, 179

XorBasis, 144

上升阶乘, 117
下降阶乘, 117
二元一次不定方程, 24
二分图博弈, 164
二项式反演, 114
互质数对, 53
伯努利数, 126

区间筛素数, 21
十二重计数, 112
升幂引理, 36
原根, 48
反 Nim, 163
复合逆, 109
威佐夫博弈, 162
容斥原理, 114
对称策略, 164
巴什博弈, 162
形式留数, 109

恰好计数, 114
扩展中国剩余定理, 35
批量逆元, 29
排列数, 37
斐波那契博弈, 163
斯特林反演, 125
斯特林数, 117
无序数对, 53
有限状态计数, 153
树上删边, 163
模高斯消元, 179
欧拉数, 127

混合进制, 35
满射, 114

球盒问题, 112
矩形 GCD 计数, 54
策略窃取, 164
筛法选型, 52
类欧几里德, 64
线性同余方程, 30
线性逆元表, 29
线性递推, 104, 153
群作用, 135

自然数幂和, 126
范德蒙德卷积, 131
计数建模, 112
贝尔数, 126

错位排列, 115
阶乘表, 37
阶梯博弈, 163
隐式生成函数, 109